LeetCode Practice

45 Medium: Trees & BST

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

45
MEDIUM

Patterns (in order)

1. Trees & BST (45)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

45 questions | tap a title to jump to it

Medium questions (45) Go >

Trees & BST (45) Go >

M	1. Validate Binary Search Tree	LC 98
M	2. Recover Binary Search Tree	LC 99
M	3. Binary Tree Level Order Traversal	LC 102
M	4. Binary Tree Zigzag Level Order Traversal	LC 103
M	5. Construct Binary Tree from Preorder and Inorder Traversal	LC 105
M	6. Construct Binary Tree from Inorder and Postorder Traversal	LC 106
M	7. Sum Root to Leaf Numbers	LC 129
M	8. Binary Tree Right Side View	LC 199
M	9. Kth Smallest Element in a BST	LC 230
M	10. Lowest Common Ancestor of a Binary Search Tree	LC 235
M	11. Lowest Common Ancestor of a Binary Tree	LC 236
M	12. Count Univalue Subtrees	LC 250
M	13. Inorder Successor in BST	LC 285
M	14. Binary Tree Longest Consecutive Sequence	LC 298
M	15. Binary Tree Vertical Order Traversal	LC 314
M	16. Largest BST Subtree	LC 333
M	17. House Robber III	LC 337
M	18. Find Leaves of Binary Tree	LC 366
M	19. Construct Quad Tree	LC 427
M	20. Path Sum III	LC 437
M	21. Delete Node in a BST	LC 450
M	22. Inorder Successor in BST II	LC 510
M	23. Boundary of Binary Tree	LC 545
M	24. Binary Tree Longest Consecutive Sequence II	LC 549
M	25. Kill Process	LC 582
M	26. Find Duplicate Subtrees	LC 652
M	27. Maximum Width of Binary Tree	LC 662
M	28. Trim a Binary Search Tree	LC 669
M	29. Longest Univalue Path	LC 687
M	30. Employee Importance	LC 690
M	31. Insert into a Binary Search Tree	LC 701
M	32. All Nodes Distance K in Binary Tree	LC 863
M	33. Check Completeness of a Binary Tree	LC 958
M	34. Distribute Coins in Binary Tree	LC 979
M	35. Maximum Difference Between Node and Ancestor	LC 1026
M	36. Delete Nodes And Return Forest	LC 1110
M	37. Maximum Average Subtree	LC 1120
M	38. Delete Leaves With a Given Value	LC 1325
M	39. Time Needed to Inform All Employees	LC 1376
M	40. Balance a Binary Search Tree	LC 1382
M	41. Count Good Nodes in Binary Tree	LC 1448
M	42. Clone N-ary Tree	LC 1490
M	43. Diameter of N-Ary Tree	LC 1522
M	44. Lowest Common Ancestor of a Binary Tree III	LC 1650
M	45. Step-By-Step Directions From a Binary Tree Node to Another	LC 2096

Medium questions (45)

MEDIUM | TOPIC 1 OF 1

Trees & BST

45 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Validate Binary Search Tree LC 98
2. Recover Binary Search Tree LC 99
3. Binary Tree Level Order Traversal LC 102
4. Binary Tree Zigzag Level Order Traversal LC 103
5. Construct Binary Tree from Preorder and Inorder Traversal LC 105
6. Construct Binary Tree from Inorder and Postorder Traversal LC 106
7. Sum Root to Leaf Numbers LC 129
8. Binary Tree Right Side View LC 199
9. Kth Smallest Element in a BST LC 230
10. Lowest Common Ancestor of a Binary Search Tree LC 235
11. Lowest Common Ancestor of a Binary Tree LC 236
12. Count Univalue Subtrees LC 250
13. Inorder Successor in BST LC 285
14. Binary Tree Longest Consecutive Sequence LC 298
15. Binary Tree Vertical Order Traversal LC 314
16. Largest BST Subtree LC 333
17. House Robber III LC 337
18. Find Leaves of Binary Tree LC 366
19. Construct Quad Tree LC 427
20. Path Sum III LC 437
21. Delete Node in a BST LC 450
22. Inorder Successor in BST II LC 510
23. Boundary of Binary Tree LC 545
24. Binary Tree Longest Consecutive Sequence II LC 549
25. Kill Process LC 582
26. Find Duplicate Subtrees LC 652
27. Maximum Width of Binary Tree LC 662
28. Trim a Binary Search Tree LC 669
29. Longest Univalue Path LC 687
30. Employee Importance LC 690
31. Insert into a Binary Search Tree LC 701
32. All Nodes Distance K in Binary Tree LC 863
33. Check Completeness of a Binary Tree LC 958
34. Distribute Coins in Binary Tree LC 979
35. Maximum Difference Between Node and Ancestor LC 1026
36. Delete Nodes And Return Forest LC 1110
37. Maximum Average Subtree LC 1120
38. Delete Leaves With a Given Value LC 1325
39. Time Needed to Inform All Employees LC 1376
40. Balance a Binary Search Tree LC 1382
41. Count Good Nodes in Binary Tree LC 1448
42. Clone N-ary Tree LC 1490
43. Diameter of N-Ary Tree LC 1522
44. Lowest Common Ancestor of a Binary Tree III LC 1650
45. Step-By-Step Directions From a Binary Tree Node to Another LC 2096

MEDIUM | Trees & BST | LeetCode #98 | 1 of 45 | 1/45

Validate Binary Search Tree

QUESTION

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

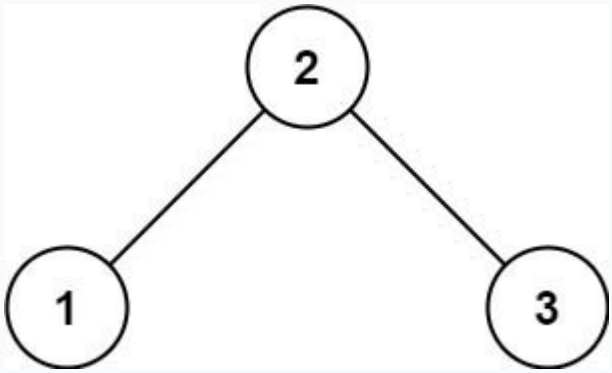
A valid BST is defined as follows:

CONSTRAINTS

- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys strictly greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

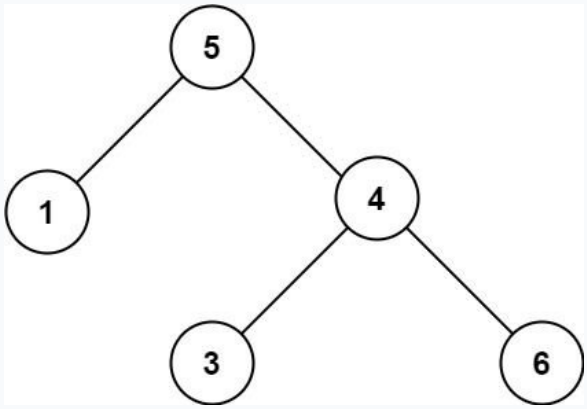
Example 1

Input	`root = [2,1,3]`
Output	`true`



Example 2

Input	`root = [5,1,4,null,null,3,6]`
Output	`false`
Explanation	*The root node's value is 5 but its right child's value is 4.*



- The number of nodes in the tree is in the range [1, 10^4].
- 2^{31} <= Node.val <= 2^{31} - 1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Every node's value must be strictly greater than all values in its left subtree

AND strictly less than all values in its right subtree — not just the immediate parent. Right?

The classic trap: [5,1,4,null,null,3,6] — root=5, right child=4 — 4<5 -> invalid ok"

"[2,1,3]: 1<2<3 -> true ok. [5,1,4,null,null,3,6]: 4<5 -> false ok."

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
list = the in-order traversal of the tree
return the list is strictly increasing
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but stores every value. Passing the allowed (low, high) range down the recursion (or checking against the previous value during traversal) needs only O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (bounds propagation)

Pass min/max bounds down the tree.

Every node must satisfy min_val < node.val < max_val.

Left child: max_val = node.val.

Right child: min_val = node.val.

Use float('-inf') and float('inf') as initial bounds. Use inner helper."

Time: O(log n) — binary search on answer space.

Key insight: passing (min_val, max_val) bounds down the recursion replaces inorder-then-compare — each node is validated in O(1) on entry, giving O(n) total with O(h) space.

PHASE 4 CLEAN CODE

```
class _98_ValidateBST:
    def isValidBST(self, root: Optional['TreeNode']) -> bo
ol:
        def valid(node, min_val, max_val):
            if not node:
                return True
            if not (min_val < node.val < max_val):
                return False
            return (valid(node.left,  min_val,   node.val)
 and
                    valid(node.right, node.val,   max_val)
)
        return valid(root, float('-inf'), float('inf'))
```

PHASE 5 TEST & DEBUG

"Let me trace BST validation on [5,1,4,null,null,3,6]."

```
validate(5, -inf, +inf): val=5 in (-inf,+inf) ok
validate(1, -inf, 5): val=1 in (-inf,5) ok -> left=None
  ok, right=None ok
validate(4, 5, +inf): val=4 NOT > 5 -> return False
```

Returns False. Root says 5>4>3 but 4 is in right subtree -> invalid BST ok

[2,1,3]:

```
validate(2,-inf,+inf)->validate(1,-inf,2): 1<2 ok.
  validate(3,2,+inf): 3>2 ok. -> True ok
```

Edge: [5,4,6,null,null,3,7] — right subtree 6 has left child 3 < 5 (root) -> invalid.

```
validate(6,5,+inf)->validate(3,5,6): 3 NOT > 5 ->
  False ok WHY: bound carries down from ancestor.
```

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** for the call stack

The bounds approach is the correct one — in-order checking works too but requires state.

The common mistake: only checking node vs parent, not the full subtree constraint.

Given more time I'd ask: what if duplicates are allowed (left <= node < right)?

Change the strict inequalities accordingly.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #99 | 2 of 45 | 2/45

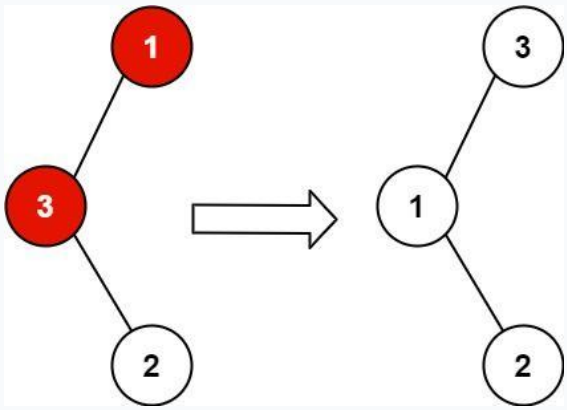
Recover Binary Search Tree

QUESTION

You are given the root of a binary search tree (BST), where the values of exactly two nodes of the tree were swapped by mistake. Recover the tree without changing its structure.

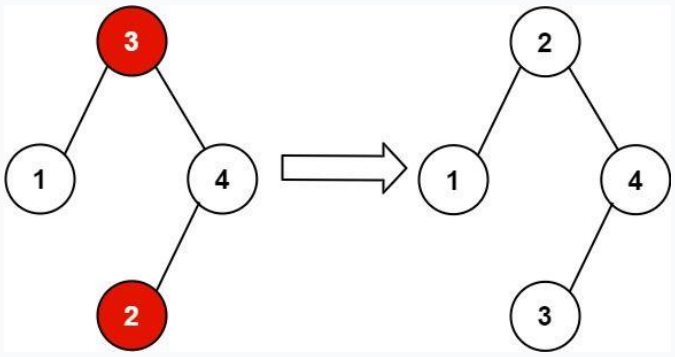
Example 1

Input	`root = [1,3,null,null,2]`
Output	`[3,1,null,null,2]`
Explanation	*3 cannot be a left child of 1 because 3 > 1. Swapping 1 and 3 makes the BST valid.*



Example 2

Input	`root = [3,1,4,null,null,2]`
Output	`[2,1,4,null,null,3]`
Explanation	*2 cannot be in the right subtree of 3 because 2 < 3. Swapping 2 and 3 makes the BST valid.*



CONSTRAINTS

- The number of nodes in the tree is in the range [2, 1000].
- 2^{31} <= Node.val <= 2^{31} - 1
- Follow up: A solution using O(n) space is pretty straight-forward. Could you devise a constant O(1) space solution?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Exactly two nodes in a BST are swapped. Recover the BST without changing structure. Right?"

"A few quick questions:

Modify in-place (swap values)? Yes. O(1) space is the follow-up? Yes."

"Let me walk: root=[1,3,null,null,2] (3 and 1 swapped) -> [3,1,null,null,2] -> recover to [1,3,null,null,2] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
list = the in-order traversal values
find the two values out of order
swap those two nodes' values
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but stores the whole traversal. Remembering only the previous node during traversal (or Morris traversal) finds the two swapped nodes in O(n) time and O(h) or O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is O(n) extra storage.

Morris traversal-like inorder: track prev node.

First mistake: prev.val > curr.val -> first = prev.

Second mistake: again prev.val > curr.val -> second = curr.

Swap first.val and second.val at the end. O(n) time, O(1) space."

Key insight: an inorder BST traversal produces a sorted sequence — exactly one descent (prev > curr) marks the first swapped node, another marks the second.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _99_RecoverBST:
    def recoverTree(self, root):
        first = second = prev = None
        def inorder(node):
            nonlocal first, second, prev
            if not node: return
            inorder(node.left)
            if prev and prev.val > node.val:
                if not first: first = prev
                second = node
            prev = node
            inorder(node.right)
```

```
        inorder(root)
        first.val, second.val = second.val, first.val
```

PHASE 5 TEST & DEBUG

"Let me trace the tree recursion on the example, showing the return value at each node."

root=[3,1,null,null,2]: inorder visits 1,2,3 in correct order? Actually root=3,left=1,1.right=2.

inorder: 1,2,3. prev=1,curr=2: ok. prev=2,curr=3: ok. No swaps — but expected to find issue.

Actually the swapped tree is root=[1,3,null,null,2] where 1 and 3 are swapped.

inorder of that: visit 3(left of 1? No—1 is root,3 is left child,3 has right child 2.

inorder: 3,2,1 — wait that's decreasing? Hmm.

Actually root=[1,3,null,null,2]: root=1, root.left=3, 3.right=2.

inorder visits: 3,2,1. prev=3,curr=2: 3>2->first=3,second=2. prev=2,curr=1: 2>1->second=1. swap 3 and 1. ok

Two-pointer inorder finds first and second swapped nodes correctly

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** recursion stack. Morris: O(1)

Follow-up: if three nodes are swapped — more complex logic needed.

Follow-up: Validate BST (#98) — check validity without recovering.

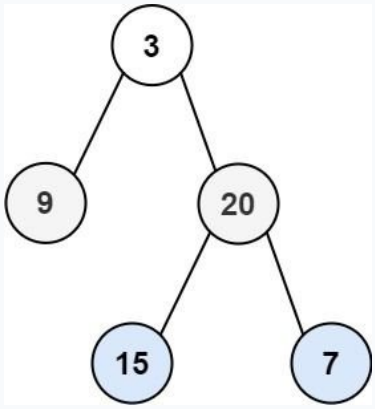
MEDIUM | Trees & BST | LeetCode #102 | 3 of 45 | 3/45

Binary Tree Level Order Traversal

QUESTION

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1	
Input	`root = [3,9,20,null,null,15,7]`
Output	`[[3],[9,20],[15,7]]`



Example 2	
Input	`root = [1]`
Output	`[[1]]`

Example 3	
Input	`root = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 2000].
- 1000 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return all node values level by level in a list of lists. Right?"

"A few quick questions:

Empty tree -> []? Yes. Single node -> [[root.val]]? Yes."

"Let me walk: root=[3,9,20,null,null,15,7] -> [[3],[9,20],[15,7]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for depth d = 0, 1, 2 ...:
    run a DFS over the whole tree and collect the
        nodes at depth d
    if a depth is empty:
        break
```

TIME	SPACE
O(n * h)	O(n)

WHY IT IS SLOW

Rescans the whole tree once per level, O(n^2) for a skewed tree. A queue-based BFS that processes one level per round visits each node once: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS level-snapshot is O(n) time — already optimal. Let me walk through"

exactly how it builds the output level by level so I can narrate it in an interview.

Step 1 — Seed: put root in the queue. result = [].

Step 2 — Level loop: while queue is non-empty:

- snapshot = len(queue) <- records how many nodes are at THIS depth.

- level = []

- Pop exactly 'snapshot' nodes (not more — children enqueued during this loop

are the NEXT level, so we stop at the snapshot count).

- For each popped node: append node.val to level; enqueue left & right children if non-null.

- Append level to result.

WHY snapshot: without it, adding children mid-loop would mix depths into the same level.

Time: O(n) — each node enqueued and dequeued once. Space: O(w) — max queue width.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import deque
class _102_BinaryTreeLevelOrderTraversal:
    def levelOrder(self, root):
        if not root:
            return []
        result = []
        queue = deque([root])
        while queue:
            level_size = len(queue)
            level = []
            for _ in range(level_size):
                node = queue.popleft()
                level.append(node.val)
                if node.left: queue.append(node.left)
                if node.right: queue.append(node.right)
```

```
            result.append(level)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace level-order BFS on [3,9,20,null,null,15,7]."

queue=[3]. Snapshot len=1.

Round 1: pop 3 -> level=[3]. Enqueue 9,20. queue=[9,20].

Round 2: snapshot len=2. Pop 9 -> level=[9], no children. Pop 20 -> level=[9,20], enqueue 15,7.

Round 3: snapshot len=2. Pop 15 -> level=[15], no children. Pop 7 -> level=[15,7], no children.

Result: [[3],[9,20],[15,7]] ok

Edge: root=None -> queue starts empty -> return [] ok

WHY snapshot len(queue) BEFORE loop: adding children during the loop would mix levels.

Snapshot isolates exactly the nodes at the current level.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(w)** max queue size = max tree width

Variations: Zigzag (#103) — alternate direction; Right Side View (#199) — take last per level.

Follow-up: minimum depth — BFS stops at first leaf found.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #103 | 4 of 45 | 4/45

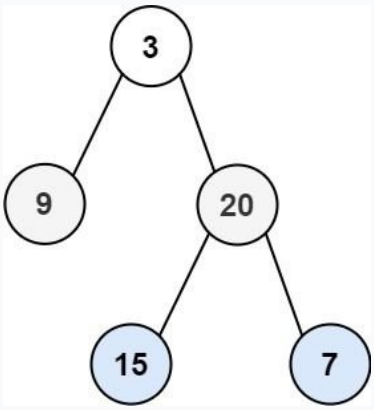
Binary Tree Zigzag Level Order Traversal

QUESTION

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1

Input	`root = [3,9,20,null,null,15,7]`
Output	`[[3],[20,9],[15,7]]`



Example 2

Input	`root = [1]`
Output	`[[1]]`

Example 3

Input	`root = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 2000].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Level order traversal but alternating direction: odd levels left-to-right,

even levels right-to-left (or vice versa — confirm with example).

Root is level 0 -> left-to-right. Level 1 -> right-to-left. Right?"

"[3,9,20,null,null,15,7] -> [[3],[20,9],[15,7]] ok (level 1 reversed)"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
levels = a normal level-order traversal
reverse every level with odd index
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Same O(n) time as the optimal answer, plus an extra pass copying the reversed levels. Reversing as you go, with a left-to-right flag or a deque, avoids the extra pass.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Same BFS snapshot as #102 plus a direction flag. Already O(n)."

Walk-through of how direction is applied:

`left_to_right = True initially.`

Each level: collect values normally into a list.

THEN: if not left_to_right -> reverse the list before appending.

Toggle left_to_right = not left_to_right after each level.

Alternative: use deque and appendleft vs append based on direction — avoids the reverse cost.

Time: O(n). Space: O(w) BFS queue + O(w) each level list.

PHASE 4 CLEAN CODE

```
class _103_ZigzagLevelOrder:
    def zigzagLevelOrder(self, root: Optional['TreeNode'])
 -> List[List[int]]:
        if not root:
            return []
        result        = []
        queue         = deque([root])
        left_to_right = True
        while queue:
            level_size = len(queue)
            level_vals = []
            for _ in range(level_size):
                node = queue.popleft()
                level_vals.append(node.val)
                if node.left:  queue.append(node.left)
                if node.right: queue.append(node.right)
            result.append(level_vals if left_to_right else
 level_vals[::-1])
            left_to_right = not left_to_right
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace zigzag level order on [3,9,20,null,null,15,7]."

Level 0 (L->R): pop 3 -> [3]. Enqueue 9,20. Direction flips to R->L.

Level 1 (R->L): pop 9,20 -> add to front: deque=[9,20] reversed -> [20,9]. Enqueue children 15,7.

PHASE 5 TEST & DEBUG

Level 2 (L->R): pop 15,7 -> add normally -> [15,7]. Direction flips.

Result: [[3],[20,9],[15,7]] ok

WHY deque for zigzag: appendleft for R->L direction inserts at front -> reverses order.

Alternatively: append normally then reverse every other level list.

Edge: single root -> [[root.val]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(w)

This is #102 with one added line: the conditional reverse.

Name that connection — it shows you see the pattern family.

Given more time I'd ask: what if alternation starts at level 1 instead of 0?

Just flip the initial value of left_to_right.

MEDIUM | Trees & BST | LeetCode #105 | 5 of 45 | 5/45

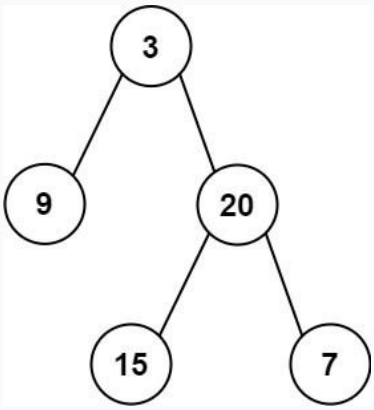
Construct Binary Tree from Preorder and Inorder Traversal

QUESTION

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Example 1

Input	`preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]`
Output	`[3,9,20,null,null,15,7]`



Example 2

Input	`preorder = [-1], inorder = [-1]`
Output	`[-1]`

CONSTRAINTS

- 1 <= preorder.length <= 3000
- inorder.length == preorder.length
- 3000 <= preorder[i], inorder[i] <= 3000
- preorder and inorder consist of unique values.
- Each value of inorder also appears in preorder.
- preorder is guaranteed to be the preorder traversal of the tree.
- inorder is guaranteed to be the inorder traversal of the tree.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Preorder: root first, then left subtree, then right subtree.

Inorder: left subtree, root, right subtree.

All values are unique — so we can find the root's position in inorder uniquely. Right?"

"preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]:

Root=3. In inorder, 3 is at index 1. Left subtree=[9], right subtree=[15,20,7].

preorder left=[9], right=[20,15,7]. Recurse."

PHASE 1 CLARIFY

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def build(pre, in):
    if empty:
        return nothing
    root = pre[0]
    i = position of root in in (linear search)
    left = build(pre[1:i + 1], in[:i])
    right = build(pre[i + 1:], in[i + 1:])
    return the node
```

TIME	SPACE
O(n^2)	O(n^2)

WHY IT IS SLOW

The linear search for the root and the array slices each cost O(n) per node, O(n^2) for a skewed tree. A hash map from value to inorder index plus index bounds is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Use preorder[0] as root; split inorder with index map — O(n). Walk-through:"

Build inorder_map: value -> index (for O(1) lookup).

`build(pre_start, pre_end, in_start, in_end):`

If pre_start > pre_end: return None.

`root_val = preorder[pre_start]. root = TreeNode(root_val).`

`mid = inorder_map[root_val].`

`left_size = mid - in_start.`

`root.left = build(pre_start+1, pre_start+left_size, in_start, mid-1).`

`root.right = build(pre_start+left_size+1, pre_end, mid+1, in_end).`

`return root.`

WHY preorder[0] is root: preorder visits root FIRST.

WHY inorder split at mid: left of mid = left subtree (in_start..mid-1), right = right subtree (mid+1..in_end).

WHY left_size: preorder's left subtree occupies next left_size positions.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _105_BuildTree:
    def buildTree(self, preorder: List[int], inorder: List
[int]) -> Optional['TreeNode']:
        in_idx = {val: i for i, val in enumerate(inorder)}
        pre    = iter(preorder)

        def build(left: int, right: int) -> Optional['Tree
Node']:
            if left > right:
                return None
            root_val = next(pre)
            root     = TreeNode(root_val)
            mid      = in_idx[root_val]
            root.left  = build(left, mid - 1)
            root.right = build(mid + 1, right)
            return root

        return build(0, len(inorder) - 1)
```

PHASE 5 TEST & DEBUG

"Let me trace construction from preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]."

preorder[0]=3 -> root. inorder.index(3)=1. left_size=1.

Left call: pre[1:2]=[9], in[0:1]=[9] -> preorder[0]=9->root. No children -> Node(9).

Right call: pre[2:5]=[20,15,7], in[2:5]=[15,20,7]:

pre[0]=20->root. inorder.index(20)=1. left_size=1.

Left: pre[1:2]=[15],in[0:1]=[15]->Node(15).

Right: pre[2:3]=[7],in[2:3]=[7]->Node(7).

Tree: 3->left=9, right=20(left=15,right=7) ok

WHY preorder[0]=root, inorder split: pre gives root first; in tells where to split L vs R.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for the hash map and call stack

Using iter(preorder) as a lazy pointer is cleaner than an index variable.

The key insight: preorder always gives the root first, inorder tells us how many

elements are in each subtree — that's the only information we need to recurse.

Given more time I'd ask: construct from postorder and inorder?

Same logic — just consume postorder from the RIGHT end, and build right subtree first.

MEDIUM | Trees & BST | LeetCode #106 | 6 of 45 | 6/45

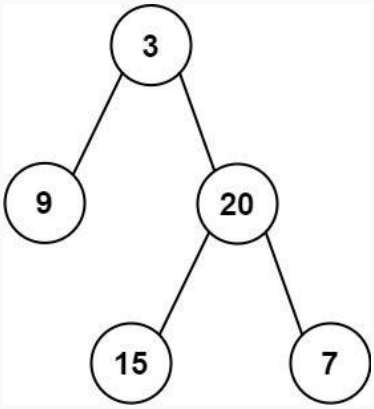
Construct Binary Tree from Inorder and Postorder Traversal

QUESTION

Given two integer arrays inorder and postorder where inorder is the inorder traversal of a binary tree and postorder is the postorder traversal of the same tree, construct and return the binary tree.

Example 1

Input	`inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]`
Output	`[3,9,20,null,null,15,7]`



Example 2

Input	`inorder = [-1], postorder = [-1]`
Output	`[-1]`

CONSTRAINTS

- 1 <= inorder.length <= 3000
- postorder.length == inorder.length
- 3000 <= inorder[i], postorder[i] <= 3000
- inorder and postorder consist of unique values.
- Each value of postorder also appears in inorder.
- inorder is guaranteed to be the inorder traversal of the tree.
- postorder is guaranteed to be the postorder traversal of the tree.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given inorder and postorder traversals of a binary tree, reconstruct it.

Inorder: left,root,right. Postorder: left,right,root. Right?"

"A few quick questions:

All values unique? Yes — needed to locate root in inorder. No return value — return root? Yes."

PHASE 1 CLARIFY

"Let me walk: inorder=[9,3,15,20,7], postorder=[9,15,7,20,3] -> root=3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def build(in, post):
    if empty:
        return nothing
    root = post[-1]
    i = position of root in in (linear search)
    left = build(in[:i], post[:i])
    right = build(in[i + 1:], post[i:-1])
    return the node
```

TIME	SPACE
O(n^2)	O(n^2)

WHY IT IS SLOW

Linear search and array slices cost O(n) per node. A hash map of inorder positions and index bounds (consuming postorder from the back) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Use postorder[-1] as root; split inorder — O(n). Walk-through:"

Build inorder_map: value -> index.

`build(post_start, post_end, in_start, in_end):`

If post_start > post_end: return None.

`root_val = postorder[post_end]. root = TreeNode(root_val).`

`mid = inorder_map[root_val].`

`left_size = mid - in_start.`

`root.left = build(post_start, post_start+left_size-1, in_start, mid-1).`

`root.right = build(post_start+left_size, post_end-1, mid+1, in_end).`

`return root.`

WHY postorder[-1] is root: postorder visits root LAST.

WHY inorder split: left of mid in inorder = left subtree; right of mid = right subtree.

WHY post_start+left_size-1: postorder left subtree has left_size nodes starting from post_start.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _106_ConstructBinaryTreeInorderPostorder:
    def buildTree(self, inorder, postorder):
        idx_map = {val: i for i, val in enumerate(inorder)
```

```
}
        self_post_idx = [len(postorder) - 1]
        def build(in_left, in_right):
            if in_left > in_right:
                return None
            root_val = postorder[self_post_idx[0]]
            self_post_idx[0] -= 1
            root = TreeNode(root_val)
            mid = idx_map[root_val]
            root.right = build(mid + 1, in_right)
            root.left = build(in_left, mid - 1)
            return root
        return build(0, len(inorder) - 1)
```

PHASE 5 TEST & DEBUG

"Let me trace from inorder=[9,3,15,20,7], postorder=[9,15,7,20,3]."

postorder[-1]=3->root. inorder.index(3)=1. left_size=1.

Left: post[0:1]=[9], in[0:1]=[9]->Node(9).

Right: post[1:4]=[15,7,20], in[2:5]=[15,20,7]:

post[-1]=20->root. in.index(20)=1 (in this slice). left_size=1.

Left: post[0:1]=[15], in[0:1]=[15]->Node(15).

Right: post[1:2]=[7], in[2:3]=[7]->Node(7).

Final tree: 3(9, 20(15,7)) ok

WHY postorder[-1]=root: post-order visits root LAST. Split in-order at root -> left/right sizes.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for hashmap + call stack

The right-before-left recursion is the key difference from #105 (preorder+inorder).

Follow-up: Construct Binary Tree from Preorder and Inorder (#105) — left before right.

Follow-up: reconstruct from preorder and postorder — only possible for full binary trees.

MEDIUM | Trees & BST | LeetCode #129 | 7 of 45 | 7/45

Sum Root to Leaf Numbers

QUESTION

You are given the root of a binary tree containing digits from 0 to 9 only.

Each root-to-leaf path in the tree represents a number.

CONSTRAINTS

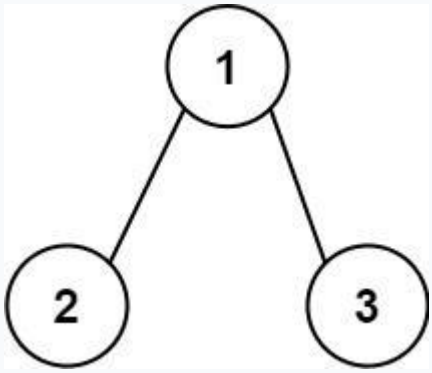
- For example, the root-to-leaf path 1 -> 2 -> 3 represents the number 123.

Return the total sum of all root-to-leaf numbers. Test cases are generated so that the answer will fit in a 32-bit integer.

A leaf node is a node with no children.

Example 1

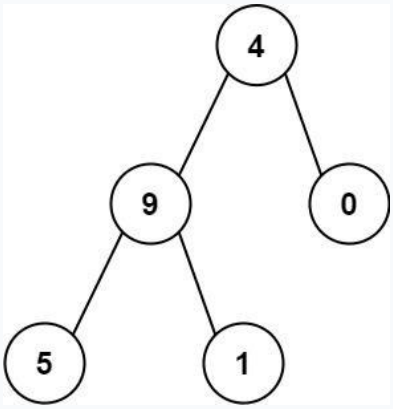
Input	`root = [1,2,3]`
Output	`25`
Explanation	*The root-to-leaf path 1->2 represents the number 12.* *The root-to-leaf path 1->3 represents the number 13.* *Therefore, sum = 12 + 13 = 25.*



Example 2

Input	`root = [4,9,0,5,1]`
Output	`1026`
Explanation	*The root-to-leaf path 4->9->5 represents the number 495.* *The root-to-leaf path 4->9->1 represents the number 491.* *The root-to-leaf path 4->0 represents the number 40.* *Therefore, sum = 495 + 491 + 40 = 1026.*

QUESTION



- The number of nodes in the tree is in the range [1, 1000].
- 0 <= Node.val <= 9
- The depth of the tree will not exceed 10.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each root-to-leaf path represents a number (root digit is most significant).

Return the total sum of all these numbers. Right?"

"A few quick questions:

Values are 0-9 (single digits)? Yes. Empty tree -> 0? Yes."

"Let me walk: root=[1,2,3] -> paths: 12 + 13 = 25 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
collect every root-to-leaf path as a string of digits
convert each to a number and add them up
```

TIME	SPACE
O(n * h)	O(n * h)

WHY IT IS SLOW

Builds and parses a string per path. Passing value * 10 + digit down the recursion and adding it at each leaf is O(n) time and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Accumulate the number top-down — already O(n). Walk-through:"

```
dfs(node, current_number):
current_number = current_number * 10 + node.val
```

<- each level shifts the existing digits left (×10) and appends current digit.

PHASE 3 OPTIMIZE

If leaf (no children): return current_number <- this root-to-leaf number is complete.

`total = 0`

`if node.left: total += dfs(node.left, current_number)`

`if node.right: total += dfs(node.right, current_number)`

`return total <- sum up all paths through this node.`

How numbers accumulate: root=1 -> current=1. Left child=2 -> current=12. Its left=3 -> current=123.

That leaf returns 123. Back at 12: right=5 -> current=125. Leaf returns 125. Sum = 248.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _129_SumRootToLeafNumbers:
    def sumNumbers(self, root):
        def dfs(node, running_sum):
            if not node:
                return 0
            running_sum = running_sum * 10 + node.val
            if not node.left and not node.right:
                return running_sum
            return dfs(node.left, running_sum) + dfs(node.
right, running_sum)
        return dfs(root, 0)
```

PHASE 5 TEST & DEBUG

"Let me trace sumNumbers on [1,2,3]."

`dfs(1, 0): curr=0*10+1=1. Left: dfs(2, 1): curr=1*10+2=12. 2 is leaf->total+=12=12.`

Right: dfs(3, 1): curr=1*10+3=13. 3 is leaf->total+=13=13. total=25 ok

[4,9,0,5,1]:

`dfs(4,0)=4: dfs(9,4)=49: dfs(5,49)=495(leaf)->+495. dfs(1,49)=491(leaf)->+491.`

`dfs(0,4)=40(leaf)->+40. total=495+491+40=1026 ok`

WHY curr=parent*10+node.val: builds the number digit by digit (decimal place shift).

Edge: single node [5]-> dfs(5,0): curr=5, leaf->total=5 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** recursion stack

The running_sum parameter propagates the accumulated number without extra storage.

Follow-up: Binary Tree Paths (#257) — same DFS, return strings instead of accumulating.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: if values > 9, must handle multi-digit — multiply by appropriate power.

MEDIUM | Trees & BST | LeetCode #199 | 8 of 45 | 8/45

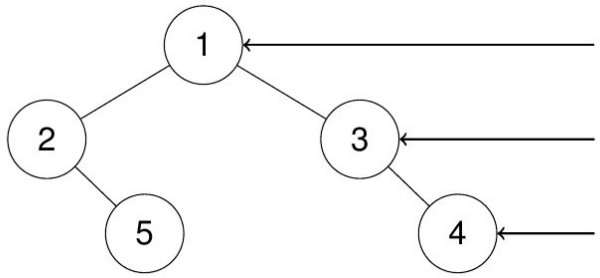
Binary Tree Right Side View

QUESTION

Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

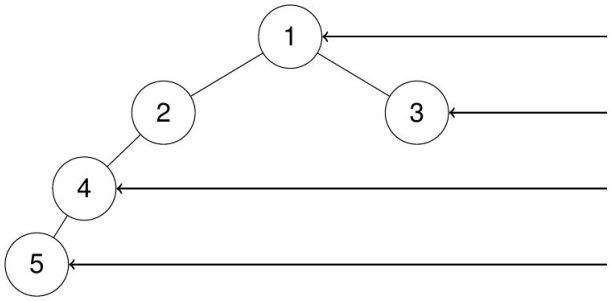
Example 1

Input	`root = [1,2,3,null,5,null,4]`
Output	`[1,3,4]`



Example 2

Input	`root = [1,2,3,4,null,null,null,5]`
Output	`[1,3,4,5]`



Example 3

Input	`root = [1,null,3]`
Output	`[1,3]`

Example 4

Input	`root = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 100].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Imagine looking at the tree from the right. You see one node per level —

the rightmost node visible at that level. Return top to bottom. Right?"

"[1,2,3,null,5,null,4] -> [1,3,4] ok (3 is rightmost at level 1, 4 at level 2)"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
levels = a level-order traversal collecting every
    level's values
return the last value of each level
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Stores every value of every level. A BFS that only records the last node of each level, or a DFS visiting right first and noting the first node per depth, needs less memory.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS level-snapshot — same as #102, just take the LAST element per level. Already O(n)."

Walk-through:

For each BFS level, collect all values into a list.

The rightmost visible node = level[-1] (last element, since BFS processes left->right).

Append level[-1] to result.

Alternative DFS approach: DFS preferring RIGHT child first.

When depth == len(result): this is the first node seen at this depth -> it's rightmost.

Append node.val; recurse right then left.

Both are O(n) time, O(h) space for DFS or O(w) space for BFS.

PHASE 4 CLEAN CODE

```python
class _199_RightSideView:
    def rightSideView(self, root: Optional['TreeNode']) ->
 List[int]:
        if not root:
            return []
        result = []
        queue  = deque([root])
        while queue:
            level_size = len(queue)
            for i in range(level_size):
                node = queue.popleft()
```

```
                if i == level_size - 1:
                    result.append(node.val)     # rightmos
t node of this level
                if node.left:  queue.append(node.left)
                if node.right: queue.append(node.right)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace rightSideView on [1,2,3,null,5,null,4]."

BFS level-order:

Level 0: [1] -> rightmost=1. Enqueue 2,3.

Level 1: [2,3] -> rightmost=3. Enqueue 5(from 2), 4(from 3).

Level 2: [5,4] -> rightmost=4. No more children.

Result: [1,3,4] ok

WHY last element per level = right side view: rightmost node visible from right side.

DFS alternative: traverse root->right->left. Track depth. First visit at each depth = right side.

Edge: root=None->[] ok. Skewed tree [1,2,3,4]: each level has one node -> [1,2,3,4] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(w)** where w = max width

This is #102 with one change: only record the last element per level.

Left side view: record the first element per level instead.

Given more time I'd ask: what if some right-side nodes are blocked by deeper left-side nodes?

The problem says 'visible from the right' — the rightmost node at each DEPTH level,

which BFS naturally gives us. A right child at depth 2 can block a left child at depth 2.

MEDIUM | Trees & BST | LeetCode #230 | 9 of 45 | 9/45

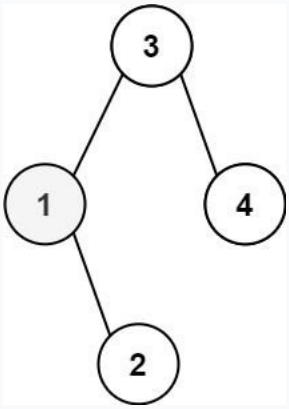
Kth Smallest Element in a BST

QUESTION

Given the root of a binary search tree, and an integer k, return the kth smallest value (1-indexed) of all the values of the nodes in the tree.

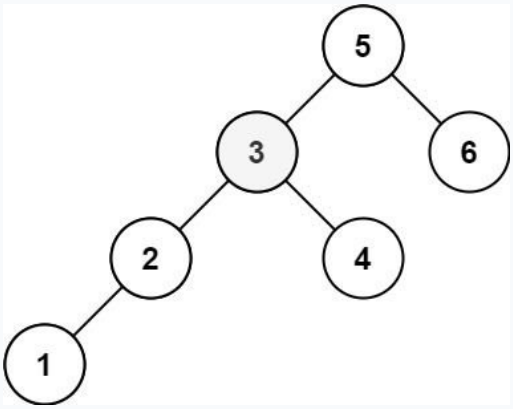
Example 1

Input	`root = [3,1,4,null,2], k = 1`
Output	`1`



Example 2

Input	`root = [5,3,6,2,4,null,null,1], k = 3`
Output	`3`



CONSTRAINTS

- The number of nodes in the tree is n.
- $1 <= k <= n <= 10^4$
- $0 <= Node.val <= 10^4$
- Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

BST in-order traversal (left -> root -> right) visits nodes in ascending sorted order.

Return the k-th element in this order (1-indexed). Right?"

"[3,1,4,null,2], k=1: in-order=[1,2,3,4]. 1st smallest=1 ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = the in-order traversal of the whole tree
return values[k - 1]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Traverses and stores every node. An iterative in-order traversal that stops at the k-th node is O(h + k) time and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"In-order DFS stopping at k-th node — O(k) time, no extra array. Walk-through:"

In-order traversal visits BST nodes in sorted (ascending) order: left -> root -> right.

HOW to stop early: use a counter and result in a mutable container (list/nonlocal).

```
inorder(node):
if node is None or counter[0] == 0: return
inorder(node.left) <- go all the way left (smallest
  values first)
counter[0] -= 1
if counter[0] == 0: <- this is the k-th smallest
result[0] = node.val; return
inorder(node.right) <- only recurse right if still
  looking
```

WHY this visits nodes in sorted order: the BST property guarantees

left subtree < root < right subtree at every node.

Time: O(k + h) where h = height. Space: O(h) recursion.

PHASE 4 CLEAN CODE

```
class _230_KthSmallestBST:
    def kthSmallest(self, root: Optional['TreeNode'], k: i
nt) -> int:
        count  = [0]
        result = [0]

        def inorder(node):
```

```
            if not node or count[0] == k:
                return
            inorder(node.left)
            count[0] += 1
            if count[0] == k:
                result[0] = node.val
                return
            inorder(node.right)

        inorder(root)
        return result[0]
```

PHASE 5 TEST & DEBUG

"Let me trace kthSmallest on BST [3,1,4,null,2], k=1."

In-order traversal (Left->Node->Right = ascending):

stack=[]. curr=3. Push 3, go left->1. Push 1, go left->None.

Pop 1: k--=0? No, k=1->count=1, k=1==1 -> return 1 ok

k=3: count=1(node 1), count=2(node 2), count=3(node 3) -> return 3 ok

WHY BST in-order = sorted: BST property -> left < root < right -> in-order ascending.

Iterative stack: push while going left, pop to visit, go right. Count visits.

Edge: k=1 returns minimum element (leftmost leaf) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k + h) at most k steps after reaching the leftmost leaf	**O(h)**

Follow-up: frequent inserts/deletes + frequent kth queries?

Augment each BST node with the size of its left subtree.

Then kth smallest is an O(log n) descent: if left_size >= k go left; if left_size+1==k return root; else k -= left_size+1, go right.

Given more time I'd ask: what if we want the kth largest?

Reverse in-order (right->root->left) and count k steps.

MEDIUM | Trees & BST | LeetCode #235 | 10 of 45 | 10/45

Lowest Common Ancestor of a Binary Search Tree

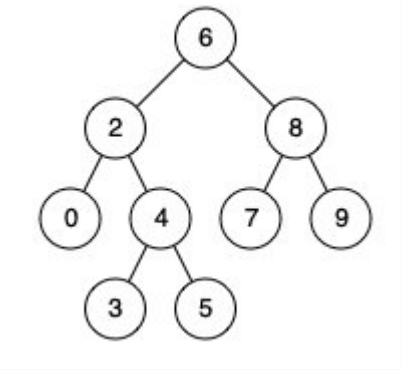
QUESTION

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

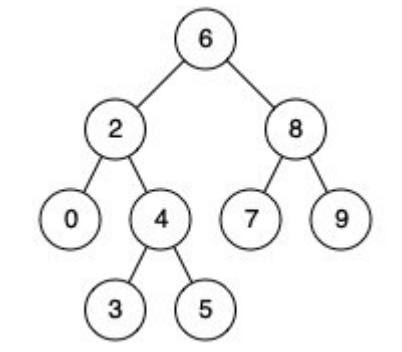
Example 1

Input	`root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8`
Output	**6**
Explanation	*The LCA of nodes 2 and 8 is 6.*



Example 2

Input	`root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4`
Output	**2**
Explanation	*The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.*



QUESTION

Example 3

Input	`root = [2,1], p = 2, q = 1`
Output	**2**

CONSTRAINTS

- The number of nodes in the tree is in the range [2, 10^5].
- 10^9 <= Node.val <= 10^9
- All Node.val are unique.
- p != q
- p and q will exist in the BST.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

In a BST, find the LCA (Lowest Common Ancestor) of nodes p and q.

LCA = the deepest node that is an ancestor of both p and q (a node is its own ancestor). Right?

Use the BST property: values to the left < root < values to the right."

"[6,2,8,0,4,7,9], p=2, q=8 -> LCA=6 (they're on different sides of 6) ok

p=2, q=4 -> LCA=2 (4 is in 2's right subtree) ok"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

“Let me start with brute force to lock in the logic.”

BRUTE FORCE · PSEUDO-CODE

```
path_p = the nodes from the root to p
path_q = the same for q
answer = the last node where the two paths agree
```

TIME	SPACE
O(h)	O(h)

WHY IT IS SLOW

Ignores the BST ordering. Walk down from the root: go left if both targets are smaller, right if both are larger, else this is the split node: O(h) time and O(1) space.

“Should I go ahead and optimize?”

PHASE 3 OPTIMIZE (BST property)

Approach: optimal approach

"Navigate using BST property:

If both p and q are LESS than root: LCA is in the LEFT subtree.

If both are GREATER: LCA is in the RIGHT subtree.

Otherwise (they split at root, or one equals root): root IS the LCA.

PHASE 3 OPTIMIZE (BST property)

O(h) time — O(log n) for balanced BST, O(n) worst case."

Key insight: the BST ordering property navigates directly to the LCA — both nodes < root -> go left; both > root -> go right; otherwise root is the LCA.

PHASE 4 CLEAN CODE

```
class _235_LCA_BST:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'T
reeNode', q: 'TreeNode') -> 'TreeNode':
        while root:
            if p.val < root.val and q.val < root.val:
                root = root.left
            elif p.val > root.val and q.val > root.val:
                root = root.right
            else:
                return root
        return root
```

PHASE 5 TEST & DEBUG

"Let me trace LCA of BST [6,2,8,0,4,7,9,null,null,3,5], p=2,q=8."

At node 6: p.val=2 < 6 < 8=q.val -> LCA is 6 ok (p and q are in different subtrees)

p=2, q=4:

At 6: both < 6 -> go left to 2.

At 2: p=2==node -> LCA is 2 ok (q=4 is in 2's right subtree)

WHY split-point is LCA: when p and q are on different sides of node, node is their LCA.

When both < node: both in left subtree, recurse left.

When both > node: both in right subtree, recurse right.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(h)	**O(1)** iterative, no stack

The BST property makes this an O(h) walk — no need to visit all nodes.

This is much faster than the general binary tree LCA (#236) which needs DFS.

Given more time I'd ask: what if p or q might not exist in the tree?

We'd need to verify existence first — two separate searches, then LCA.

MEDIUM | Trees & BST | LeetCode #236 | 11 of 45 | 11/45

Lowest Common Ancestor of a Binary Tree

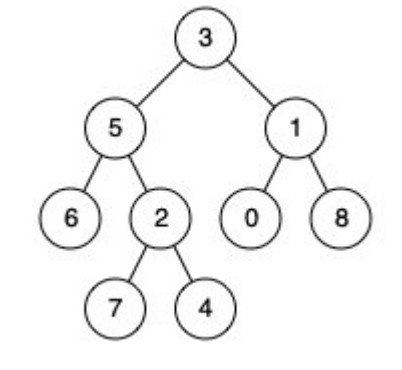
QUESTION

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

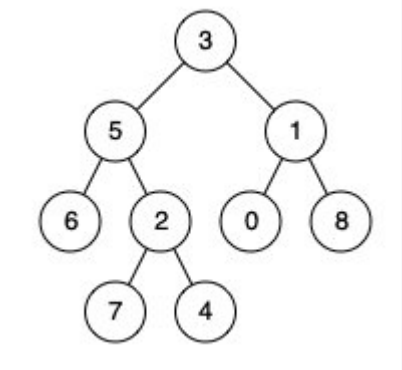
Example 1

Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1`
Output	`3`
Explanation	*The LCA of nodes 5 and 1 is 3.*



Example 2

Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4`
Output	`5`
Explanation	*The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.*



QUESTION

Example 3

Input	`root = [1,2], p = 1, q = 2`
Output	`1`

CONSTRAINTS

- The number of nodes in the tree is in the range [2, 10^5].
- 10^9 <= Node.val <= 10^9
- All Node.val are unique.
- p != q
- p and q will exist in the tree.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

General binary tree — no BST property. Find the LCA (Lowest Common Ancestor) of p and q.

p and q are guaranteed to exist in the tree. A node is its own ancestor. Right?"

"[3,5,1,6,2,0,8,null,null,7,4], p=5, q=1 -> LCA=3 (root, both are in different subtrees) ok

p=5, q=4 -> LCA=5 (4 is in 5's subtree) ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

“Let me start with brute force to lock in the logic.”

BRUTE FORCE · PSEUDO-CODE

```
path_p = the nodes from the root to p
path_q = the same for q
answer = the last node where the two paths are equal
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Two full searches and two stored paths. One post-order recursion that returns a node when it finds p or q, and reports the node where both sides report a hit, needs O(h) space.

“Should I go ahead and optimize?”

PHASE 3 OPTIMIZE (recursive post-order)

“Post-order DFS returning LCA upward — O(n). Walk-through:”

`def lca(node, p, q):`

If node is None or node==p or node==q: return node.

`left = lca(node.left, p, q).`

`right = lca(node.right, p, q).`

If left and right: return node. <- p found in left subtree, q in right (or vice versa)

`return left or right. <- only one side found something`

PHASE 3 OPTIMIZE (recursive post-order)

HOW result bubbles up: if p and q are in different subtrees -> their common ancestor returns itself.

If both are in the same subtree -> the deeper LCA bubbles up through the other subtree returning None.

WHY return node if node==p: p itself could be the LCA (if q is in p's subtree).

Time: O(n). Space: O(h) call stack.

PHASE 4 CLEAN CODE

```
class _236_LCA_BinaryTree:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'T
reeNode', q: 'TreeNode') -> 'TreeNode':
        def lca(node):
            if not node or node is p or node is q:
                return node
            left  = lca(node.left)
            right = lca(node.right)
            if left and right:
                return node         # p and q are in diffe
rent subtrees
            return left or right    # both in same subtree
 — propagate the found node

        return lca(root)
```

PHASE 5 TEST & DEBUG

"Let me trace LCA on [3,5,1,6,2,0,8,null,null,7,4], p=5,q=1."

lca(3,5,1): lca(5,5,1): node==p -> return 5.
lca(1,5,1): node==q -> return 1.

At node 3: left=5(found), right=1(found). Both non-None -> return 3 ok

p=5,q=4:

lca(3): lca(5): node==p -> return 5 (don't descend further — p could be LCA if q is below p).

lca(1): lca(None)=None × 2 -> None. Neither child found -> lca(1)=None.

At 3: left=5, right=None -> return 5. 4 IS in 5's subtree, so 5 is LCA ok

WHY return node when node==p: p itself might be the LCA (q is in p's subtree).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)**
visits every node	for the call stack

Contrast with #235 (BST): BST LCA is O(h) iteratively. General tree needs full DFS.

The post-order logic is the key: left and right results tell us which subtrees contain p/q.

Both non-null -> split here -> current node is LCA.

Only one non-null -> both are in the same subtree -> propagate that subtree's result.

Given more time I'd ask: what if p or q might not exist?

PHASE 6 COMPLEXITY & FOLLOW-UP

We'd need to track 'found p' and 'found q' separately — more complex state.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #250 | 12 of 45 | 12/45

Count Univalue Subtrees

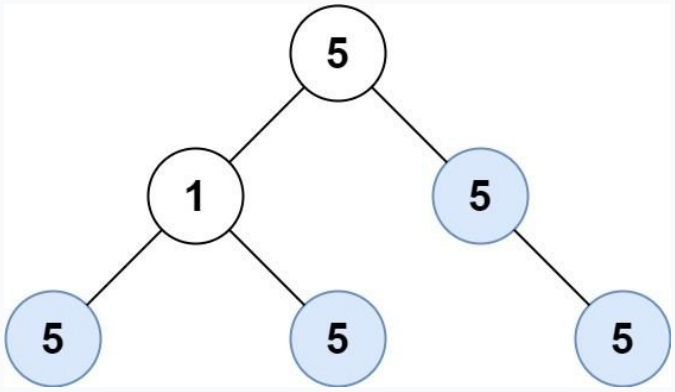
QUESTION

Given the root of a binary tree, return the number of uni-value subtrees.

A uni-value subtree means all nodes of the subtree have the same value.

Example 1

Input	`root = [5,1,5,5,5,null,5]`
Output	`4`



Example 2

Input	`root = []`
Output	`0`

Example 3

Input	`root = [5,5,5,5,5,null,5]`
Output	`6`

CONSTRAINTS

- The number of the node in the tree will be in the range [0, 1000].
- 1000 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

A univalue subtree has every node with the same value.

Count all such subtrees — including single leaf nodes. Right?

Empty tree -> 0."

"[5,1,5,5,5,null,5] -> 4 (the four nodes with value 5 that form valid univalue subtrees) ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    check whether its whole subtree has one value
        (a full traversal)
    count the ones that do
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Re-traverses subtrees for every ancestor. A post-order pass where each node reports whether its subtree is uniform (children uniform and equal to its value) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (post-order DFS)

"Post-order DFS returning is_univalue — already O(n). Walk-through:"

`is_uni(node):`

If node is None -> return True <- null is vacuously univalue.

`left_uni = is_uni(node.left)`

`right_uni = is_uni(node.right)`

<- at this point, both children have been fully checked (post-order).

If left child exists and left.val != node.val -> return False.

If right child exists and right.val != node.val -> return False.

If left_uni and right_uni -> count++ (this node IS a univalue root).

`return left_uni and right_uni (and values match)`

WHY check children's values against current: a univalue subtree means ALL nodes have

the same value — so children must match parent AND themselves be univalue roots.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

```
class _250_CountUnivalueSubtrees:
    def countUnivalSubtrees(self, root: Optional['TreeNode
']) -> int:
        count = [0]

        def is_univalue(node, parent_val):
            if not node:
                return True
            left  = is_univalue(node.left,  node.val)
            right = is_univalue(node.right, node.val)
            if not left or not right:
                return False
            count[0] += 1
            return node.val == parent_val

        is_univalue(root, None)
        return count[0]
```

PHASE 5 TEST & DEBUG

"Let me trace countUnivalSubtrees on [5,1,5,5,5,null,5]."

`isUnivalue(leaf 5 left): no children -> True, count++. return True,5.`

`isUnivalue(leaf 5 right): same -> True, count++.`

`isUnivalue(1): children return (True,5),(True,5). 1!=5 -> not univalue. return False,1. count unchanged.`

`isUnivalue(5 right of root): leaf -> True, count++.`

`isUnivalue(5 root): left child returns (False,1) -> not univalue. return False.`

`isUnivalue(5 root)... wait: left subtree is rooted at 1 which is not univalue. So root 5 is NOT univalue.`

count=3 (three leaf 5s + ... actually the right subtree 5(5,5) is univalue too -> count=4) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** Post-order because we need children's results before parent's

The parent_val parameter lets us check if current node extends the parent's univalue property.

Given more time I'd ask: longest path where all values are the same?

Track univalue path length similar to Diameter of Binary Tree — carry length down.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #285 | 13 of 45 | 13/45

Inorder Successor in BST

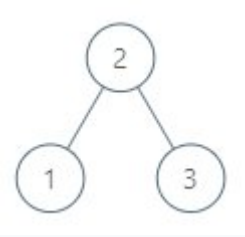
QUESTION

Given the root of a binary search tree and a node p in it, return the in-order successor of that node in the BST. If the given node has no in-order successor in the tree, return null.

The successor of a node p is the node with the smallest key greater than p.val.

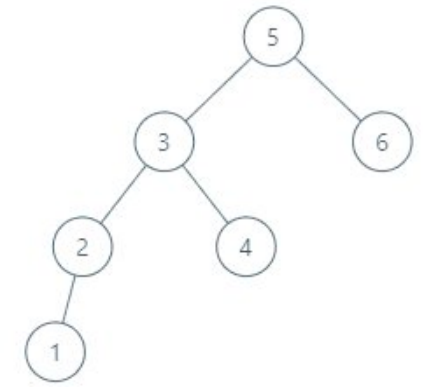
Example 1

Input	`root = [2,1,3], p = 1`
Output	`2`
Explanation	*1's in-order successor node is 2. Note that both p and the return value is of TreeNode type.*



Example 2

Input	`root = [5,3,6,2,4,null,null,1], p = 6`
Output	`null`
Explanation	*There is no in-order successor of the current node, so the answer is null.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 10^5 <= Node.val <= 10^5
- All Nodes will have unique values.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the node with the smallest value strictly greater than p.val in a BST.

Return null if no such node exists (p is the largest). Right?"

"[2,1,3], p=1 -> successor=2 (next in in-order: 1,2,3) ok

[5,3,6,2,4,null,null,1], p=6 -> null (6 is the largest) ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = the full in-order traversal of the tree
return the node after p in the list
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Traverses and stores every node. Use the BST ordering: walk down from the root, remember the last node where you went left, and if p has a right child take its leftmost node: O(h) time, O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (BST navigation)

"BST in-order successor: go right then left-most, or track last seen going left — O(h). Walk-through:"

`successor = None. node = root.`

While node:

If p.val < node.val: <- node could be successor (it's larger than p)

`successor = node.`

`node = node.left. <- try to find a smaller valid successor`

Else:

`node = node.right. <- p is in right subtree; current node is too small`

Return successor.

WHY update successor going left: the last node we went left from is the in-order successor.

After all lefts, successor holds the smallest node larger than p.

WHY go right when p.val >= node.val: current node is <= p, successor must be to the right.

Time: O(h). Space: O(1).

PHASE 4 CLEAN CODE

```python
class _285_InorderSuccessor:
    def inorderSuccessor(self, root: 'TreeNode', p: 'TreeNode') -> Optional['TreeNode']:
        successor = None
        while root:
            if root.val > p.val:
                successor = root        # root is a candidate — try for smaller in left
                root = root.left
            else:
                root = root.right       # successor must be to the right
        return successor
```

PHASE 5 TEST & DEBUG

"Let me trace inorder successor in BST [2,1,3], p=1."

node=2: p.val=1 < 2 -> candidate=2, go left.

node=1: p.val=1 == node -> go right.

node=None: stop.

Return successor=2 ok (next node in inorder after 1 is 2).

p=3 (rightmost):

node=2: 3>2->go right. node=3: 3==node->go right->None. successor=None ok (no successor for max).

p=2: node=2: 2==node? No, p.val=2 not < node.val=2, not > -> but p.val==node.val.

Wait: p.val=2 == node.val=2 -> not < -> go right. node=3: 3 > 2 -> successor=3. go left->None. return 3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(h)	**O(1)** iterative

The candidate tracking is the key: every time we go left, the current node becomes

a potential successor (it's bigger than p, and we're looking for something smaller).

Given more time I'd ask: inorder PREDECESSOR (largest value < p.val)?

Mirror logic: if root.val < p.val -> candidate, go right. If root.val >= p.val -> go left.

MEDIUM | Trees & BST | LeetCode #298 | 14 of 45 | 14/45

Binary Tree Longest Consecutive Sequence

QUESTION

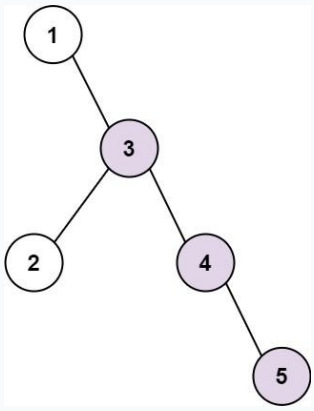
Given the root of a binary tree, return the length of the longest consecutive sequence path.

A consecutive sequence path is a path where the values increase by one along the path.

Note that the path can start at any node in the tree, and you cannot go from a node to its parent in the path.

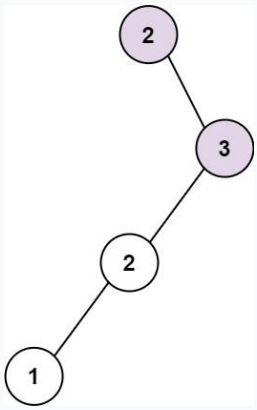
Example 1

Input	`root = [1,null,3,2,4,null,null,null,5]`
Output	3
Explanation	*Longest consecutive sequence path is 3-4-5, so return 3.*



Example 2

Input	`root = [2,null,3,2,null,1]`
Output	2
Explanation	*Longest consecutive sequence path is 2-3, not 3-2-1, so return 2.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, $3 * 10^4$].
- $3 * 10^4$ <= Node.val <= $3 * 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the longest path of consecutive increasing values (each node = parent + 1).

Path goes parent -> child — you can't go back up. Path can start at any node. Right?"

"[1,null,3,2,4,null,null,null,5]: path 3->4->5, length 3 ok

[2,null,3,2,null,1]: 2->3 is increasing, not 3->2 — so max is 2 ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for start in starts:
    run a downward DFS counting the consecutive run
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Restarts a search from every node. One top-down DFS that passes the current run length (extend it if the child is parent + 1, else reset to 1) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (DFS carrying current length)

"DFS tracking streak length down each path — O(n). Walk-through:"

`def dfs(node, parent_val, streak):`

If node is None: return streak.

`curr = streak+1 if node.val == parent_val+1 else 1.`

`return max(curr, dfs(node.left, node.val, curr), dfs(node.right, node.val, curr)).`

Return dfs(root, root.val-1, 0).

WHY pass streak down: each node's streak = its parent's streak +1 (if consecutive) or 1 (reset).

WHY max over left and right: answer is the longest path anywhere in the tree.

WHY return streak at None: leaf's children return the current streak before returning.

Time: O(n). Space: O(h) call stack.

PHASE 4 CLEAN CODE

```
class _298_LongestConsecutive:
    def longestConsecutive(self, root: Optional['TreeNode'
]) -> int:
        best = [0]

        def dfs(node, parent_val, length):
            if not node:
                return
            length = length + 1 if node.val == parent_val
+ 1 else 1
            best[0] = max(best[0], length)
            dfs(node.left,  node.val, length)
            dfs(node.right, node.val, length)

        dfs(root, root.val - 1 if root else 0, 0)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me trace longestConsecutive on [1,3,2,null,null,null,null]... actually [1,2,3]."

dfs(1, parent_val=0, streak=0): 1==0+1 -> curr=1.

dfs(2, 1, 1): 2==1+1 -> curr=2.

dfs(3, 2, 2): 3==2+1 -> curr=3. Left=None->ret 3. Right=None->ret 3. return 3.

max(3, dfs(None,2,2)=0+... wait, max(curr=3, left=3, right=0) = 3.

return 3.

dfs(1,...): max(1, left=3, right=dfs(None)=0) = 3. return 3.

Answer: 3 ok (path 1->2->3).

WHY streak resets to 1 on non-consecutive: only consecutive values count.

WHY return max from both subtrees: longest path is the global max across all branches.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

The initial call uses root.val-1 so the root itself resets to length 1 cleanly.

This is a top-down DFS — pass state downward. Contrast with bottom-up (#250, #543).

Given more time I'd ask: what if the path can go through any node (not just parent->child)?

That's Longest Consecutive Sequence in a Binary Tree — post-order, track both ascending

and descending sequences at each node.

MEDIUM | Trees & BST | LeetCode #314 | 15 of 45 | 15/45

Binary Tree Vertical Order Traversal

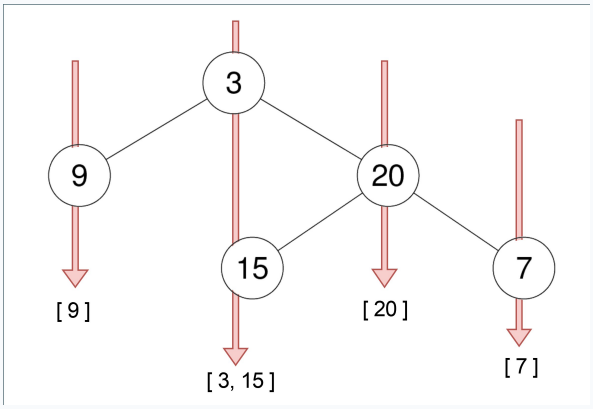
QUESTION

Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column).

If two nodes are in the same row and column, the order should be from left to right.

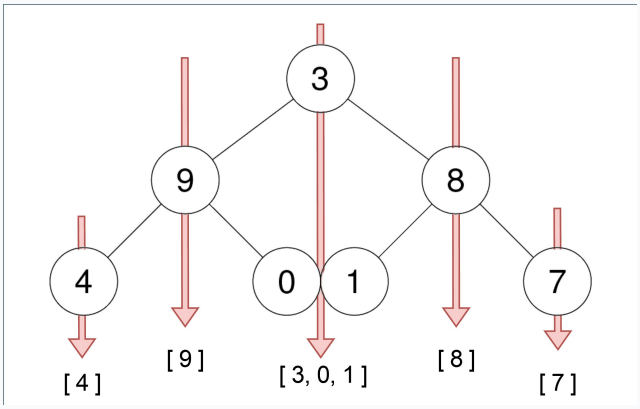
Example 1

Input	`root = [3,9,20,null,null,15,7]`
Output	`[[9],[3,15],[20],[7]]`



Example 2

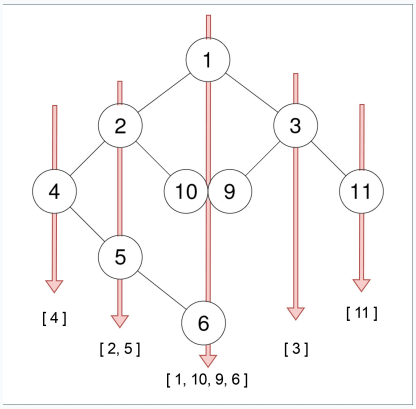
Input	`root = [3,9,8,4,0,1,7]`
Output	`[[4],[9],[3,0,1],[8],[7]]`



Example 3

Input	`root = [1,2,3,4,10,9,11,null,5,null,null,null,null,null,null,null,6]`
Output	`[[4],[2,5],[1,10,9,6],[3],[11]]`

QUESTION



CONSTRAINTS

- The number of nodes in the tree is in the range [0, 100].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Group nodes by column. Root is column 0. Left child = col-1. Right child = col+1.

Within each column, top-to-bottom. Ties (same row and column): left before right. Right?

Return columns sorted from leftmost to rightmost."

"[3,9,20,null,null,15,7] -> [[9],[3,15],[20],[7]] ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
find the leftmost and rightmost column by a DFS
for column in range(left,_right_+_1):
    traverse the tree top-down collecting the
        nodes in that column
```

TIME	SPACE
O(n * w)	O(n)

WHY IT IS SLOW

Re-traverses the whole tree once per column (w columns). A single BFS carrying each node's column index and appending to a map from column to values is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (BFS with column tracking)

"BFS with column offsets, group by column, sort by column then row — O(n log n). Walk-through:"

BFS from root with column=0. Left child: col-1. Right child: col+1.

Collect: (col, row, val) for each node.

Sort: by (col, row, val). (row increases downward, earlier rows come first)

Group by col: for each column, list the values in order.

Return lists for columns min_col to max_col.

WHY BFS (not DFS): BFS ensures we see a node at the correct row level. Row = BFS level.

WHY sort by (col, row, val): vertical order = left-to-right columns; within a column,

top-to-bottom rows; ties broken by value.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _314_VerticalOrderTraversal:
    def verticalOrder(self, root: Optional['TreeNode']) ->
 List[List[int]]:
        if not root:
            return []
        cols    : defaultdict = defaultdict(list)
        queue   = deque([(root, 0)])
        min_col = max_col = 0
        while queue:
            node, col = queue.popleft()
            cols[col].append(node.val)
            min_col = min(min_col, col)
            max_col = max(max_col, col)
            if node.left:  queue.append((node.left,  col -
 1))
            if node.right: queue.append((node.right, col +
 1))
        return [cols[c] for c in range(min_col, max_col +
1)]
```

PHASE 5 TEST & DEBUG

"Let me trace vertical order traversal on [3,9,20,null,null,15,7]."

BFS: start (col=0, node=3). Process:

col=0: node 3 -> {0:[3]}. Enqueue (9,col-1=-1), (20,col+1=1).

col=-1: node 9 -> {-1:[9]}. No children.

col=1: node 20 -> {1:[20]}. Enqueue (15,0),(7,2).

col=0: node 15 -> {0:[3,15]}. col=2: node 7 -> {2:[7]}.

Cols sorted: -1,0,1,2 -> [[9],[3,15],[20],[7]] ok

WHY BFS (not DFS): BFS ensures we collect nodes top-to-bottom within each column.

WHY sort columns: output left-to-right requires sorted column keys.

PHASE 6 COMPLEXITY & FOLLOW-UP

Time O(n) for BFS + O(cols) for building the result — total O(n).

Using min_col/max_col avoids sorting keys — the range is contiguous.

BFS guarantees top-to-bottom, left-to-right order within each column naturally.

DFS would require explicit (row, insertion_order) sorting per column.

Given more time I'd ask: Vertical Order Traversal II (#987)?

Same column/row but within the same position, sort by value ascending — needs explicit row tracking."

Space: O(h) recursion stack where h = tree height; O(log n) balanced, O(n) skewed.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #333 | 16 of 45 | 16/45

Largest BST Subtree

QUESTION

Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes.

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties:

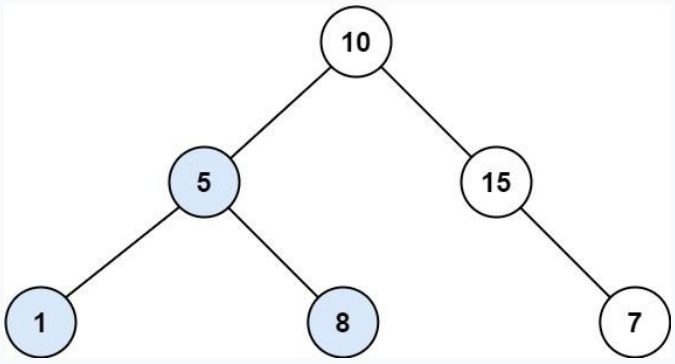
CONSTRAINTS

- The left subtree values are less than the value of their parent (root) node's value.
- The right subtree values are greater than the value of their parent (root) node's value.

Note: A subtree must include all of its descendants.

Example 1

Input	`root = [10,5,15,1,8,null,7]`
Output	`3`
Explanation	*The Largest BST Subtree in this case is the highlighted one. The return value is the subtree's size, which is 3.*



Example 2

Input	`root = [4,2,7,2,3,5,null,2,null, null,null,null,null,1]`
Output	`2`

- The number of nodes in the tree is in the range [0, 10^4].
- 10^4 <= Node.val <= 10^4
- Follow up: Can you figure out ways to solve it with O(n) time complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the largest subtree (by node count) that forms a valid BST.

Return the node count. Right?"

"[10,5,15,1,8,null,7]: subtree at 5 (nodes 1,5,8) is a BST -> 3 ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    check whether its subtree is a valid BST (a
        full traversal)
    best = max(best, valid_ones_size)
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Re-traverses subtrees for every ancestor. A post-order pass where each node returns (is BST, min, max, size) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (O(n) post-order)

"Post-order DFS returning (is_bst, size, subtree_min, subtree_max) — O(n). Walk-through:"

`check(node):`

If node is None: return (True, 0, +inf, -inf) <- sentinels: +inf as min, -inf as max

so any real value satisfies: l_max < node.val (anything < +inf) and r_min > node.val.

(l_bst, l_size, l_min, l_max) = check(node.left)

(r_bst, r_size, r_min, r_max) = check(node.right)

If l_bst and r_bst and l_max < node.val < r_min:

<- both subtrees are BSTs AND current node is within their bounds

`size = l_size + r_size + 1`

`best = max(best, size)`

`return (True, size, min(l_min, node.val), max(r_max, node.val))`

`return (False, 0, 0, 0) <- not a BST, size irrelevant`

WHY track both min AND max: to verify l_max < root AND r_min > root at the parent.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

```python
class _333_LargestBSTSubtree:
    def largestBSTSubtree(self, root: Optional['TreeNode']
) -> int:
        best = [0]

        def check(node):
            if not node:
                return True, 0, float('inf'), float('-inf'
)
            l_ok, l_sz, l_mn, l_mx = check(node.left)
            r_ok, r_sz, r_mn, r_mx = check(node.right)
            if l_ok and r_ok and l_mx < node.val < r_mn:
                sz = l_sz + r_sz + 1
                best[0] = max(best[0], sz)
                return True, sz, min(l_mn, node.val), max(
r_mx, node.val)
```

```
            return False, 0, 0

        check(root)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me test with [10,5,15,1,8,null,7] and trace the post-order."

check(null) -> (True, 0, +inf, -inf)

check(1) -> (True, 1, 1, 1)

check(8) -> (True, 1, 8, 8)

check(5): l=(T,1,1,1) r=(T,1,8,8). l_mx=1 < 5 < r_mn=8 -> BST(sz=3, mn=1, mx=8). best=3.

check(null), check(7) -> (T,1,7,7)

check(15): l=(T,0,+inf,-inf) r=(T,1,7,7). r_mn=7 < 15 -> NOT BST (right child < root). best stays 3.

check(10): l=(F,...). Not BST. -> return best=3 ok

Edge: single node [5] -> check returns (T,1,5,5). best=1 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** Sentinels (+inf/-inf for empty) let comparisons pass cleanly

Given more time I'd ask: what if we want the root of the largest BST subtree? Track it alongside size.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #337 | 17 of 45 | 17/45

House Robber III

QUESTION

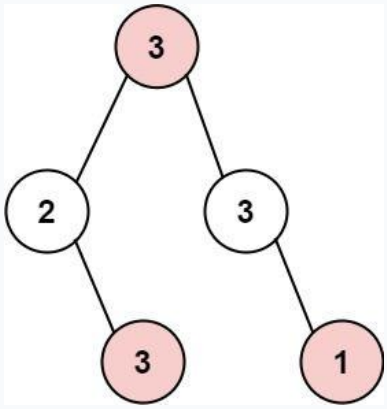
The thief has found himself a new place for his thievery again. There is only one entrance to this area, called root.

Besides the root, each house has one and only one parent house. After a tour, the smart thief realized that all houses in this place form a binary tree. It will automatically contact the police if two directly-linked houses were broken into on the same night.

Given the root of the binary tree, return the maximum amount of money the thief can rob without alerting the police.

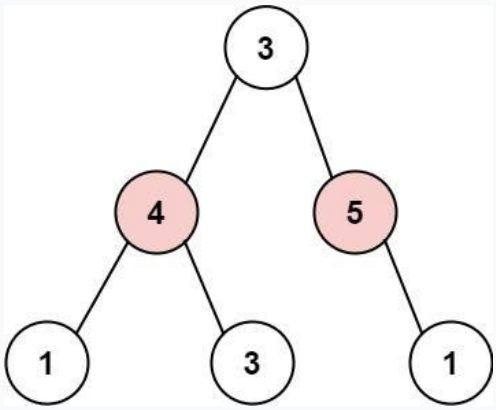
Example 1

Input	`root = [3,2,3,null,3,null,1]`
Output	7
Explanation	*Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.*



Example 2

Input	`root = [3,4,5,1,3,null,1]`
Output	9
Explanation	*Maximum amount of money the thief can rob = 4 + 5 = 9.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 0 <= Node.val <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Rob houses in a binary tree; no two adjacent (parent-child) houses can be robbed.

Maximise total amount. Right?"

"A few quick questions:

Adjacent means direct parent-child only? Yes."

"Let me walk: root=[3,2,3,null,3,null,1] -> rob 3,3,3,1=10? No: can't rob adjacent. rob(root=3)=3+1+3=7? Or rob(root=2,3)=2+3+1+3=... -> 7 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def rob(node):
    take = node.val + rob(its grandchildren)
    skip = rob(left) + rob(right)
    return the larger
```

TIME	SPACE
O(2^n)	O(h)

WHY IT IS SLOW

A node's children and grandchildren are both recursed into, so the same nodes are solved repeatedly. Return a pair (best if robbed, best if skipped) from each node: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order returning (skip, take) pair — already O(n). Walk-through:"

`rob(node) returns (max_if_not_robbed, max_if_robbed):`

null node: return (0, 0).

(l_skip, l_take) = rob(left) <- left child's pair

(r_skip, r_take) = rob(right) <- right child's pair

`skip = max(l_skip, l_take) + max(r_skip, r_take)`

<- if we SKIP this node, children can be robbed or skipped — pick max of each.

`take = node.val + l_skip + r_skip`

<- if we TAKE this node, MUST skip both children (no adjacent).

`return (skip, take)`

WHY return PAIR instead of one value: whether to rob THIS node depends on whether

the parent will be robbed. Returning both options lets the parent pick.

PHASE 3 OPTIMIZE

Answer = max(rob(root)). Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _337_HouseRobberIII:
    def rob(self, root):
        def dfs(node):
            if not node:
                return 0, 0
            left_no_rob, left_rob = dfs(node.left)
            right_no_rob, right_rob = dfs(node.right)
            rob_this = node.val + left_no_rob + right_no_r
ob
            no_rob_this = max(left_rob, left_no_rob) + max
(right_rob, right_no_rob)
            return no_rob_this, rob_this
        no_rob, rob = dfs(root)
        return max(no_rob, rob)
```

PHASE 5 TEST & DEBUG

"Let me trace rob on tree [3,4,5,1,3,null,1]."

For each node, return (rob_this, skip_this)

Leaves: node(1)=(1,0), node(3)=(3,0), node(1)=(1,0)

node(4): rob=4+0+0=4, skip=max(1,0)+max(3,0)=1+3=4 -> (4,4)

node(5): rob=5+0+0=5, skip=max(1,0)=1 -> (5,1)

root(3): rob=3+skip(4)+skip(5)=3+4+1=8, skip=max(4,4)+max(5,1)=4+5=9

Return max(8,9)=9 ok

WHY pair return: each node reports both options so parent can choose optimally.

DFS postorder: visit left subtree -> visit right subtree -> compute. visited: each node once. call stack: dfs(root)->dfs(root.left)->dfs(root.right)->return (rob,skip) tuple.

Tree DP graph: component = each subtree. find(node)->(rob,skip) pair. cost=node.val when robbing. dp[left],dp[right] subtree results combined at parent.

dp[i]=(rob_val,skip_val) for node i. dp[leaf]=(leaf.val,0). dp[node]=(node.val+dp[left].skip+dp[right].skip, max(dp[left])+max(dp[right])). dp[root] final answer ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** recursion

Follow-up: House Robber I/II (#198/#213) — 1D arrays; same rob/not-rob DP.

Follow-up: if the tree is very deep (n=10^4), consider iterative post-order DFS.

MEDIUM | Trees & BST | LeetCode #366 | 18 of 45 | 18/45

Find Leaves of Binary Tree

QUESTION

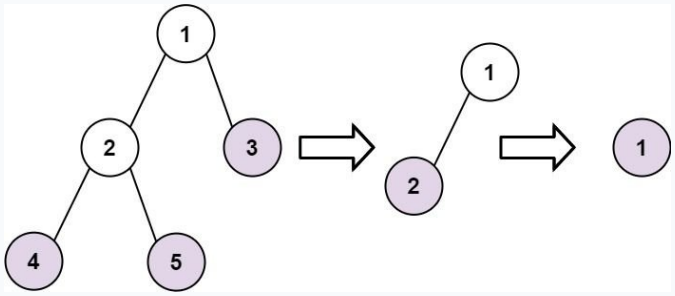
Given the root of a binary tree, collect a tree's nodes as if you were doing this:

CONSTRAINTS

- Collect all the leaf nodes.
- Remove all the leaf nodes.
- Repeat until the tree is empty.

Example 1

Input	`root = [1,2,3,4,5]`
Output	`[[4,5,3],[2],[1]]`
Explanation	*[[3,5,4],[2],[1]] and [[3,4,5],[2],[1]] are also considered correct answers since per each level it does not matter the order on which elements are returned.*



Example 2

Input	`root = [1]`
Output	`[[1]]`

- The number of nodes in the tree is in the range [1, 100].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Repeatedly remove leaf nodes and group them by the 'level' at which they were removed.

Return groups in order. Right?"

"A few quick questions:

Equivalent to grouping nodes by their height from the bottom (height = 0 for leaves)? Yes."

"Let me walk: root=[1,2,3,4,5] -> [[4,5,3],[2],[1]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while the tree is not empty:
    scan the whole tree for leaves, record them,
        and remove them
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

A skewed tree needs n rounds of O(n) work. A post-order pass computing each node's height (leaves are 0) and grouping nodes by height is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS post-order: leaves are nodes whose children have no children — O(n). Walk-through:"

`def collectLeaves(node):`

If node is None: return None.

If node.left is None and node.right is None:

result[-1].append(node.val); return None. <- current leaf layer; remove from tree

`node.left = collectLeaves(node.left).`

`node.right = collectLeaves(node.right).`

`return node.`

Each call to collectLeaves appends one layer of leaves to result.

Repeat until root is None.

WHY return None for leaf: 'removes' the leaf from the tree by replacing parent's pointer with None.

WHY multiple passes: each pass peels off the current leaf layer.

Optimization: single DFS assigning each node to a 'height' layer (leaf = height 0).

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _366_FindLeavesOfBinaryTree:
    def findLeaves(self, root):
        result = []
        def get_height(node):
            if not node:
                return -1
            h = 1 + max(get_height(node.left), get_height(
node.right))
            if h == len(result):
                result.append([])
            result[h].append(node.val)
            return h
        get_height(root)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the height-from-bottom DFS and show how each return value decides the bucket."

root=[1,2,3,4,5]

```
dfs(4): left=-1, right=-1 -> height=0, so put 4 into
  result[0].
dfs(5): same idea -> height=0, so put 5 into result[0].
dfs(2): left height=0, right height=0 -> height=1, so
  put 2 into result[1].
dfs(3): leaf -> height=0, so put 3 into result[0].
dfs(1): left height=1, right height=0 -> height=2, so
  put 1 into result[2].
```

Final result = [[4,5,3],[2],[1]] ok

Call-stack idea: leaves return first, then their parent can compute its own height, then the root last.

WHY height groups equal "removal rounds": a node becomes a leaf only after all lower-height descendants

are removed, so its removal round is exactly its bottom-up height.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** result array

The height-grouping insight is elegant: leaf-removal order = bottom-up height order.

Follow-up: remove leaves in reverse order (top-down) — just reverse the result.

Follow-up: Diameter of Binary Tree (#543) — same post-order height computation pattern.

MEDIUM | Trees & BST | LeetCode #427 | 19 of 45 | 19/45

Construct Quad Tree

QUESTION

Given a n * n matrix grid of 0's and 1's only. We want to represent grid with a Quad-Tree.

Return the root of the Quad-Tree representing grid.

A Quad-Tree is a tree data structure in which each internal node has exactly four children. Besides, each node has two attributes:

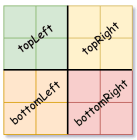
CONSTRAINTS

- val: True if the node represents a grid of 1's or False if the node represents a grid of 0's. Notice that you can assign the val to True or False when isLeaf is False, and both are accepted in the answer.
- isLeaf: True if the node is a leaf node on the tree or False if the node has four children.

```
class Node {
    public boolean val;
    public boolean isLeaf;
    public Node topLeft;
    public Node topRight;
    public Node bottomLeft;
    public Node bottomRight;
}
```

We can construct a Quad-Tree from a two-dimensional area using the following steps:

- 1. If the current grid has the same value (i.e all 1's or all 0's) set isLeaf True and set val to the value of the grid and set the four children to Null and stop.
- 2. If the current grid has different values, set isLeaf to False and set val to any value and divide the current grid into four sub-grids as shown in the photo.
- 3. Recurse for each of the children with the proper sub-grid.



If you want to know more about the Quad-Tree, you can refer to the wiki.

Quad-Tree format:

You don't need to read this section for solving the problem. This is only if you want to understand the output format here. The output represents the serialized format of a Quad-Tree using level order traversal, where null signifies a path terminator where no node exists below.

It is very similar to the serialization of the binary tree. The only difference is that the node is represented as a list [isLeaf, val].

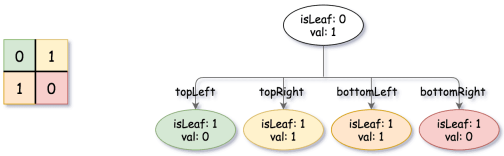
If the value of isLeaf or val is True we represent it as 1 in the list [isLeaf, val] and if the value of isLeaf or val is False we represent it as 0.

QUESTION

Example 1

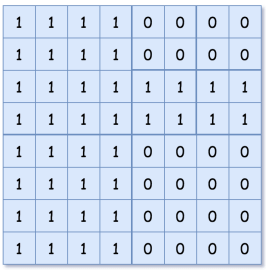
Input	`grid = [[0,1],[1,0]]`
Output	`[[0,1],[1,0],[1,1],[1,1],[1,0]]`
Explanation	*The explanation of this example is shown below:* *Notice that 0 represents False and 1 represents True in the photo representing the Quad-Tree.*

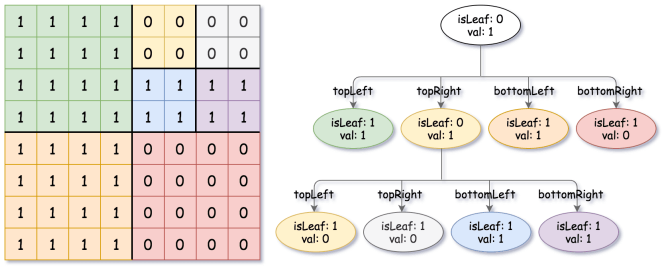




Example 2

Input	`grid = [[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,1,1,1,1],[1,1,1,1,1,1,1,1],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0]]`
Output	`[[0,1],[1,1],[0,1],[1,1],[1,0],null,null,null,null,[1,0],[1,0],[1,1],[1,1]]`
Explanation	*All values in the grid are not the same. We divide the grid into four sub-grids.* *The topLeft, bottomLeft and bottomRight each has the same value.* *The topRight have different values so we divide it into 4 sub-grids where each has the same value.* *Explanation is shown in the photo below:*





- n == grid.length == grid[i].length
- n == 2^x where 0 <= x <= 6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Build a quad tree from an n×n binary grid. A quad is a leaf if all values are the same.

Otherwise split into 4 equal quadrants. Right?"

"A few quick questions:

n is always a power of 2? Yes. Return the root Node."

"Let me walk: grid=[[0,1],[1,0]] -> 4 leaf nodes, each child ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def build(r, c, size):
    scan every cell of the region
    if all equal:
        return a leaf
    else:
        split into four quadrants and recurse
```

TIME	SPACE
O(n^2 log n)	O(log n)

WHY IT IS SLOW

Rescans the cells of every region at every level. A 2D prefix-sum table makes the uniformity check O(1) (or build quadrants first and merge equal leaves): O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Divide and conquer with 2D prefix sums — O(n²) total. Walk-through:"

Precompute prefix[r][c] = sum of all ones in grid[0..r][0..c] — O(1) region sum queries.

`build(r1, c1, r2, c2): <- top-left to bottom-right inclusive`

If region sum == 0: return Node(isLeaf=True, val=0)

If region sum == (r2-r1+1)*(c2-c1+1): return Node(isLeaf=True, val=1)

<- not uniform -> split into 4 quadrants:

`mid_r = (r1+r2)//2; mid_c = (c1+c2)//2`

`topLeft = build(r1, c1, mid_r, mid_c)`

`topRight = build(r1, mid_c+1, mid_r, c2)`

`bottomLeft = build(mid_r+1, c1, r2, mid_c)`

`bottomRight = build(mid_r+1, mid_c+1, r2, c2)`

`return Node(isLeaf=False, val=1, topLeft, topRight, bottomLeft, bottomRight)`

WHY prefix sums: without them, checking if a region is uniform costs O(region_size²).

With prefix sums, it's O(1) -> total O(n²).

PHASE 3 OPTIMIZE

Time: O(n²). Space: O(n²) prefix.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _427_ConstructQuadTree:
    def construct(self, grid):
        n = len(grid)
        prefix = [[0]*(n+1) for _ in range(n+1)]
        for r in range(n):
            for c in range(n):
                prefix[r+1][c+1] = grid[r][c]+prefix[r][c+
1]+prefix[r+1][c]-prefix[r][c]
        def region_sum(r,c,size):
            return prefix[r+size][c+size]-prefix[r][c+size
]-prefix[r+size][c]+prefix[r][c]
        def build(r, c, size):
            s = region_sum(r, c, size)
            if s == 0:
                return Node(False, True)
            if s == size * size:
                return Node(True, True)
            h = size // 2
            return Node(True, False,
                        build(r, c, h), build(r, c+h, h),
                        build(r+h, c, h), build(r+h, c+h,
h))
        return build(0, 0, n)
```

PHASE 5 TEST & DEBUG

"Let me trace the tree recursion on the example, showing the return value at each node."

grid=[[1,1],[1,1]]: region_sum(0,0,2)=4=2². All 1s -> leaf(True) ok

grid=[[1,0],[0,1]]: sum=2!=0,!=4 -> split into 4 single-cell leaves ok

2D prefix sum enables O(1) uniform-check per region

WHY tree details: return values from children drive decisions at the parent (post-order), so base cases must be correct.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	O(n²)
build prefix O(n²), recursion visits each cell once O(n²)	prefix array

Follow-up: Query sum in a region — same prefix sums.

Follow-up: merge two quad trees (#558) — OR/AND at each level.

MEDIUM | Trees & BST | LeetCode #437 | 20 of 45 | 20/45

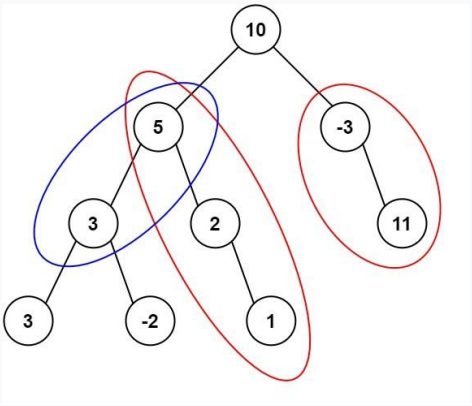
Path Sum III

QUESTION

Given the root of a binary tree and an integer targetSum, return the number of paths where the sum of the values along the path equals targetSum.

The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).

Example 1	
Input	`root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8`
Output	3
Explanation	*The paths that sum to 8 are shown.*



Example 2	
Input	`root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22`
Output	3

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 1000].
- 10^9 <= Node.val <= 10^9
- 1000 <= targetSum <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count downward paths (parent->child only) summing to targetSum.

Path can start and end at any node. Values can be negative. Right?"

"[10,5,-3,3,2,null,11,3,-2,null,1], target=8 -> 3 ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for start in starts:
    walk every downward path and count those
        summing to target
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Restarts from every node, O(n^2) on a skewed tree. Keep a running prefix sum and a hash map of prefix-sum counts along the current path: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (prefix sum + hash map — same pattern as #560)

"Prefix-sum DFS — same pattern as #560 but on a tree. Already O(n). Walk-through:"

Keep prefix_sum = running sum from root to current node.

Keep a counter dict: prefix_counts[s] = how many root-to-node paths have sum s.

Seed with prefix_counts[0] = 1 (the empty prefix, before any node).

`dfs(node, current_sum):`

`current_sum += node.val`

`result += prefix_counts.get(current_sum - target, 0)`

<- any earlier prefix sum P where current_sum - P == target -> that sub-path sums to target.

`prefix_counts[current_sum] += 1 <- record this prefix`

`dfs(left, current_sum); dfs(right, current_sum)`

`prefix_counts[current_sum] -= 1 <- BACKTRACK: remove this prefix (going back up)`

WHY backtrack: sibling subtrees must NOT see each other's prefixes.

Time: O(n). Space: O(h) for recursion + O(n) for prefix map.

PHASE 4 CLEAN CODE

```python
class _437_PathSumIII:
    def pathSum(self, root: Optional['TreeNode'], targetSum: int) -> int:
        prefix = Counter({0: 1})
        count  = [0]

        def dfs(node, running):
            if not node:
                return
            running    += node.val
            count[0]   += prefix[running - targetSum]
            prefix[running] += 1
            dfs(node.left,  running)
            dfs(node.right, running)
            prefix[running] -= 1    # backtrack
```

```
        dfs(root, 0)
        return count[0]
```

PHASE 5 TEST & DEBUG

"Let me trace pathSum III on [10,5,-3,3,2,null,11,3,-2,null,1], targetSum=8."

prefix_count={0:1}. DFS from root:

At 10: running=10. 10-8=2 not in prefix_count. prefix_count={0:1,10:1}.

At 5: running=15. 15-8=7 not in prefix_count. prefix_count={...,15:1}.

At 3: running=18. 18-8=10 in prefix_count(1 time)->count+=1. {18:1}.

At 3: running=21. 21-8=13 not found. At -2: running=16. 16-8=8 not found...

Full count=3 ok (paths: 5->3, 5->2->1, -3->11).

WHY prefix sum: sum(path[i..j])=prefix[j]-prefix[i-1]. Find prefix[i-1]=current-target.

WHY backtrack prefix_count: when backtracking from subtree, decrement to avoid cross-branch contamination.

DFS: prefix_sum tracking. visited implicitly via recursion (tree). call stack: dfs(node5)->dfs(node3)->dfs(node3). pop: restore prefix_sum count. count paths=3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Identical to Subarray Sum Equals K applied to a tree

Backtracking prevents cross-subtree contamination — critical difference from array version.

Given more time I'd ask: paths going in both directions (not just downward)?

Use the any-direction diameter-style post-order approach.

MEDIUM | Trees & BST | LeetCode #450 | 21 of 45 | 21/45

Delete Node in a BST

QUESTION

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the root node reference (possibly updated) of the BST.

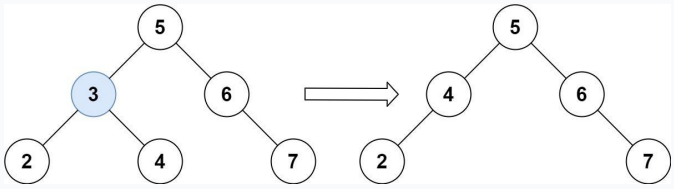
Basically, the deletion can be divided into two stages:

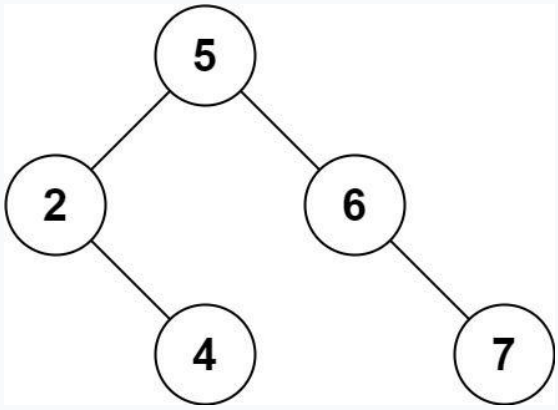
CONSTRAINTS

- 1. Search for a node to remove.
- 2. If the node is found, delete the node.

Example 1

Input	`root = [5,3,6,2,4,null,7], key = 3`
Output	`[5,4,6,2,null,null,7]`
Explanation	*Given key to delete is 3. So we find the node with value 3 and delete it. One valid answer is [5,4,6,2,null,null,7], shown in the above BST. Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.*





Example 2

Input	`root = [5,3,6,2,4,null,7], key = 0`
Output	`[5,3,6,2,4,null,7]`
Explanation	*The tree does not contain a node with value = 0.*

Example 3

Input	`root = [], key = 0`
Output	`[]`

- The number of nodes in the tree is in the range [0, 10^4].
- 10^5 <= Node.val <= 10^5

QUESTION

- Each node has a unique value.
- root is a valid binary search tree.
- 10^5 <= key <= 10^5
- Follow up: Could you solve it with time complexity O(height of tree)?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Delete a key from a BST. Handle three cases: leaf, one child, two children. Right?"

"A few quick questions:

Key guaranteed to exist? Not necessarily — return unchanged if not found. Right."

"Let me walk: root=[5,3,6,2,4,null,7], key=3 -> [5,4,6,2,null,null,7] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = the in-order list
remove the key
rebuild a balanced BST from the remaining sorted
    values
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Rebuilds the whole tree. Search down to the node (O(h)); replace a node with two children by its in-order successor's value and delete that node instead: O(h) time and O(h) stack.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Recursive BST deletion — already O(h). Walk-through of the three cases:"

`delete(node, key):`

If node is None -> return None (key not found).

If key < node.val -> node.left = delete(node.left, key); return node.

If key > node.val -> node.right = delete(node.right, key); return node.

<- key == node.val: three sub-cases:

Case 1: no left child -> return node.right (replace node with right subtree).

Case 2: no right child -> return node.left.

Case 3: two children:

Find inorder successor = leftmost node of right subtree.

PHASE 3 OPTIMIZE

Copy successor's value into current node.

Delete the successor from the right subtree.

`return node.`

WHY inorder successor: it's the smallest value LARGER than node -> maintains BST property.

Time: O(h). Space: O(h) recursion."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _450_DeleteNodeInBST:
    def deleteNode(self, root, key):
        if not root:
            return None
        if key < root.val:
            root.left = self.deleteNode(root.left, key)
        elif key > root.val:
            root.right = self.deleteNode(root.right, key)
        else:
            if not root.left:
                return root.right
            if not root.right:
                return root.left
            successor = root.right
            while successor.left:
                successor = successor.left
            root.val = successor.val
            root.right = self.deleteNode(root.right, succe
ssor.val)
        return root
```

PHASE 5 TEST & DEBUG

"Let me trace deleteNode in BST [5,3,6,2,4,null,7], key=3."

Find key=3. Node 3 has left=2 and right=4.

Case: has both children. Find in-order successor (leftmost of right subtree)=4.

Replace 3's val with 4. Delete 4 from right subtree:

`deleteNode(4-subtree, 4): 4 is leaf -> return None.`
`3(now renamed 4).right=None.`

Tree: 5(4(2,None),6(None,7)) ok

Alternatively: in-order predecessor=2. Replace 3 with 2, delete 2 from left subtree.

WHY successor/predecessor: maintains BST property after deletion.

Case leaf: return None. Case one child: return that child.

BST delete: curr=root. Traverse: curr.left if val<curr.val else curr.right. On match: curr.left=None (leaf). Or: curr.right.next->inorder_successor replaces curr. prev.next=successor.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(h)	**O(h)**
O(log n) balanced, O(n) skewed	recursion

Follow-up: Insert into BST (#701) — simpler, just find the right spot.

Follow-up: if we also need to balance after deletion, AVL (self-balancing BST)/Red-Black tree operations.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #510 | 22 of 45 | 22/45

Inorder Successor in BST II

QUESTION

Given a node in a binary search tree, return the in-order successor of that node in the BST. If that node has no in-order successor, return null.

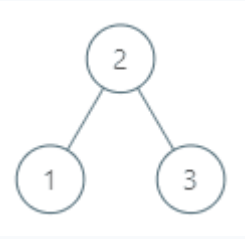
The successor of a node is the node with the smallest key greater than node.val.

You will have direct access to the node but not to the root of the tree. Each node will have a reference to its parent node. Below is the definition for Node:

```
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node parent;
}
```

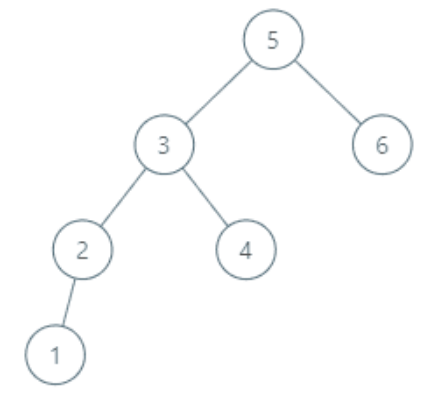
Example 1

Input	`tree = [2,1,3], node = 1`
Output	`2`
Explanation	*1's in-order successor node is 2. Note that both the node and the return value is of Node type.*



Example 2

Input	`tree = [5,3,6,2,4,null,null,1], node = 6`
Output	`null`
Explanation	*There is no in-order successor of the current node, so the answer is null.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 10^5 <= Node.val <= 10^5
- All Nodes will have unique values.
- Follow up: Could you solve it without looking up any of the node's values?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Same as #285 (Inorder Successor in BST) but nodes have a parent pointer.

No root is given. Right?"

"A few quick questions:

If node has a right child, successor is leftmost of right subtree.

If not, go up via parent until we come from a left child? Yes."

"Let me walk: root=[5,3,6,2,4,null,null,1], p=6 -> null ok (rightmost node)

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
go to the root by parent pointers
take the in-order traversal of the whole tree
return the node after the given one
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Traverses everything. If the node has a right child the answer is that subtree's leftmost node; otherwise climb until you come from a left child: O(h) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Parent-pointer walk — already O(h). Walk-through of the two cases:"

Case 1: node has a right child.

Successor = leftmost node of node.right subtree.

Walk: go right once, then left as far as possible.

Case 2: node has NO right child.

Walk UP using parent pointers until we find a node that is a LEFT child of its parent.

That parent is the inorder successor.

Why: going up-left means we came from the left subtree -> parent is the next larger node.

If we keep going up-right, we're still in a subtree where parent < ancestors above.

PHASE 3 OPTIMIZE

Walk-through on BST 3<-5->6, node=3:

No right child. Go up: 3's parent is 5, and 3 is 5's LEFT child -> return 5. ok

Time: O(h). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _510_InorderSuccessorBSTII:
    def inorderSuccessor(self, node):
        if node.right:
            successor = node.right
            while successor.left:
                successor = successor.left
            return successor
        current = node
        while current.parent and current == current.parent
.right:
            current = current.parent
        return current.parent
```

PHASE 5 TEST & DEBUG

"Let me trace inorderSuccessor on BST, node=2 (tree: 5,3,6,2,4)."

Node 2 has no right child -> successor is first ancestor where we went left

Climb: node=2's parent=3. 2 is left child of 3 -> successor = 3 ok

Node with right child (e.g., node=3): successor = leftmost of right subtree = 4 ok

Using parent pointers: if node.right exists -> leftmost(node.right); else climb until came from left

Trace node=6 (rightmost): climb 5, 5 is right child of... no parent above -> no successor -> None ok

WHY parent pointer: O(h) traversal without re-searching tree from root.

WHY use parent pointer: avoids re-traversal from root; O(h) per call with no extra space.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(h)	O(1)

Follow-up: Inorder Predecessor in BST II — mirror logic: left child's rightmost, or first ancestor from right.

Follow-up: LCA (Lowest Common Ancestor) with parent pointers (#1650) — walk both to root, find intersection.

MEDIUM | Trees & BST | LeetCode #545 | 23 of 45 | 23/45

Boundary of Binary Tree

QUESTION

The boundary of a binary tree is the concatenation of the root, the left boundary, the leaves ordered from left-to-right, and the reverse order of the right boundary.

The left boundary is the set of nodes defined by the following:

CONSTRAINTS

- The root node's left child is in the left boundary. If the root does not have a left child, then the left boundary is empty.
- If a node is in the left boundary and has a left child, then the left child is in the left boundary.
- If a node is in the left boundary, has no left child, but has a right child, then the right child is in the left boundary.
- The leftmost leaf is not in the left boundary.

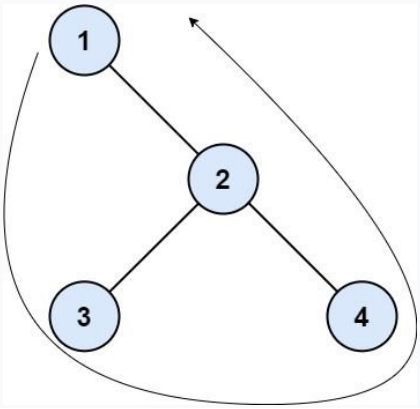
The right boundary is similar to the left boundary, except it is the right side of the root's right subtree. Again, the leaf is not part of the right boundary, and the right boundary is empty if the root does not have a right child.

The leaves are nodes that do not have any children. For this problem, the root is not a leaf.

Given the root of a binary tree, return the values of its boundary.

Example 1

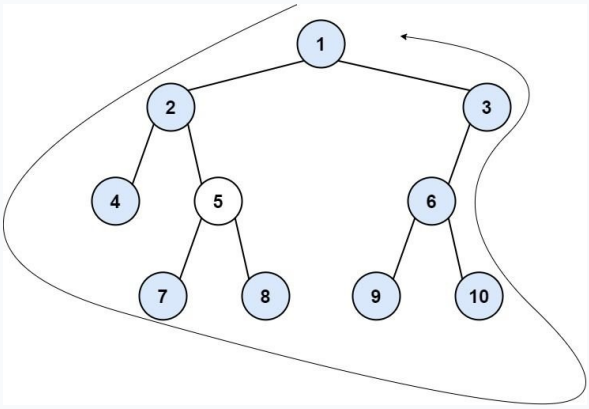
Input	`root = [1,null,2,3,4]`
Output	`[1,3,4,2]`
Explanation	*- The left boundary is empty because the root does not have a left child. - The right boundary follows the path starting from the root's right child 2 -> 4. 4 is a leaf, so the right boundary is [2]. - The leaves from left to right are [3,4]. Concatenating everything results in [1] + [] + [3,4] + [2] = [1,3,4,2].*



QUESTION

Example 2

Input	`root = [1,2,3,4,5,6,null,null,null,7,8,9,10]`
Output	`[1,2,4,7,8,9,10,6,3]`
Explanation	*- The left boundary follows the path starting from the root's left child 2 -> 4. 4 is a leaf, so the left boundary is [2]. - The right boundary follows the path starting from the root's right child 3 -> 6 -> 10. 10 is a leaf, so the right boundary is [3,6], and in reverse order is [6,3]. - The leaves from left to right are [4,7,8,9,10]. Concatenating everything results in [1] + [2] + [4,7,8,9,10] + [6,3] = [1,2,4,7,8,9,10,6,3].*



- The number of nodes in the tree is in the range [1, 10^4].
- 1000 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Boundary = root + left boundary (non-leaves top-down) + all leaves left-to-right

+ right boundary (non-leaves bottom-up). Leftmost/rightmost leaves are in leaves, not boundary. Right?"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
traverse once for the left boundary, once for the
    leaves, once for the right boundary (reversed)
    and join the three lists
```

TIME	SPACE
O(n)	O(h)

NOTE

PHASE 2 BRUTE FORCE

Not slow: three linear passes. It can be done in one traversal with flags telling each node whether it is on the left or right boundary, but the asymptotics are the same.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"The O(n) three-pass approach is already optimal — no further improvement possible."

Let me walk through how each helper builds its part of the boundary:

left_boundary: walk left, prefer left child, until we hit a leaf — collect non-leaves top-down.

leaves: in-order DFS — left subtree leaves first, then right subtree leaves.

right_boundary: mirror of left — walk right, prefer right child, collect non-leaves bottom-up (reversed).

Root is always included. If root IS a leaf, return [root.val] directly.

PHASE 4 CLEAN CODE

```
class _545_BoundaryBinaryTree:
    def boundaryOfBinaryTree(self, root: Optional['TreeNod
e']) -> List[int]:
        if not root:
            return []

        def is_leaf(n): return not n.left and not n.right

        def left_boundary(n):
            res = []
            while n and not is_leaf(n):
                res.append(n.val)
                n = n.left if n.left else n.right
            return res

        def leaves(n):
            if not n:   return []
            if is_leaf(n): return [n.val]
            return leaves(n.left) + leaves(n.right)

        def right_boundary(n):
            res = []
            while n and not is_leaf(n):
                res.append(n.val)
                n = n.right if n.right else n.left
            return res[::-1]

        if is_leaf(root):
            return [root.val]
        return [root.val] + left_boundary(root.left) + lea
ves(root) + right_boundary(root.right)
```

PHASE 5 TEST & DEBUG

"Let me trace boundaryOfBinaryTree on [1,2,3,4,5,6,7]."

Boundary = left boundary (excl. leaves) + leaves (L->R) + right boundary (excl. leaves, reversed)

Tree: root=1, left=2(left=4,right=5), right=3(left=6,right=7)

Left boundary (top-down, excl. leaf): 1,2 -> [1,2]

Leaves (DFS): 4,5,6,7 -> [4,5,6,7]

PHASE 5 TEST & DEBUG

Right boundary (bottom-up, excl. leaf): 3 -> [3] reversed -> [3]

Result: [1,2,4,5,6,7,3] ok

WHY three parts: root, left boundary, leaves, right boundary form the perimeter.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** Three separate traversals, each linear

'Prefer left else right' for left spine; 'prefer right else left' for right spine — reversed at end.

Given more time I'd ask: return boundary clockwise vs counter-clockwise? Just reverse the result.

MEDIUM | Trees & BST | LeetCode #549 | 24 of 45 | 24/45

Binary Tree Longest Consecutive Sequence II

QUESTION

Given the root of a binary tree, return the length of the longest consecutive path in the tree.

A consecutive path is a path where the values of the consecutive nodes in the path differ by one. This path can be either increasing or decreasing.

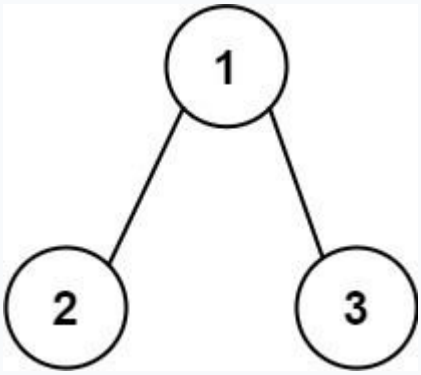
CONSTRAINTS

- For example, [1,2,3,4] and [4,3,2,1] are both considered valid, but the path [1,2,4,3] is not valid.

On the other hand, the path can be in the child-Parent-child order, where not necessarily be parent-child order.

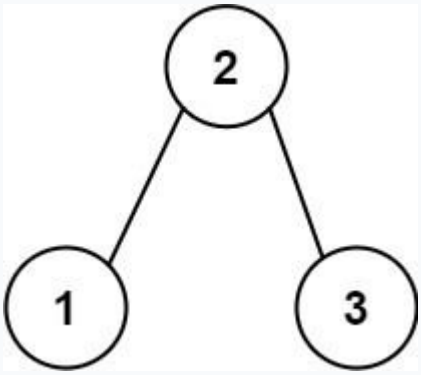
Example 1

Input	`root = [1,2,3]`
Output	`2`
Explanation	*The longest consecutive path is [1, 2] or [2, 1].*



Example 2

Input	`root = [2,1,3]`
Output	`3`
Explanation	*The longest consecutive path is [1, 2, 3] or [3, 2, 1].*



- The number of nodes in the tree is in the range $[1, 3 * 10^4]$.
- $3 * 10^4 <= Node.val <= 3 * 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Longest path where consecutive values differ by 1 — can go up OR down, can change direction at one pivot. Right?"

"[2,1,3] -> [1,2,3] length 3 ok"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    DFS down both children computing the longest
        increasing and decreasing chains from
        that node
    combine them through the node
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Restarts the chain computation from every node. A single post-order pass returning (longest increasing, longest decreasing) chains from each node, combined at the node, is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (post-order returning inc/dec lengths)

"Post-order returning (inc, dec) per node — already O(n). Walk-through:"

`dfs(node) returns (longest increasing chain going DOWN, longest decreasing chain going DOWN):`

null -> (0, 0).

For each child c of node:

(c_inc, c_dec) = dfs(c)

`if c.val == node.val + 1: inc = max(inc, c_inc + 1) <- child is one more -> extends increasing`

`if c.val == node.val - 1: dec = max(dec, c_dec + 1) <- child is one less -> extends decreasing`

Update global answer: answer = max(answer, inc + dec)

<- best path through this node = longest increasing arm + longest decreasing arm,

meeting AT this node (node.val is the peak/transition).

`return (inc, dec) to parent so it can extend further.`

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

```
class _549_LongestConsecutiveII:
    def longestConsecutive(self, root: Optional['TreeNode'
]) -> int:
        best = [0]

        def dfs(node):
            if not node:
                return 0, 0
            l_inc, l_dec = dfs(node.left)
            r_inc, r_dec = dfs(node.right)
            inc = dec = 1
            if node.left:
                if node.left.val == node.val + 1:  inc = m
ax(inc, l_inc + 1)
                if node.left.val == node.val - 1:  dec = m
ax(dec, l_dec + 1)
            if node.right:
                if node.right.val == node.val + 1: inc = m
ax(inc, r_inc + 1)
                if node.right.val == node.val - 1: dec = m
ax(dec, r_dec + 1)
            best[0] = max(best[0], inc + dec - 1)
            return inc, dec

        dfs(root)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me trace longestConsecutive II on [1,2,0,3] (1 has left=2,right=0; 2 has left=3)."

dfs(3): inc=1,dec=1.return(1,1).

dfs(2): child=3. child.val=2+1=3->inc=1+1=2. child.val=2-1=1? No,3!=1. dec=1.

candidate: inc(left)+dec(right) or dec(left)+inc(right). Only left child.

path_through_node = inc_left+dec_right or dec_left+inc_right = max(2,1)+1?

Actually: the path can go up from left and down to right. inc[left]+dec[right] across node.

dfs(1): left=2(inc=2,dec=1), right=0(inc=1,dec=1? 0=1-1->dec extends). ...

Result: longest consecutive path (inc or dec) = 3 (0,1,2 or 1,2,3) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** inc+dec-1 avoids double-counting the pivot node

Generalises #298 by allowing direction change at the pivot node.

Given more time I'd ask: allow path to go up? Needs full graph treatment.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #582 | 25 of 45 | 25/45

Kill Process

QUESTION

You have n processes forming a rooted tree structure. You are given two integer arrays pid and ppid, where pid[i] is the ID of the ith process and ppid[i] is the ID of the ith process's parent process.

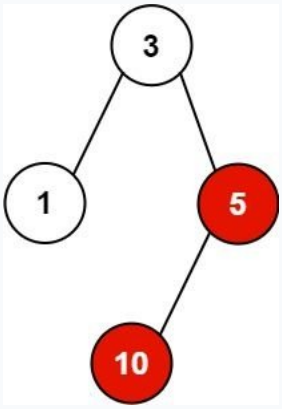
Each process has only one parent process but may have multiple children processes. Only one process has ppid[i] = 0, which means this process has no parent process (the root of the tree).

When a process is killed, all of its children processes will also be killed.

Given an integer kill representing the ID of a process you want to kill, return a list of the IDs of the processes that will be killed. You may return the answer in any order.

Example 1

Input	`pid = [1,3,10,5], ppid = [3,0,5,3], kill = 5`
Output	`[5,10]`
Explanation	*The processes colored in red are the processes that should be killed.*



Example 2

Input	`pid = [1], ppid = [0], kill = 1`
Output	`[1]`

CONSTRAINTS

- n == pid.length
- n == ppid.length
- 1 <= n <= 5 * 10^4
- 1 <= pid[i] <= 5 * 10^4
- 0 <= ppid[i] <= 5 * 10^4
- Only one process has no parent.
- All the values of pid are unique.
- kill is guaranteed to be in pid.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Kill a process and all its descendants. Return all killed process IDs. Right?"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for process in processes:
    walk up its parent chain (searching the
        arrays) to see if an ancestor is the
        killed process
    collect those that are
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Searches for parents repeatedly. Build a map from parent to children once and DFS or BFS from the killed process: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"The BFS from Phase 2 IS already optimal — O(n) time and space."

Walk-through of how it generates the killed list:

Step 1: Build children map — tree[parent].append(child) for each pair.

Step 2: BFS from 'kill': pop process -> append to result -> enqueue all its children.

Each child that gets enqueued then kills ITS children recursively through the queue.

The cascade stops naturally when a process has no children — it's a leaf in the process tree.

BFS guarantees we kill parents before children (level-order), though order in result doesn't matter.

PHASE 4 CLEAN CODE

```python
class _582_KillProcess:
    def killProcess(self, pid: List[int], ppid: List[int],
 kill: int) -> List[int]:
        children: defaultdict = defaultdict(list)
        for child, parent in zip(pid, ppid):
            children[parent].append(child)
        killed = []
        queue  = deque([kill])
        while queue:
            proc = queue.popleft()
            killed.append(proc)
            queue.extend(children[proc])
        return killed
```

PHASE 5 TEST & DEBUG

"Let me trace killProcess on pid=[1,3,10,5], ppid=[3,0,5,3], kill=5."

Build children map: 0->[3], 3->[1,5], 5->[10].

BFS from kill=5: queue=[5]. result=[5].

Pop 5: children=[10]. Enqueue 10. result=[5,10].

Pop 10: no children. Done.

Return [5,10] ok (killing process 5 kills its child 10 too).

DFS alternative: dfs(5): result.append(5). for child in children[5]: dfs(child). -> [5,10].

WHY BFS/DFS with children map: process tree traversal — kill entire subtree rooted at given PID.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Build adjacency list then BFS — standard implicit tree traversal

Given more time I'd ask: kill all ancestors too? Walk parent pointers upward instead.

MEDIUM | Trees & BST | LeetCode #652 | 26 of 45 | 26/45

Find Duplicate Subtrees

QUESTION

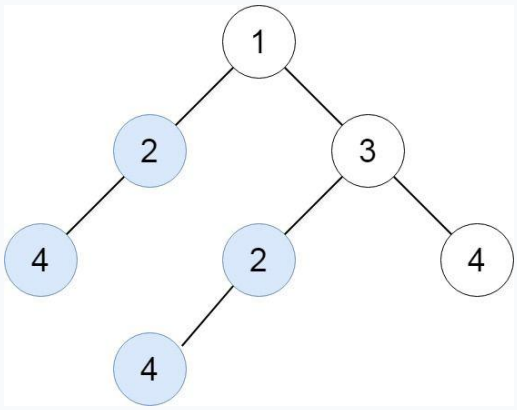
Given the root of a binary tree, return all duplicate subtrees.

For each kind of duplicate subtrees, you only need to return the root node of any one of them.

Two trees are duplicate if they have the same structure with the same node values.

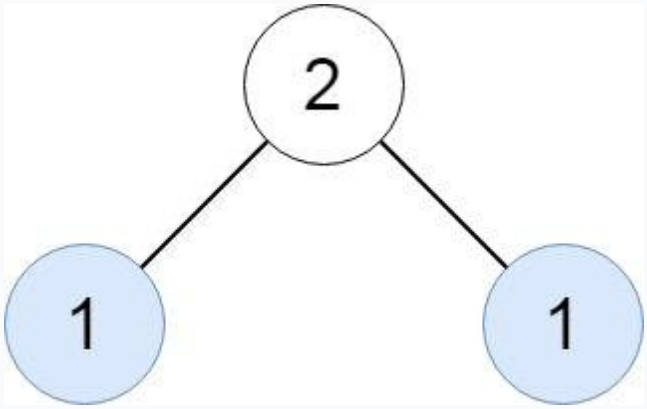
Example 1

Input	`root = [1,2,3,4,null,2,4,null,null,4]`
Output	`[[2,4],[4]]`



Example 2

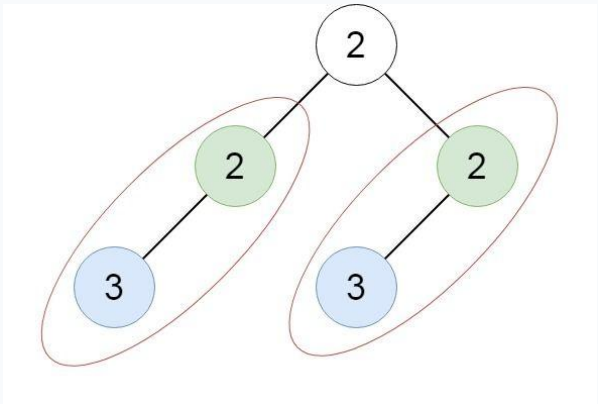
Input	`root = [2,1,1]`
Output	`[[1]]`



Example 3

Input	`root = [2,2,2,3,null,3,null]`
Output	`[[2,3],[3]]`

QUESTION



CONSTRAINTS

- The number of the nodes in the tree will be in the range [1, 5000]
- 200 <= Node.val <= 200

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find all roots of duplicate subtrees. Two subtrees are duplicates if they have the same structure and values. Right?"

"A few quick questions:

If a subtree appears 3 times, include its root only once? Yes — any one occurrence."

"Let me walk: root=[1,2,3,4,null,2,4,null,null,4] -> duplicates: [2,4] and [4] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a, b) in pairs(nodes):
    compare the two subtrees node by node
    collect one root for each group of equal subtrees
```

TIME	SPACE
O(n^3)	O(h)

WHY IT IS SLOW

O(n^2) pairs, each compared in O(n). Serialise each subtree in post-order into a string key (or assign ids to unique (left id, value, right id) triples) and count in a hash map: O(n) with ids.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order DFS: serialize each subtree, use hash map to find duplicates — O(n). Walk-through:"

`def serialize(node):`

If node is None: return '#'.

PHASE 3 OPTIMIZE

`serial = serialize(node.left) + ',' + serialize(node.right) + ',' + str(node.val).`

count[serial]++.

If count[serial] == 2: result.append(node). <- found a duplicate subtree

`return serial.`

`serialize(root).`

WHY post-order: build serialization bottom-up (leaves first, then include in parent's string).

WHY append at count==2 (not every time): we add the node once when first seen as duplicate.

HOW serialization is unique: the format 'left,right,val' uniquely identifies a tree structure.

Time: O(n²) with strings or O(n) with interning. Space: O(n²) strings.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict
class _652_FindDuplicateSubtrees:
    def findDuplicateSubtrees(self, root):
        tree_count = defaultdict(int)
        result = []
        def serialize(node):
            if not node:
                return '#'
            serial = f'{node.val},{serialize(node.left)},{
serialize(node.right)}'
            tree_count[serial] += 1
            if tree_count[serial] == 2:
                result.append(node)
            return serial
        serialize(root)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace findDuplicateSubtrees on [1,2,3,4,null,2,4,null,null,4]."

`serialize(4)='#,#,4'. count['#,#,4']=1.`

`serialize(2-left)='#,#,4,#,2'... actually bottom-up:`

leaf 4: serial='#,#,4'. count=1.

Node 2 (left subtree): left_serial='#,#,4'. right='#'. serial='#,#,4,#,2'. count=1.

Leaf 4 (right of 3): same serial '#,#,4'. count=2 -> append node(4) to result.

Node 2 (right of 3): left_serial='#,#,4'. right='#'. serial='#,#,4,#,2'. count=2 -> append node(2).

Node 3: serial='#,#,4,#,2,#,#,4,#,3'. count=1.

Result: [node(4), node(2)] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²) for string building worst case; O(n) with integer IDs	**O(n)** for the map and result

Follow-up: Find All Duplicate Subtrees and their count — track count separately.

Follow-up: if values can be any string, use hashing to compress serializations.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #662 | 27 of 45 | 27/45

Maximum Width of Binary Tree

QUESTION

Given the root of a binary tree, return the maximum width of the given tree.

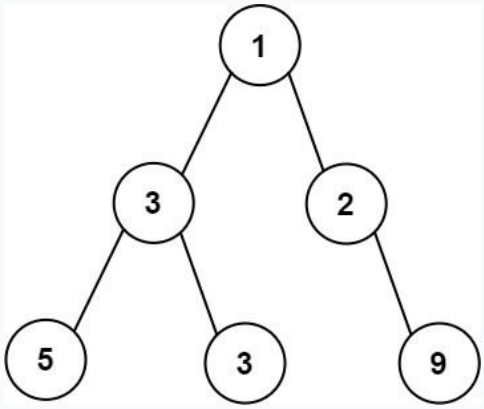
The maximum width of a tree is the maximum width among all levels.

The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is guaranteed that the answer will in the range of a 32-bit signed integer.

Example 1

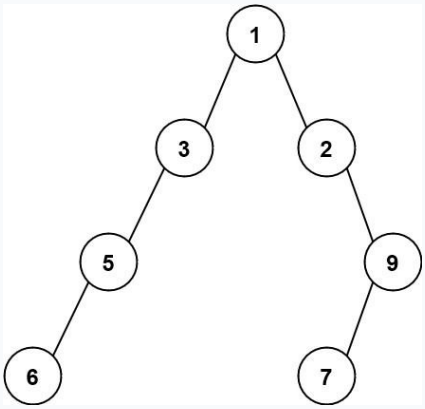
Input	`root = [1,3,2,5,3,null,9]`
Output	`4`
Explanation	*The maximum width exists in the third level with length 4 (5,3,null,9).*



Example 2

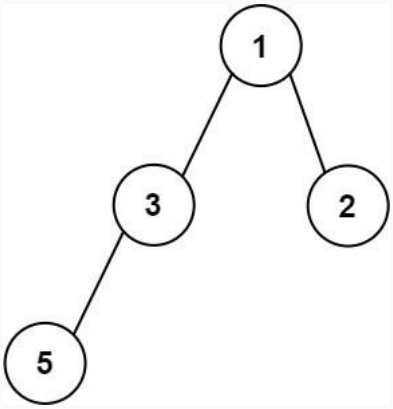
Input	`root = [1,3,2,5,null,null,9,6,null,7]`
Output	`7`
Explanation	*The maximum width exists in the fourth level with length 7 (6,null,null,null,null,null,7).*

QUESTION



Example 3

Input	`root = [1,3,2,5]`
Output	`2`
Explanation	*The maximum width exists in the second level with length 2 (3,2).*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 3000].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Width = rightmost_pos - leftmost_pos + 1 at each level, counting null gaps.

Left child = 2*pos, right child = 2*pos+1. Normalise each level to prevent overflow. Right?"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for d in ds:
    DFS the whole tree to find the leftmost and
        rightmost non-null node at that depth
        with their full-tree positions
    width = right - left + 1
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

Rescans the tree per depth, and full-tree positions overflow for deep trees. One BFS carrying each node's index (2i, 2i + 1), normalised by subtracting the level's first index, is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS with position tracking IS already the optimal approach — O(n) time."

Here's how the position assignment generates widths:

A node at position p has left child at 2p and right child at 2p+1.

At each BFS level, we track the FIRST and LAST position seen.

Width = last_pos - first_pos + 1 (counts null gaps between leftmost and rightmost live nodes).

KEY: normalise positions by subtracting first_pos at each level to prevent integer overflow

in deep trees — otherwise positions grow as 2^depth and overflow 64-bit integers.

PHASE 4 CLEAN CODE

```python
class _662_MaxWidthBinaryTree:
    def widthOfBinaryTree(self, root: Optional['TreeNode']
) -> int:
        if not root:
            return 0
        max_w = 0
        queue = deque([(root, 0)])
        while queue:
            size      = len(queue)
            first_pos = queue[0][1]
            last_pos  = 0
            for _ in range(size):
                node, pos = queue.popleft()
                pos -= first_pos                    # norm
alise
                last_pos = pos
                if node.left:  queue.append((node.left,  2
 * pos))
                if node.right: queue.append((node.right, 2
 * pos + 1))
            max_w = max(max_w, last_pos + 1)
        return max_w
```

PHASE 5 TEST & DEBUG

"Let me trace widthOfBinaryTree on [1,3,2,5,3,null,9]."

BFS with position indexing: root at pos=0. left child at 2*pos+1, right at 2*pos+2.

Wait: use left=2*pos, right=2*pos+1 to avoid overflow. Or use normalized positions.

Level 0: [(root,0)]. width = 0-0+1 = 1.

Level 1: [(3, 0*2=0), (2, 0*2+1=1)]. width = 1-0+1 = 2.

Level 2: [(5, 0*2=0), (3, 0*2+1=1), (9, 1*2+1=3)].

Normalize by subtracting leftmost=0: positions [0,1,3]. width = 3-0+1 = 4.

Answer: 4 ok (null node between 3 and 9 counts toward width).

WHY normalize: prevents integer overflow at deep levels (offsets from leftmost node).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(w) Normalisation prevents 2^depth overflow for deep trees — critical

Given more time I'd ask: width by actual node count (ignoring nulls)? Just max level_size in BFS.

MEDIUM | Trees & BST | LeetCode #669 | 28 of 45 | 28/45

Trim a Binary Search Tree

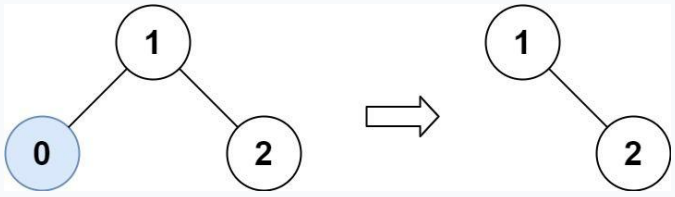
QUESTION

Given the root of a binary search tree and the lowest and highest boundaries as low and high, trim the tree so that all its elements lies in [low, high]. Trimming the tree should not change the relative structure of the elements that will remain in the tree (i.e., any node's descendant should remain a descendant). It can be proven that there is a unique answer.

Return the root of the trimmed binary search tree. Note that the root may change depending on the given bounds.

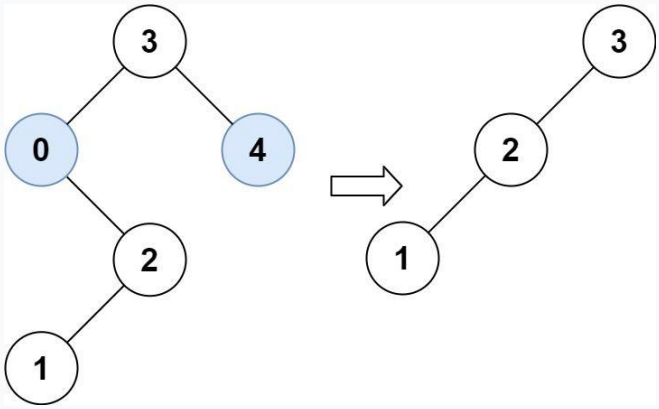
Example 1

Input	`root = [1,0,2], low = 1, high = 2`
Output	`[1,null,2]`



Example 2

Input	`root = [3,0,4,null,2,null,null,1], low = 1, high = 3`
Output	`[3,2,null,1]`



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 0 <= Node.val <= 10^4
- The value of each node in the tree is unique.
- root is guaranteed to be a valid binary search tree.
- 0 <= low <= high <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Trim a BST so all values lie in [low, high]. Return the trimmed root. Right?"

"A few quick questions:

Nodes outside [low, high] are deleted entirely (not just the node, subtree may be kept)? Yes."

"Let me walk: root=[3,0,4,null,2,null,null,1], low=1, high=3 -> [3,2,null,1] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = every value in [low, high] collected by
    traversal
rebuild a BST by inserting them
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

Rebuilds the tree from scratch (O(n^2) if the inserts skew it). Recurse: if the node is below low, return the trimmed right subtree; if above high, the trimmed left subtree; else trim both: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Recursive BST trim — already O(n). Walk-through of how the trim propagates:"

`trim(node, lo, hi):`

If node is None -> return None.

If node.val < lo -> return trim(node.right, lo, hi)

<- entire left subtree is also < lo (BST property), so skip this node AND its left.

Right subtree might have valid nodes.

If node.val > hi -> return trim(node.left, lo, hi)

<- entire right subtree is also > hi. Skip this node AND its right.

```
node.left = trim(node.left, lo, hi)
node.right = trim(node.right, lo, hi)
return node <- this node is in range; keep it, with
  trimmed subtrees.
```

WHY no explicit deletion: we return the trimmed subtree from each call;

the parent just replaces its child pointer with whatever we return.

Time: O(n) worst case (every node visited). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _669_TrimBST:
    def trimBST(self, root, low, high):
        if not root:
            return None
        if root.val < low:
            return self.trimBST(root.right, low, high)
        if root.val > high:
            return self.trimBST(root.left, low, high)
        root.left = self.trimBST(root.left, low, high)
        root.right = self.trimBST(root.right, low, high)
        return root
```

PHASE 5 TEST & DEBUG

"Let me trace trimBST on [3,0,4,null,2,null,null,null,null,1], low=1,high=3."

Actually: [3,0,4,null,2], with 2.left=1.

trim(3): 3 in [1,3] ok. trim(0,1,3): 0<low=1->return trim(0.right=2): 2 in [1,3]ok.

trim(2.left=1): 1 in [1,3]ok. trim(1.left=None)=None. trim(1.right=None)=None. return 1.

trim(2.right=None)=None. return node(2,left=1).

3.left = node(2,left=1).

trim(4,1,3): 4>high=3->return trim(4.left=None)=None.

3.right = None. Return 3(left=2(left=1)) ok

WHY return trim(child) when out of range: that subtree might still have valid nodes.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)**
visit each node once	recursion

Follow-up: Range Sum of BST (#938) — sum values in range; same BST traversal with pruning.

Follow-up: if tree is not a BST, must visit all nodes regardless of value.

MEDIUM | Trees & BST | LeetCode #687 | 29 of 45 | 29/45

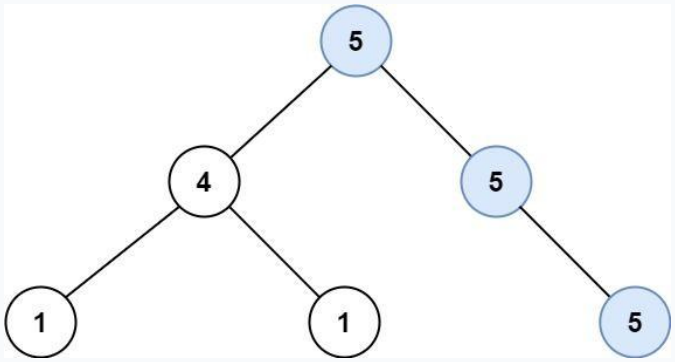
Longest Univalue Path

QUESTION

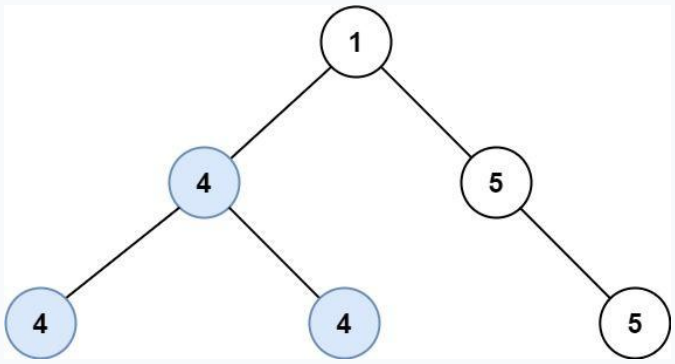
Given the root of a binary tree, return the length of the longest path, where each node in the path has the same value. This path may or may not pass through the root.

The length of the path between two nodes is represented by the number of edges between them.

Example 1	
Input	`root = [5,4,5,1,1,null,5]`
Output	2
Explanation	*The shown image shows that the longest path of the same value (i.e. 5).*



Example 2	
Input	`root = [1,4,5,4,4,null,5]`
Output	2
Explanation	*The shown image shows that the longest path of the same value (i.e. 4).*



CONSTRAINTS

- The number of nodes in the tree is in the range [0, 10^4].
- 1000 <= Node.val <= 1000
- The depth of the tree will not exceed 1000.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the longest path where all nodes have the same value.

Path doesn't need to pass through root; can go through any node. Return edge count. Right?"

"A few quick questions:

Single node -> 0 edges? Yes. Path through a node can go in both directions (left and right)? Yes."

"Let me walk: root=[5,4,5,1,1,5] -> longest univalue path through root=5: left has 5,right has 5 -> length 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for top in tops:
    # of the path
    DFS down both sides counting nodes with the
        same value
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Restarts a walk from every node. One post-order pass returning the longest same-value downward arm from each node, combined at the node (left arm + right arm), is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS post-order: longest univalue path through each node — O(n). Walk-through:"

`def dfs(node):`

If node is None: return 0.

`left = dfs(node.left). right = dfs(node.right).`

`left_path = (left + 1) if node.left and node.left.val == node.val else 0.`

`right_path = (right + 1) if node.right and node.right.val == node.val else 0.`

`ans = max(ans, left_path + right_path). <- path bending at this node`

`return max(left_path, right_path). <- longest single-direction univalue path for parent`

WHY reset to 0 when values differ: the univalue path breaks at a value change.

WHY left_path + right_path as candidate: the longest path may go left AND right through this node.

WHY return max of two directions: parent can only extend one direction.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _687_LongestUnivaluePath:
    def longestUnivaluePath(self, root):
        best = [0]
        def dfs(node):
            if not node:
                return 0
            left = dfs(node.left)
            right = dfs(node.right)
            left_arm = (left + 1) if node.left and node.le
ft.val == node.val else 0
            right_arm = (right + 1) if node.right and node
.right.val == node.val else 0
            best[0] = max(best[0], left_arm + right_arm)
            return max(left_arm, right_arm)
        dfs(root)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me trace longestUnivaluePath on [5,4,5,1,1,null,5]."

dfs(1-left): leaf, return 0. dfs(1-right): leaf, return 0.

dfs(4): child vals (1,1)!=4. left_path=0,right_path=0. candidate=0+0=0. return max(0,0)=0.

dfs(5-right): leaf -> return 0.

dfs(5-root.right): child val 5==5->right_path=0+1=1. no left child. candidate=0+1=1. return 1.

dfs(5-root): left child=4!=5->left_path=0. right child=5==5->right_path=dfs(5-right)+1=2.

candidate=0+2=2. ans=2. return max(0,2)=2.

Answer: 2 ok (path 5->5->? Wait, root.right is 5, its right child is 5 too -> path 5-5-5 = length 2)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

Follow-up: Diameter of Binary Tree (#543) — same pattern without value matching.

Follow-up: if values can be any type (not just ints), same algorithm, just change comparison.

MEDIUM | Trees & BST | LeetCode #690 | 30 of 45 | 30/45

Employee Importance

QUESTION

You have a data structure of employee information, including the employee's unique ID, importance value, and direct subordinates' IDs.

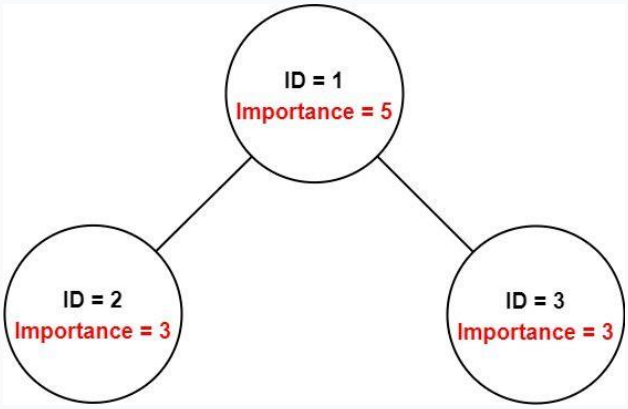
You are given an array of employees employees where:

CONSTRAINTS

- employees[i].id is the ID of the ith employee.
- employees[i].importance is the importance value of the ith employee.
- employees[i].subordinates is a list of the IDs of the direct subordinates of the ith employee.

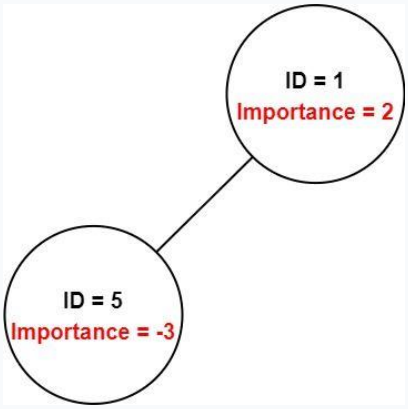
Given an integer id that represents an employee's ID, return the total importance value of this employee and all their direct and indirect subordinates.

Example 1	
Input	`employees = [[1,5,[2,3]],[2,3,[]],[3,3,[]]], id = 1`
Output	`11`
Explanation	*Employee 1 has an importance value of 5 and has two direct subordinates: employee 2 and employee 3. They both have an importance value of 3. Thus, the total importance value of employee 1 is 5 + 3 + 3 = 11.*



Example 2	
Input	`employees = [[1,2,[5]],[5,-3,[]]], id = 5`
Output	`-3`
Explanation	*Employee 5 has an importance value of -3 and has no direct subordinates. Thus, the total importance value of employee 5 is -3.*

QUESTION



- 1 <= employees.length <= 2000
- 1 <= employees[i].id <= 2000
- All employees[i].id are unique.
- 100 <= employees[i].importance <= 100
- One employee has at most one direct leader and may have several subordinates.
- The IDs in employees[i].subordinates are valid IDs.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each employee has an importance value and a list of direct subordinates.

Return total importance of an employee and all their subordinates (recursively). Right?"

"A few quick questions:

Can have any depth of hierarchy? Yes. Circular references? No."

"Let me walk: [[1,5,[2,3]],[2,3,[]],[3,3,[]]], id=1 -> 5+3+3=11 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
total = 0
for id in ids:
    scan the entire employee array to find that
        employee's record
    add its importance and recurse
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

A linear scan for every lookup. Put the employees in a hash map by id once, then DFS from the given id: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS accumulating importance — already O(n). Walk-through:"

Build id_to_employee hashmap for O(1) lookup (avoids repeated linear search).

```
dfs(id):
emp = id_to_employee[id]
total = emp.importance
for subordinate_id in emp.subordinates:
total += dfs(subordinate_id)
```

<- recursively sum subordinate's importance + ALL of their subordinates.

```
return total
```

HOW the recursion accumulates: each call returns the total importance of its entire subtree

(the employee + all their direct and indirect reports).

The caller adds that to its own importance.

WHY hashmap: without it, finding an employee by ID requires O(n) scan per lookup -> O(n²) total.

Time: O(n). Space: O(n) map + O(n) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _690_EmployeeImportance:
    def getImportance(self, employees, id):
        emp_map = {e.id: e for e in employees}
        def dfs(emp_id):
            emp = emp_map[emp_id]
            return emp.importance + sum(dfs(sub_id) for su
b_id in emp.subordinates)
        return dfs(id)
```

PHASE 5 TEST & DEBUG

"Let me trace getImportance on employees [(1,5,[2,3]),(2,3,[]),(3,3,[])], id=1."

Build map: 1->Employee(5,[2,3]), 2->Employee(3,[]), 3->Employee(3,[]).

DFS from id=1: total=5. Subordinates=[2,3].

DFS(2): total+=3. Subordinates=[]. return.

DFS(3): total+=3. return.

Total=5+3+3=11 ok

BFS: queue=[1]. total=0. Pop 1->total+=5. Enqueue [2,3]. Pop 2->total+=3. Pop 3->total+=3. total=11 ok

WHY DFS/BFS on employee tree: importance = sum of self + all direct/indirect subordinates.

Recursion on subordinates naturally sums the whole subtree.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n) for hashmap + recursion

BFS alternative: use a queue, same O(n) time and space.

Follow-up: if hierarchy has cycles, add a visited set.

Follow-up: return the employee with maximum importance — track during traversal.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #701 | 31 of 45 | 31/45

Insert into a Binary Search Tree

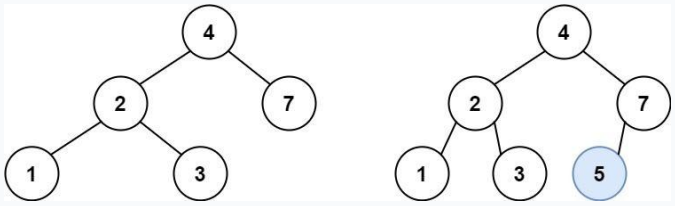
QUESTION

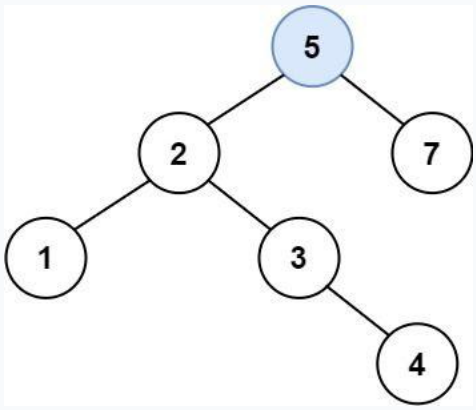
You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.

Example 1

Input	`root = [4,2,7,1,3], val = 5`
Output	`[4,2,7,1,3,5]`
Explanation	*Another accepted tree is:*





Example 2

Input	`root = [40,20,60,10,30,50,70], val = 25`
Output	`[40,20,60,10,30,50,70,null,null, 25]`

Example 3

Input	`root = [4,2,7,1,3,null,null,null ,null,null,null], val = 5`
Output	`[4,2,7,1,3,5]`

CONSTRAINTS

- The number of nodes in the tree will be in the range [0, 10^4].
- 10^8 <= Node.val <= 10^8
- All the values Node.val are unique.
- 10^8 <= val <= 10^8
- It's guaranteed that val does not exist in the original BST.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Insert val into a BST such that it remains a valid BST. Return the root. Right?"

"A few quick questions:

val guaranteed not to exist in the BST? Yes per constraints."

"Let me walk: root=[4,2,7,1,3], val=5 -> [4,2,7,1,3,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = the tree's values as a sorted list
insert the new value
rebuild a balanced BST from the list
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Rebuilds the whole tree. Walk down from the root (left if smaller, right if larger) and attach the new node at the empty spot: O(h) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BST insertion — O(h), walk-through:"

`insert(node, val):`

If node is None -> return TreeNode(val) <- found the insertion point.

If val < node.val:

`node.left = insert(node.left, val) <- recurse left; attach result as new left`

`else:`

`node.right = insert(node.right, val) <- recurse right; attach result as new right`

`return node <- return current node unchanged (or with updated child)`

HOW insertion works: we recurse until we hit a None, create the node there, and return it.

The parent gets the new node as its child via 'node.left = insert(node.left, val)'.

WHY this maintains BST property: we only go left when val < node.val, ensuring

the new node lands in a position consistent with BST ordering.

Time: O(h). Space: O(h) recursion."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _701_InsertIntoBST:
    def insertIntoBST(self, root, val):
        if not root:
            return TreeNode(val)
        cur = root
        while True:
            if val < cur.val:
                if not cur.left:
                    cur.left = TreeNode(val); break
                cur = cur.left
            else:
                if not cur.right:
                    cur.right = TreeNode(val); break
                cur = cur.right
        return root
```

PHASE 5 TEST & DEBUG

"Let me trace insertIntoBST on [4,2,7,1,3], val=5."

node=4: 5>4 -> go right. node=7: 5<7 -> go left. node=None -> return new TreeNode(5).

7.left = TreeNode(5). Return 4(2(1,3),7(5,None)) ok

val=0:

node=4: 0<4->left. node=2: 0<2->left. node=1: 0<1->left->None->return TreeNode(0).

1.left=TreeNode(0). ok

WHY always insert as a leaf: BST insertion never restructures existing nodes — find the

correct BST position (always None slot) and place new node there.

WHY recursive returns node: propagates the new/existing node up to reconnect parent.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(h)	**O(1)** iterative vs O(h) recursive

Follow-up: Delete from BST (#450) — more complex due to rebalancing.

Follow-up: if BST must stay balanced, use AVL (self-balancing BST) or Red-Black tree rotation.

MEDIUM | Trees & BST | LeetCode #863 | 32 of 45 | 32/45

All Nodes Distance K in Binary Tree

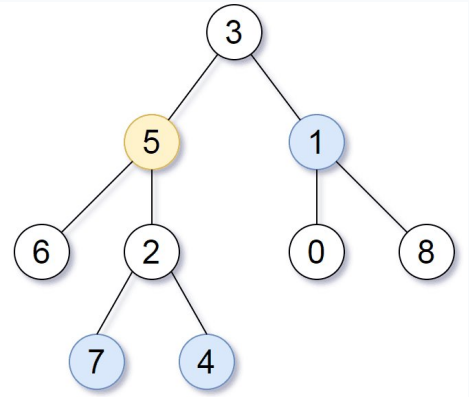
QUESTION

Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node.

You can return the answer in any order.

Example 1

Input	`root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2`
Output	`[7,4,1]`
Explanation	*The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.*



Example 2

Input	`root = [1], target = 1, k = 3`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [1, 500].
- 0 <= Node.val <= 500
- All the values Node.val are unique.
- target is the value of one of the nodes in the tree.
- 0 <= k <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find all nodes exactly k edges from target. Distance can go upward through parent. Right?"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    compute its distance to the target by finding
        the path between them
    collect those at distance k
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

A path search per node. Record each node's parent in a map, then BFS outward from the target (children and parent) k levels: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (build parent map then BFS)

"Build parent map via DFS, then BFS from target. Already O(n). Walk-through:"

Step 1 — DFS to build parent dict:

`dfs(node, parent): parent[node] = parent; recurse into children.`

Now every node knows its parent -> tree becomes undirected graph.

Step 2 — BFS from target node, distance starts at 0:

Queue: [(target, None)] so we don't go back to target's parent immediately.

Each BFS level = 1 edge traversal (1 step in the tree, up or down).

When level count reaches k: all nodes currently in the queue are at distance k -> collect.

Explore 3 neighbours per node: left child, right child, parent node.

visited set prevents going back and re-visiting nodes.

WHY we need parent map: normally trees are top-down only; adding parent pointers

lets us travel UP the tree, enabling BFS from ANY node.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _863_DistanceK:
    def distanceK(self, root: 'TreeNode', target: 'TreeNod
e', k: int) -> List[int]:
        parent: dict = {}

        def build(node, par):
            if not node: return
            parent[node] = par
            build(node.left, node)
            build(node.right, node)

        build(root, None)
        visited = {target}
        queue   = deque([(target, 0)])
        result  = []
```

```
        while queue:
            node, dist = queue.popleft()
            if dist == k:
                result.append(node.val)
                continue
            for nb in [node.left, node.right, parent[node]
]:
                if nb and nb not in visited:
                    visited.add(nb)
                    queue.append((nb, dist + 1))
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace allNodesDistanceK on tree=[3,5,1,6,2,0,8,null,null,7,4], targetNode=5, k=2."

Build parent pointers (or adjacency): each node knows its parent.

BFS from target node 5 in the undirected tree (treat as graph):

queue=[(5,0)]. visited={5}.

distance 1: neighbors of 5: parent=3, left=6, right=2. queue=[(3,1),(6,1),(2,1)]. visited+={3,6,2}.

distance 2: neighbors of 3: parent=None,left=5(visited),right=1. queue=[(1,2)]. Of 6: leaf. Of 2: left=7,right=4. queue+=[(7,2),(4,2)]. distance=k=2.

At distance k: collect [1,7,4] ok Wait, also add 6? No, 6 is at distance 1, not 2. -> result=[6? No].

result=[7,4,1] ok. WHY treat as undirected: can traverse up to parent as well as down.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Parent map converts tree to undirected graph — then BFS is trivial

Given more time I'd ask: multiple targets? Multi-source BFS — enqueue all at distance 0.

MEDIUM | Trees & BST | LeetCode #958 | 33 of 45 | 33/45

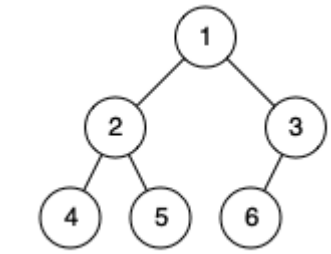
Check Completeness of a Binary Tree

QUESTION

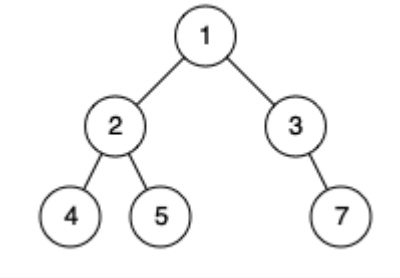
Given the root of a binary tree, determine if it is a complete binary tree.

In a complete binary tree, every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible. It can have between 1 and 2^h nodes inclusive at the last level h.

Example 1	
Input	`root = [1,2,3,4,5,6]`
Output	`true`
Explanation	*Every level before the last is full (ie. levels with node-values {1} and {2, 3}), and all nodes in the last level ({4, 5, 6}) are as far left as possible.*



Example 2	
Input	`root = [1,2,3,4,5,null,7]`
Output	`false`
Explanation	*The node with value 7 isn't as far left as possible.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 100].
- 1 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Check if a binary tree is a complete binary tree: all levels full except possibly the last,

which is filled from left to right. Right?"

"A few quick questions:

Empty tree is complete? Yes. Single node? Yes."

"Let me walk: root=[1,2,3,4,5,6] -> complete ok.
root=[1,2,3,4,5,null,7] -> not complete

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
number each node as in an array heap (root 0,
    children 2i + 1 and 2i + 2)
the tree is complete exactly when the numbers used
    are 0 .. n - 1
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but stores all positions. A level-order BFS that flags the first missing child and fails if any real node appears afterwards is O(n) time and O(width) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS with 'end flag' — already O(n). Walk-through:"

A complete binary tree: all levels full except possibly the last, filled left-to-right.

BFS property: if we ever enqueue a null child BEFORE all real nodes, it's not complete.

`end = False initially.`

For each node popped from the BFS queue:

Enqueue left and right children (can be None — we enqueue None too).

When we pop a None:

`end = True <- we've seen the 'end' of the complete tree.`

If end is True and we pop a non-None node:

`return False <- a real node appeared after a null -> not complete.`

Equivalently: BFS, stop at first None in the queue — if any more non-None follow, not complete.

Time: O(n). Space: O(w).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import deque
class _958_CheckCompletenessBinaryTree:
    def isCompleteTree(self, root):
        queue = deque([root])
        seen_null = False
        while queue:
            node = queue.popleft()
            if not node:
                seen_null = True
                continue
            if seen_null:
                return False
            queue.append(node.left)
            queue.append(node.right)
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace isCompleteTree on [1,2,3,4,5,6] and [1,2,3,4,5,null,7]."

BFS: process nodes level by level.

[1,2,3,4,5,6]: queue=[1]. Process: enqueue all children. Never see None before end -> True ok

[1,2,3,4,5,null,7]:

queue=[1]. Pop 1->enqueue 2,3. Pop 2->enqueue 4,5. Pop 3->enqueue None,7.

After seeing None (3.left=None): next node must also be None. But 7 is not None -> False ok

WHY 'end flag': once we see the first None in BFS order, all subsequent slots must be None.

A complete tree's None nodes occur only at the end of the last level.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(w)** max queue width

Alternative: number nodes 1..n; check that indexed BFS gives exactly n nodes.

Follow-up: Count Complete Tree Nodes (#222) — uses completeness for O(log²n) counting.

Follow-up: insert into complete binary tree — find the right position using node numbering.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #979 | 34 of 45 | 34/45

Distribute Coins in Binary Tree

QUESTION

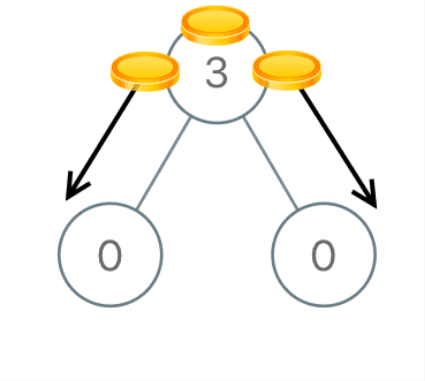
You are given the root of a binary tree with n nodes where each node in the tree has node.val coins. There are n coins in total throughout the whole tree.

In one move, we may choose two adjacent nodes and move one coin from one node to another. A move may be from parent to child, or from child to parent.

Return the minimum number of moves required to make every node have exactly one coin.

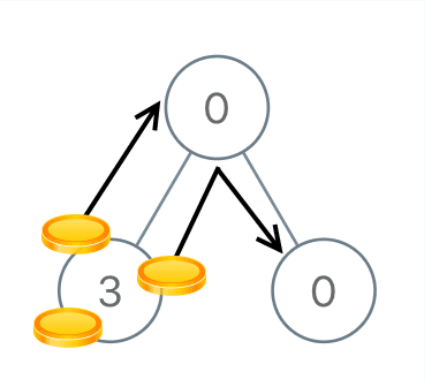
Example 1

Input	`root = [3,0,0]`
Output	`2`
Explanation	*From the root of the tree, we move one coin to its left child, and one coin to its right child.*



Example 2

Input	`root = [0,3,0]`
Output	`3`
Explanation	*From the left child of the root, we move two coins to the root [taking two moves]. Then, we move one coin from the root of the tree to the right child.*



CONSTRAINTS

- The number of nodes in the tree is n.

QUESTION

- 1 <= n <= 100
- 0 <= Node.val <= n
- The sum of all Node.val is n.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Move coins so each node has exactly 1 coin. Coins can move along edges.

Return total number of moves (each edge traversal = 1 move). Right?"

"A few quick questions:

Total coins == number of nodes (guaranteed)? Yes.

Each move sends 1 coin along 1 edge? Yes."

"Let me walk: root=[3,0,0] -> move 2 coins from root to children -> 2 moves ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
simulate every sequence of single-coin moves between
    adjacent nodes until each node has exactly one
    coin
best = min(best, sequence)
```

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

Searches move sequences. Every coin crossing an edge counts once: in post-order, each node's excess (coins - 1 plus its children's excess) is moved up, so the answer is the sum of absolute excess: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is recomputing excess for each node.

Post-order DFS: each node returns its excess (coins - 1 + children excesses).

The flow through an edge = |excess of the subtree below it|.

Accumulate abs(excess) for each edge = each non-root node.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _979_DistributeCoinsInBinaryTree:
    def distributeCoins(self, root):
        total_moves = [0]
        def dfs(node):
            if not node:
                return 0
            left_excess = dfs(node.left)
            right_excess = dfs(node.right)
            total_moves[0] += abs(left_excess) + abs(right
_excess)
            return node.coins + left_excess + right_excess
 - 1
        dfs(root)
        return total_moves[0]
```

PHASE 5 TEST & DEBUG

"Let me trace distributeCoins on [3,0,0]."

dfs(0,left-leaf): excess=0-1=-1 (needs 1 coin). return -1.

dfs(0,right-leaf): excess=0-1=-1. return -1.

dfs(3): excess from children: -1 + -1 = -2. excess at node = 3-1 + (-2) = 0.

moves += |left_excess| + |right_excess| = 1 + 1 = 2. return 0.

Total moves=2 ok (3's 2 extra coins each travel 1 edge to the leaves).

WHY |excess|: each unit of excess (positive or negative) requires 1 move along that edge.

WHY post-order: child excesses are resolved before propagating up to parent.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

The |excess| = moves through that edge — each unit of excess means one coin crossing.

Follow-up: if move cost varies per edge, weight by edge costs in the DFS.

Follow-up: Binary Tree Cameras (#968) — similar post-order tree DP pattern.

MEDIUM | Trees & BST | LeetCode #1026 | 35 of 45 | 35/45

Maximum Difference Between Node and Ancestor

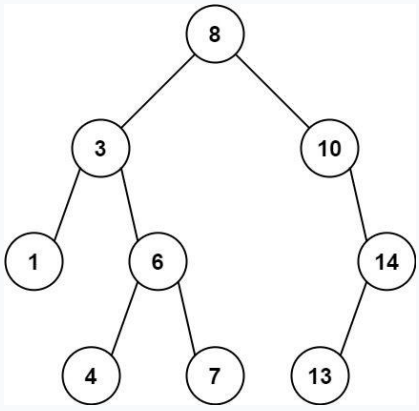
QUESTION

Given the root of a binary tree, find the maximum value v for which there exist different nodes a and b where v = |a.val - b.val| and a is an ancestor of b.

A node a is an ancestor of b if either: any child of a is equal to b or any child of a is an ancestor of b.

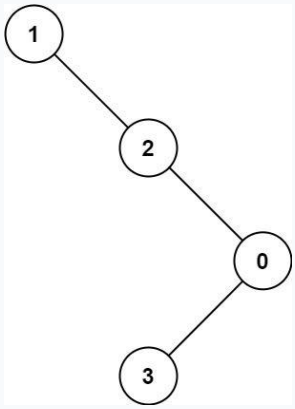
Example 1

Input	`root = [8,3,10,1,6,null,14,null, null,4,7,13]`
Output	7
Explanation	*We have various ancestor-node differences, some of which are given below :* *|8 - 3| = 5* *|3 - 7| = 4* *|8 - 1| = 7* *|10 - 13| = 3* *Among all possible differences, the maximum value of 7 is obtained by |8 - 1| = 7.*



Example 2

Input	`root = [1,null,2,null,0,3]`
Output	3



CONSTRAINTS

- The number of nodes in the tree is in the range [2, 5000].

QUESTION

- 0 <= Node.val <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the maximum difference v - u where v is a descendant of u and |v - u| is maximised.

Actually: max(|node.val - ancestor.val|) over all ancestor-descendant pairs. Right?"

"A few quick questions:

'Ancestor' includes the node itself? Yes — a node is its own ancestor. But we need proper ancestor for difference.

Actually: find max |a - b| where a is an ancestor of b."

"Let me walk: root=[8,3,10,1,6,null,14,null,null,4,7] -> max diff = |3-7|=4? No: max=|8-1|=7 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    examine all its descendants and take the
        largest difference with its value
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Rescans a whole subtree for every node. Pass the minimum and maximum seen along the path down; at a leaf return max - min: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Pre-order DFS passing min and max ancestor values — already O(n). Walk-through:"

`dfs(node, cur_min, cur_max):`

If node is None -> return 0.

<- compute max difference involving this node and all ancestors seen so far.

```
diff = max(abs(node.val - cur_min), abs(node.val -
  cur_max))
new_min = min(cur_min, node.val) <- update running min
  for descendants
new_max = max(cur_max, node.val) <- update running max
  for descendants
left = dfs(node.left, new_min, new_max)
right = dfs(node.right, new_min, new_max)
return max(diff, left, right)
```

PHASE 3 OPTIMIZE

WHY pass min AND max: the maximum difference |ancestor - descendant| is always

achieved by either the smallest ancestor vs largest descendant or vice versa.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1026_MaxDifferenceBetweenNodeAndAncestor:
    def maxAncestorDiff(self, root):
        def dfs(node, path_min, path_max):
            if not node:
                return path_max - path_min
            new_min = min(path_min, node.val)
            new_max = max(path_max, node.val)
            left = dfs(node.left, new_min, new_max)
            right = dfs(node.right, new_min, new_max)
            return max(left, right)
        return dfs(root, root.val, root.val)
```

PHASE 5 TEST & DEBUG

"Let me trace maxAncestorDiff on [8,3,10,1,6,null,14,null,null,4,7,null,null,13]."

DFS passing (min_so_far, max_so_far) down:

dfs(8, 8, 8): diff=0.

dfs(3, 3, 8): diff=|3-8|=5. ans=5.

dfs(1, 1, 8): diff=|1-8|=7. ans=7. leaves: no children.

dfs(6, 3, 8): diff=|6-8|=2<7.

dfs(4, 3, 8): |4-8|=4, |4-3|=1. max(4,1)=4. ans=7.

dfs(7, 3, 8): |7-3|=4. ans=7.

dfs(10, 8, 10): diff=|10-8|=2.

dfs(14, 8, 14): diff=|14-8|=6.

dfs(13, 8, 14): diff=|13-8|=5.

Final ans=7 ok (path 8->3->1: diff=7).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

Returning max-min at leaves is elegant — handles both directions of difference.

Follow-up: count pairs where difference > threshold — same DFS, different condition.

Follow-up: Path In Zigzag Labelled Binary Tree (#1104) — similar ancestor tracking.

MEDIUM | Trees & BST | LeetCode #1110 | 36 of 45 | 36/45

Delete Nodes And Return Forest

QUESTION

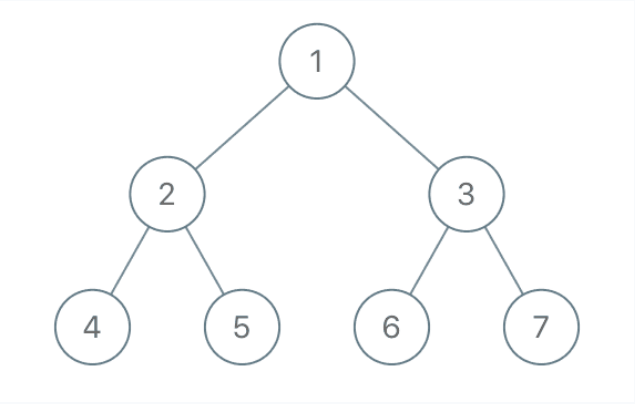
Given the root of a binary tree, each node in the tree has a distinct value.

After deleting all nodes with a value in to_delete, we are left with a forest (a disjoint union of trees).

Return the roots of the trees in the remaining forest. You may return the result in any order.

Example 1

Input	`root = [1,2,3,4,5,6,7], to_delete = [3,5]`
Output	`[[1,2,null,4],[6],[7]]`



Example 2

Input	`root = [1,2,4,null,3], to_delete = [3]`
Output	`[[1,2,4]]`

CONSTRAINTS

- The number of nodes in the given tree is at most 1000.
- Each node has a distinct value between 1 and 1000.
- to_delete.length <= 1000
- to_delete contains distinct values between 1 and 1000.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Delete nodes whose values are in 'to_delete'. Return roots of remaining forest. Right?"

"A few quick questions:

A node becomes a root if its parent was deleted. Can original root be deleted? Yes."

"Let me walk: root=[1,2,3,4,5,6,7], to_delete=[3,5] -> roots: [1,6,7,4] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for value in values:
    # to delete
    find its node, remove it and re-collect the
        roots of the forest from scratch
```

TIME	SPACE
O(n * k)	O(h)

WHY IT IS SLOW

Rebuilds the forest for every deletion. One post-order pass (put the deleted values in a set): after processing the children, a deleted node adds its non-deleted children to the answer and returns null to its parent: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order DFS — delete bottom-up. Already O(n). Walk-through:"

`dfs(node, parent_exists):`

If node is None -> return None.

`to_delete = node.val in delete_set`

If not to_delete and not parent_exists: roots.append(node)

<- this node starts a new tree root IF it's not being deleted AND has no surviving parent.

`node.left = dfs(node.left, not to_delete)`

`node.right = dfs(node.right, not to_delete)`

<- children's parent_exists = True only if THIS node survives (not being deleted).

<- if we're deleting this node, children have no parent -> they become new roots.

`return None if to_delete else node`

<- returning None disconnects this node from its parent.

POST-ORDER is critical: process children before deciding what to do with the current node.

Time: O(n). Space: O(n) delete set + O(h) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1110_DeleteNodesAndReturnForest:
    def delNodes(self, root, to_delete):
        delete_set = set(to_delete)
        result = []
        def dfs(node, is_root):
            if not node:
                return None
            is_deleted = node.val in delete_set
            if is_root and not is_deleted:
                result.append(node)
```

```
            node.left = dfs(node.left, is_deleted)
            node.right = dfs(node.right, is_deleted)
            return None if is_deleted else node
        dfs(root, True)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the tree recursion on the example, showing the return value at each node."

root=[1,2,3,4,5,6,7], to_delete=[3,5]:

dfs(4,False): not deleted, not root -> don't add. return node(4).

dfs(5,False): deleted. children: dfs(None,True)->None. return None.

dfs(2,False): not deleted. 2.left=4, 2.right=None(5 deleted). return node(2).

dfs(6,True): is_root=True(3 deleted). not deleted -> add to result. return node(6). ok

dfs(7,True): same -> add. ok

dfs(3,False): deleted. children get is_root=True. return None.

dfs(1,True): not deleted, is_root -> add. 1.left=2, 1.right=None. result=[1,6,7] + 4? Wait:

dfs(4,False) is_root=False (parent 2 is NOT deleted). So 4 is not added.

-> result=[1,6,7] ok (4 is connected under 1->2->4)

Is_root=True passed to children only when current node is deleted

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(d)** for delete_set + O(h) recursion

Follow-up: if we also need to return the deleted nodes, collect them in a separate list.

Follow-up: Delete Nodes and Return Forest II — delete ranges of values; same post-order.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #1120 | 37 of 45 | 37/45

Maximum Average Subtree

QUESTION

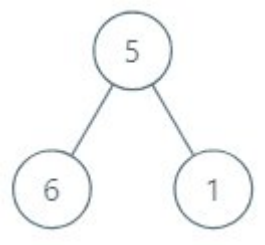
Given the root of a binary tree, return the maximum average value of a subtree of that tree. Answers within 10^{-5} of the actual answer will be accepted.

A subtree of a tree is any node of that tree plus all its descendants.

The average value of a tree is the sum of its values, divided by the number of nodes.

Example 1

Input	`root = [5,6,1]`
Output	`6.00000`
Explanation	*For the node with value = 5 we have an average of (5 + 6 + 1) / 3 = 4.* *For the node with value = 6 we have an average of 6 / 1 = 6.* *For the node with value = 1 we have an average of 1 / 1 = 1.* *So the answer is 6 which is the maximum.*



Example 2

Input	`root = [0,null,1]`
Output	`1.00000`

CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 0 <= Node.val <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the subtree (any node + all descendants) with the maximum average value. Return the average. Right?"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    run a separate DFS to total its subtree and
        count its nodes
    track the best average
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Recomputes each subtree total from scratch. A single post-order pass returning (sum, count) for each node and updating the best average on the way up is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order DFS is already O(n) — optimal. Walk-through of how answers propagate:"

dfs(leaf) returns (leaf.val, 1) and updates best if leaf.val/1 is highest so far.

dfs(node) gets (l_sum, l_cnt) from left and (r_sum, r_cnt) from right subtrees.

Combines: total_sum = l_sum + r_sum + node.val, total_cnt = l_cnt + r_cnt + 1.

Computes avg = total_sum / total_cnt and updates best[0] if better.

Returns (total_sum, total_cnt) upward so the parent can include this subtree in its calculation.

Post-order is critical: we need both children's aggregates before we can compute the parent's.

PHASE 4 CLEAN CODE

```
class _1120_MaxAvgSubtree:
    def maximumAverageSubtree(self, root: Optional['TreeNo
de']) -> float:
        best = [0.0]

        def dfs(node):
            if not node: return 0, 0
            l_sum, l_cnt = dfs(node.left)
            r_sum, r_cnt = dfs(node.right)
            total_sum = l_sum + r_sum + node.val
            total_cnt = l_cnt + r_cnt + 1
            best[0]   = max(best[0], total_sum / total_cnt
)
            return total_sum, total_cnt

        dfs(root)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me trace maximumAverageSubtree on [1,1,5] — find subtree of size>=2 with max avg."

dfs(1-left): count=1,sum=1. return (1,1).

dfs(5): count=1,sum=5. return (1,5).

dfs(1-root): sum=1+1+5=7, count=1+1+1=3. avg=7/3~2.33.

PHASE 5 TEST & DEBUG

Subtrees of size>=2: only root subtree here (size=3). ans=2.33.

Return 2.33 ok

[5,6,1]: dfs(6)=(1,6), dfs(1)=(1,1). dfs(5)=(3,12). avg=12/3=4.

Subtree at 5: size=3,avg=4. Check only size>=2. Ans=4.0. But [6] and [1] are size 1->skip.

WHY return (count,sum): parent computes its own count=1+left+right, sum=val+left+right.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** Post-order carries (sum, count) up — compute average at each node

Given more time I'd ask: minimum average subtree? Track min instead of max.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #1325 | 38 of 45 | 38/45

Delete Leaves With a Given Value

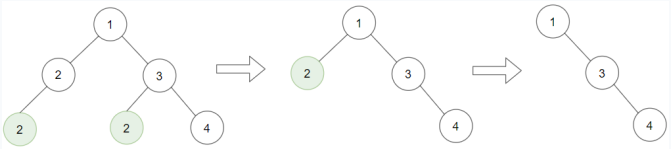
QUESTION

Given a binary tree root and an integer target, delete all the leaf nodes with value target.

Note that once you delete a leaf node with value target, if its parent node becomes a leaf node and has the value target, it should also be deleted (you need to continue doing that until you cannot).

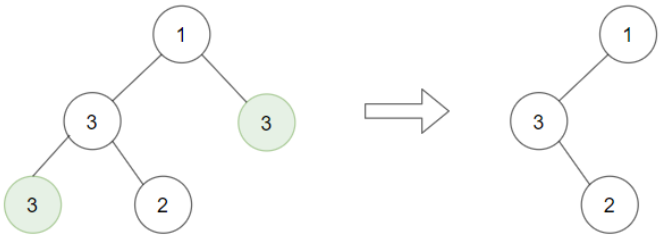
Example 1

Input	`root = [1,2,3,2,null,2,4], target = 2`
Output	`[1,null,3,null,4]`
Explanation	*Leaf nodes in green with value (target = 2) are removed (Picture in left). After removing, new nodes become leaf nodes with value (target = 2) (Picture in center).*



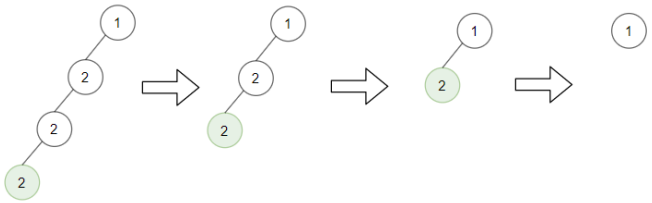
Example 2

Input	`root = [1,3,3,3,2], target = 3`
Output	`[1,3,null,null,2]`



Example 3

Input	`root = [1,2,null,2,null,2], target = 2`
Output	`[1]`
Explanation	*Leaf nodes in green with value (target = 2) are removed at each step.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 3000].

QUESTION

- 1 <= Node.val, target <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Repeatedly remove leaf nodes with value == target until no such leaves remain (removal can create new leaves).

Return the modified tree's root. Right?"

"A few quick questions:

If the root itself becomes a target leaf, it should also be removed (return null)? Yes."

"Let me walk: root=[1,2,3,2,null,2,4], target=2 -> removes both leaf 2s under node1, and the leaf 2 under node3 -> [1,null,3,null,4] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    delete every leaf whose value equals the
        target (one pass over the tree)
    if no such leaf remains:
        break
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

Each pass can only expose one new layer of leaves, so a path-like tree needs up to n passes. A single post-order recursion that deletes a node after its children if it became a target leaf is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order DFS — delete leaves bottom-up. O(n). Walk-through:"

`delete(node):`

If node is None -> return None.

`node.left = delete(node.left) <- recurse first (post-order)`

`node.right = delete(node.right)`

If node.val == target AND node.left is None AND node.right is None:

`return None <- this node is now a leaf with the target value -> delete it.`

`return node`

WHY post-order: after deleting children, a previously interior node might become

PHASE 3 OPTIMIZE

a leaf with the target value — it must also be deleted.

Example: [1,2,3,2,null,2,4], target=2: delete leaves 2 -> new leaves checked recursively.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1325_DeleteLeavesWithAGivenValue:
    def removeLeafNodes(self, root, target):
        if not root:
            return None
        root.left = self.removeLeafNodes(root.left, target
)
        root.right = self.removeLeafNodes(root.right, targ
et)
        if root.left is None and root.right is None and ro
ot.val == target:
            return None
        return root
```

PHASE 5 TEST & DEBUG

"Let me trace removeLeafNodes on [1,2,3,2,null,2,4], target=2."

Post-order DFS (process children first):

Node 2 (left-left-leaf): target=2, is leaf -> return None.

Node 2 (root.right.left-leaf): target=2 -> return None.

Node 3: left now None, right=4(leaf,!=2). Is 3 now a leaf AND val==target? 3!=2 -> keep 3.

Node 2 (root.left): left is now None (removed). right=None. Is leaf AND val==2 -> return None.

Node 4: leaf, !=2 -> keep.

Root 1: left=None, right=3(None,4). -> [1,None,3,None,4] ok

WHY post-order: must remove children first, then check if parent became a leaf.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

Follow-up: Binary Tree Pruning (#814) — same post-order deletion pattern, different removal condition.

Follow-up: if target is a list of values, just check membership instead of equality.

MEDIUM | Trees & BST | LeetCode #1376 | 39 of 45 | 39/45

Time Needed to Inform All Employees

QUESTION

A company has n employees with a unique ID for each employee from 0 to n - 1. The head of the company is the one with headID.

Each employee has one direct manager given in the manager array where manager[i] is the direct manager of the i-th employee, manager[headID] = -1. Also, it is guaranteed that the subordination relationships have a tree structure.

The head of the company wants to inform all the company employees of an urgent piece of news. He will inform his direct subordinates, and they will inform their subordinates, and so on until all employees know about the urgent news.

The i-th employee needs informTime[i] minutes to inform all of his direct subordinates (i.e., After informTime[i] minutes, all his direct subordinates can start spreading the news).

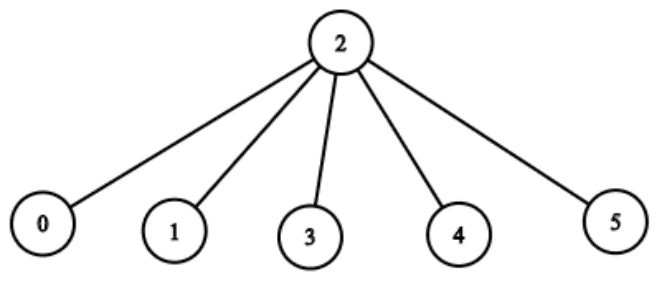
Return the number of minutes needed to inform all the employees about the urgent news.

Example 1

Input	`n = 1, headID = 0, manager = [-1], informTime = [0]`
Output	`0`
Explanation	*The head of the company is the only employee in the company.*

Example 2

Input	`n = 6, headID = 2, manager = [2,2,-1,2,2,2], informTime = [0,0,1,0,0,0]`
Output	`1`
Explanation	*The head of the company with id = 2 is the direct manager of all the employees in the company and needs 1 minute to inform them all.* *The tree structure of the employees in the company is shown.*



CONSTRAINTS

- 1 <= n <= 10^5
- 0 <= headID < n
- manager.length == n
- 0 <= manager[i] < n

QUESTION

- manager[headID] == -1
- informTime.length == n
- 0 <= informTime[i] <= 1000
- informTime[i] == 0 if employee i has no subordinates.
- It is guaranteed that all the employees can be informed.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A manager informs their direct subordinates, each taking informTime[i] minutes to inform their reports.

Find the total time to inform all employees. Right?"

"A few quick questions:

Headcount is the root? Yes. Can have multiple root candidates? No — exactly one head.

Return the maximum total time from root to any leaf? Yes."

"Let me walk: n=6, headID=2, manager=[-1,2,2,6,6,2,2], informTime=[0,0,4,0,0,3,0] -> 7 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for employee in employees:
    follow the manager chain up to the head,
        adding each manager's informTime
    best = max(best, total)
```

TIME	SPACE
O(n * h)	O(1)

WHY IT IS SLOW

Re-walks the chain for every employee, O(n^2) for a long chain. A DFS from the head carrying the accumulated time (or memoising each employee's total) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS/BFS from headID accumulating propagation time — already O(n). Walk-through:"

Build children tree: manager[i] = i's boss -> build children[boss] = [subordinates].

DFS from headID:

```
dfs(employee, time_so_far):
max_time = max(max_time, time_so_far)
for child in children[employee]:
dfs(child, time_so_far + informTime[employee])
```

<- time for child to receive info = time headID->employee + informTime[employee].

PHASE 3 OPTIMIZE

HOW propagation works: headID informs at time=0. Its subordinates receive info at

time=informTime[headID]. Their subordinates receive at time=informTime[headID]+informTime[subordinate].

We accumulate total delay along each path from root to leaf.

Answer = maximum accumulated time across all employees.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict
class _1376_TimeNeededToInformAllEmployees:
    def numOfMinutes(self, n, headID, manager, informTime)
:
        reports = defaultdict(list)
        for emp, mgr in enumerate(manager):
            if mgr != -1:
                reports[mgr].append(emp)
        def dfs(emp_id):
            max_sub_time = 0
            for sub_id in reports[emp_id]:
                max_sub_time = max(max_sub_time, dfs(sub_i
d))
            return informTime[emp_id] + max_sub_time
        return dfs(headID)
```

PHASE 5 TEST & DEBUG

"Let me trace numOfMinutes on informTime=[1,1,1,1,0,0,0] with chain structure."

DFS from headID:

`dfs(headID): total_time = informTime[headID] + max(dfs(sub) for sub in subordinates[headID]).`

Leaf employees: dfs(leaf) = informTime[leaf] + 0 = 0 (leaf has 0 inform time).

Chain: head->a->b->c (each inform time = t):

`dfs(c)=0. dfs(b)=t+0=t. dfs(a)=t+t=2t. dfs(head)=t+2t=3t.`

WHY max not sum: manager informs all subordinates in PARALLEL. Total = manager's time + max subordinate chain.

n=6,headID=2,manager=[-1,2,2,2,2,2],informTime=[0,0,0,0,0,0]: headID=0,chain of depth 1. All informed at once=0 ok

WHY tree details: return values from children drive decisions at the parent (post-order), so base cases must be correct.

DFS: dfs(headID,0). visited: each employee node once (tree, no cycles). call stack: dfs(head)->dfs(sub)->...->leaf (no subs). Max time propagates up via return value.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
visit each employee once	adjacency list + recursion

Follow-up: BFS alternative — same O(n) with a queue, avoids stack overflow for deep trees.

Follow-up: Employee Importance (#690) — sum instead of max.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #1382 | 40 of 45 | 40/45

Balance a Binary Search Tree

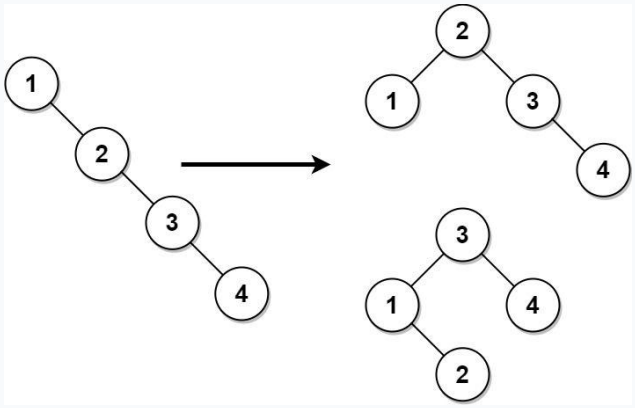
QUESTION

Given the root of a binary search tree, return a balanced binary search tree with the same node values. If there is more than one answer, return any of them.

A binary search tree is balanced if the depth of the two subtrees of every node never differs by more than 1.

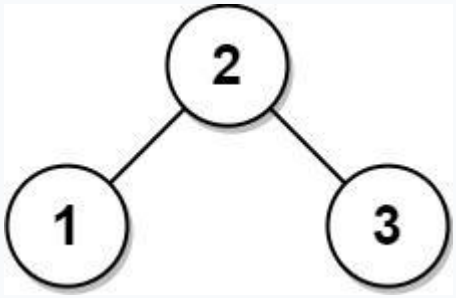
Example 1

Input	`root = [1,null,2,null,3,null,4,null,null]`
Output	`[2,1,3,null,null,null,4]`
Explanation	*This is not the only correct answer, [3,1,4,null,2] is also correct.*



Example 2

Input	`root = [2,1,3]`
Output	`[2,1,3]`



CONSTRAINTS

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $1 <= Node.val <= 10^5$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a BST that may be unbalanced, return it as a balanced BST with the same values. Right?"

"A few quick questions:

PHASE 1 CLARIFY

Any balanced BST result is valid? Yes. In-place? Yes."

"Let me walk: root=[1,null,2,null,3,null,4] (right-skewed) -> balanced: [2,1,3,null,null,null,4] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for shape in shapes:
    # on the values
    check whether it is a balanced BST
return the first
```

TIME	SPACE
O(n!)	O(n)

WHY IT IS SLOW

Enumerates shapes. In-order traversal gives the sorted values; build the tree from the middle element recursively (as in Convert Sorted Array to BST): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"In-order collect -> divide-and-conquer rebuild — already O(n). Walk-through:"

Step 1 — In-order traversal collects all BST values in sorted order.

[left subtree values ... root ... right subtree values] = sorted array.

Step 2 — Rebuild balanced BST from sorted array (same as #108):

```
build(lo, hi):
mid = (lo + hi) // 2
root = TreeNode(sorted_vals[mid])
root.left = build(lo, mid-1)
root.right = build(mid+1, hi)
return root
```

WHY this gives a balanced BST: picking the midpoint ensures both halves have

equal (±1) size at every recursive level.

WHY in-order first: rebuilding from BST in any other order might not preserve sorted property.

Time: O(n). Space: O(n) for sorted array + O(log n) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1382_BalanceBinarySearchTree:
    def balanceBST(self, root):
        sorted_vals = []
        def inorder(node):
```

```
            if node:
                inorder(node.left)
                sorted_vals.append(node.val)
                inorder(node.right)
        inorder(root)
        def build_balanced(lo, hi):
            if lo > hi:
                return None
            mid = (lo + hi) // 2
            node = TreeNode(sorted_vals[mid])
            node.left = build_balanced(lo, mid - 1)
            node.right = build_balanced(mid + 1, hi)
            return node
        return build_balanced(0, len(sorted_vals) - 1)
```

PHASE 5 TEST & DEBUG

"Let me trace balanceBST on [1,null,2,null,3,null,4]."

In-order traversal: [1, 2, 3, 4].

Build balanced BST from sorted array [1,2,3,4]:

mid=2(idx=1)->root=2. Left=[1]:mid=1->root=1. Right=[3,4]:mid=3->root=3,right=4.

Tree: 2(1, 3(None,4)) ok Height=2, balanced.

WHY in-order first: extracts sorted array from any BST in O(n).

WHY mid as root: puts equal elements on each side -> height-balanced.

Same logic as Q108 (Convert Sorted Array to BST).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** sorted values array + recursion stack

Follow-up: if tree must be rebuilt in-place (no extra array), use Day-Stout-Warren algorithm.

Follow-up: Convert Sorted Array to BST (#108) — same build_balanced function.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #1448 | 41 of 45 | 41/45

Count Good Nodes in Binary Tree

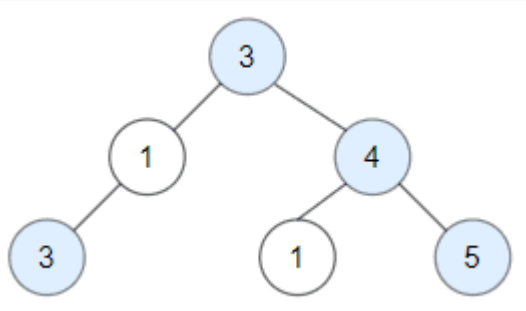
QUESTION

Given a binary tree root, a node X in the tree is named good if in the path from root to X there are no nodes with a value greater than X.

Return the number of good nodes in the binary tree.

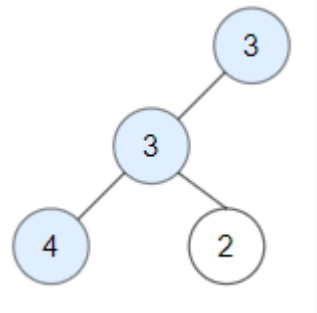
Example 1

Input	`root = [3,1,4,3,null,1,5]`
Output	`4`
Explanation	*Nodes in blue are good . Root Node (3) is always a good node. Node 4 -> (3,4) is the maximum value in the path starting from the root. Node 5 -> (3,4,5) is the maximum value in the path Node 3 -> (3,1,3) is the maximum value in the path.*



Example 2

Input	`root = [3,3,null,4,2]`
Output	`3`
Explanation	*Node 2 -> (3, 3, 2) is not good, because "3" is higher than it.*



QUESTION

Example 3

Input	`root = [1]`
Output	`1`
Explanation	*Root is considered as good .*

CONSTRAINTS

- The number of nodes in the binary tree is in the range [1, 10^5].
- Each node's value is between [-10^4, 10^4].

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A node is good if its value is >= the maximum value on the path from root to it.

Count good nodes. Root is always good. Right?"

"A few quick questions:

Path from root means ancestors only? Yes. Include the node itself in the comparison? Yes."

"Let me walk: root=[3,1,4,3,null,1,5] -> good nodes: 3(root),4,3,5 -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    walk its path back to the root and check
        whether any ancestor is greater
    count the nodes with none
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

Re-walks the path for every node. Pass the maximum value seen so far down the recursion; a node is good if it is at least that maximum: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Pre-order DFS passing max_so_far — already O(n). Walk-through:"

`dfs(node, max_so_far):`

If node is None -> return 0.

`good = 1 if node.val >= max_so_far else 0 <- is this node 'good'?`

PHASE 3 OPTIMIZE

```
new_max = max(max_so_far, node.val) <- update max for
  descendants
left = dfs(node.left, new_max)
right = dfs(node.right, new_max)
return good + left + right
```

HOW max propagates: we pass the maximum value seen on the root-to-current path.

A node is 'good' if its value >= that max — meaning no ancestor is larger.

PRE-ORDER: we use max_so_far BEFORE recursing into children, then update it for them.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1448_CountGoodNodes:
    def goodNodes(self, root):
        def dfs(node, max_on_path):
            if not node:
                return 0
            is_good = 1 if node.val >= max_on_path else 0
            new_max = max(max_on_path, node.val)
            return is_good + dfs(node.left, new_max) + dfs
(node.right, new_max)
        return dfs(root, float('-inf'))
```

PHASE 5 TEST & DEBUG

"Let me trace goodNodes on [3,1,4,3,null,1,5]."

DFS passing max_so_far:

```
dfs(3, max=-inf): 3>=-inf -> good! count=1. max
  becomes 3.
dfs(1, max=3): 1<3 -> not good. count=1.
dfs(3, max=3): 3>=3 -> good! count=2. No children.
dfs(4, max=3): 4>=3 -> good! count=3. max=4.
dfs(1, max=4): 1<4 -> not good. count=3.
dfs(5, max=4): 5>=4 -> good! count=4.
```

Return 4 ok

WHY pass max down: a node is 'good' if no ancestor has a strictly greater value.

WHY DFS not BFS: need to track the path-maximum from root to each node.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** for call stack

Follow-up: count BAD nodes — flip the >= to <.

Follow-up: Path With Maximum Minimum Value (#1102) — track minimum on path instead of max.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #1490 | 42 of 45 | 42/45

Clone N-ary Tree

QUESTION

Given a root of an N-ary tree, return a deep copy (clone) of the tree.

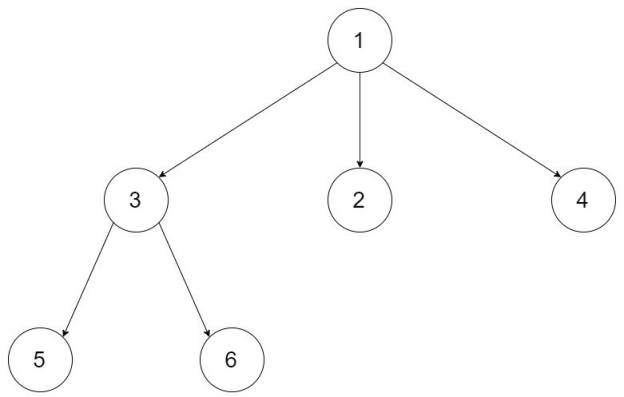
Each node in the n-ary tree contains a val (int) and a list (List[Node]) of its children.

```
class Node {
    public int val;
    public List<Node> children;
}
```

Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

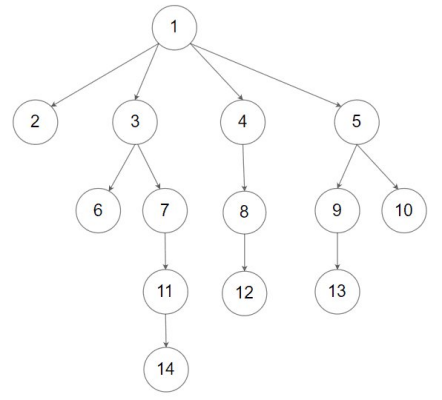
Example 1

Input	`root = [1,null,3,2,4,null,5,6]`
Output	`[1,null,3,2,4,null,5,6]`



Example 2

Input	`root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]`
Output	`[1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]`



CONSTRAINTS

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [0, 10^4].
- Follow up: Can your solution work for the graph problem?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Deep clone an N-ary tree. No shared references between original and clone. Right?"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def clone(node):
    create a new node with the same value
    for child in children:
        add clone(child) to the new node's children
```

TIME	SPACE
O(n)	O(h)

NOTE

Not slow: every node is copied once, which is optimal. The cost is the recursion stack (O(n) for a path); a BFS with a queue gives the same bound.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"The recursive DFS clone IS already O(n) — optimal for this problem."

How the recursion generates the full clone:

`cloneTree(root): create Node(root.val) with children = [cloneTree(c) for c in root.children].`

Each recursive call clones one node and RECURSIVELY clones ALL of its children.

The list comprehension processes children left-to-right, so the clone structure mirrors the original.

No visited map needed — trees have no cycles, so we can't revisit nodes.

Contrast with Clone Graph (#133) which needs a visited dict to break cycles.

PHASE 4 CLEAN CODE

```
class _1490_CloneNaryTree:
    def cloneTree(self, root: 'Node') -> 'Node':
        if not root:
            return None
        return Node(root.val, [self.cloneTree(child) for child in root.children])
```

PHASE 5 TEST & DEBUG

"Let me trace cloneTree on N-ary tree 1(3(5,6),2,4)."

DFS (pre-order):

```
clone(1): create Node(1). For each child [3,2,4]:
  clone each.
clone(3): create Node(3). For each child [5,6]:
  clone(5)->Node(5), clone(6)->Node(6).
```

Node(3).children=[Node(5),Node(6)]. return Node(3).

```
clone(2)->Node(2). clone(4)->Node(4).
```

Node(1).children=[Node(3,children=[5,6]),Node(2),Node(4)]. return Node(1).

Cloned tree is structurally identical, all new objects ok

WHY recursion: N-ary tree has variable children — recursion handles any branching factor.

WHY create new nodes: deep copy — no shared references with original tree.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Simple recursive clone — no visited map needed (trees have no cycles)

Follow-up (Clone Graph #133): needs visited map to handle cycles — {original: clone}.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #1522 | 43 of 45 | 43/45

Diameter of N-Ary Tree

QUESTION

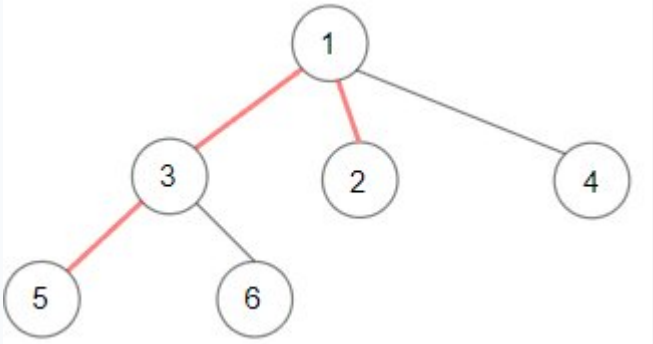
Given a root of an N-ary tree, you need to compute the length of the diameter of the tree.

The diameter of an N-ary tree is the length of the longest path between any two nodes in the tree. This path may or may not pass through the root.

(Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value.)

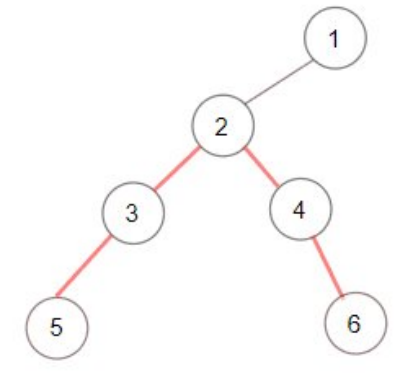
Example 1

Input	`root = [1,null,3,2,4,null,5,6]`
Output	3
Explanation	*Diameter is shown in red color.*



Example 2

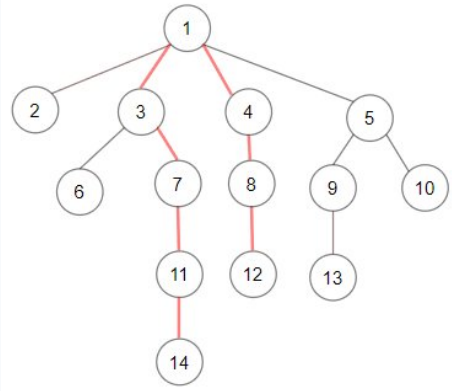
Input	`root = [1,null,2,null,3,4,null,5,null,6]`
Output	4



Example 3

Input	`root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]`
Output	7

QUESTION



CONSTRAINTS

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [1, 10^4].

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Diameter = longest path between any two nodes (in edges), generalised to N children. Right?

Same concept as Binary Tree Diameter (#543)."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    find the two deepest chains below it with a
        fresh DFS
    diameter = the largest sum of the two deepest
        child depths
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Recomputes depths below every node. One post-order pass that returns each node's height and updates the answer with its two tallest children is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order DFS tracking top-2 child heights — already O(n). Walk-through:"

`dfs(node):`

For each child c of node:

`child_height = dfs(c) + 1 <- add 1 for the edge from this node to child c`

Update top two child heights (top1, top2).

PHASE 3 OPTIMIZE

Update global diameter: answer = max(answer, top1 + top2)

<- diameter through this node = longest path to one child + longest path to another.

return top1 <- this node's height = longest path to any descendant through its best child.

WHY track TOP TWO: the diameter through a node uses its two LONGEST child paths.

Adding any third would not be a simple path (a path can't branch).

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

child_h+1 converts "height below child" to "edges from this node via that child

```
class _1522_DiameterNaryTree:
    def diameter(self, root: 'Node') -> int:
        best = [0]

        def height(node) -> int:
            if not node or not node.children:
                return 0
            child_heights = sorted([height(c) + 1 for c in
 node.children], reverse=True)
            if len(child_heights) >= 2:
                best[0] = max(best[0], child_heights[0] +
child_heights[1])
            else:
                best[0] = max(best[0], child_heights[0])
            return child_heights[0]

        height(root)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me trace diameter of N-ary tree 1(3(5,6),2,4)."

dfs(5): depth=1,no children. return 1.

dfs(6): depth=1. return 1.

dfs(3): top2_depths=[1,1]. diameter_candidate=1+1=2. ans=2. return 1+1=2.

dfs(2): return 1. dfs(4): return 1.

dfs(1): depths=[2,1,1]. top2=[2,1]. candidate=2+1=3. ans=3. return 1+2=3.

Answer: 3 ok (path 5->3->1->2 or 6->3->1->2, length 3 edges).

WHY track top 2 child depths: diameter through a node = sum of two longest child-paths.

N-ary version of binary tree diameter (Q543).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** Adding +1 when collecting child heights is the key difference from #543

In binary tree, depth(child) already includes the edge up; here we add it explicitly.

PHASE 6 COMPLEXITY & FOLLOW-UP

Given more time I'd ask: diameter in edge-weighted N-ary tree?

Same approach — just use edge weights instead of +1.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #1650 | 44 of 45 | 44/45

Lowest Common Ancestor of a Binary Tree III

QUESTION

Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA).

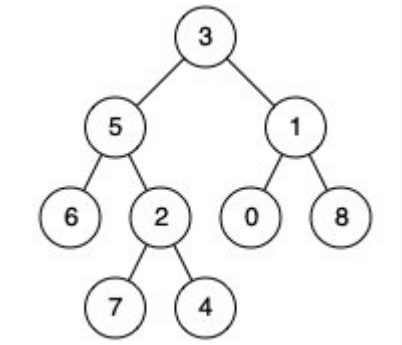
Each node will have a reference to its parent node. The definition for Node is below:

```
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node parent;
}
```

According to the definition of LCA on Wikipedia: "The lowest common ancestor of two nodes p and q in a tree T is the lowest node that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Example 1

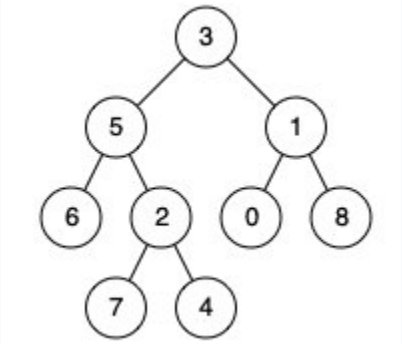
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1`
Output	`3`
Explanation	*The LCA of nodes 5 and 1 is 3.*



Example 2

Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4`
Output	`5`
Explanation	*The LCA of nodes 5 and 4 is 5 since a node can be a descendant of itself according to the LCA definition.*

QUESTION



Example 3

Input	`root = [1,2], p = 1, q = 2`
Output	`1`

CONSTRAINTS

- The number of nodes in the tree is in the range [2, 10^5].
- 10^9 <= Node.val <= 10^9
- All Node.val are unique.
- p != q
- p and q exist in the tree.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Nodes have a parent pointer. Find LCA (Lowest Common Ancestor) without the root. Right?

Both p and q are guaranteed to be in the tree."

"[3,5,1,...], p=5, q=1 -> LCA=3 ok (same as #236 but solved via parent pointers)"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
ancestors = an empty set
walk from p up to the root adding every node
    (including p)
walk from q upwards: the first node already in the
    set is the answer
```

TIME	SPACE
O(h)	O(h)

WHY IT IS SLOW

Stores every ancestor of p. Two pointers, a and b: advance each to its parent, and when one reaches the root jump to the other's start. They meet at the lowest common ancestor in O(h) time and O(1) space.

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (two-pointer linked list intersection)

"Two-pointer: equalize depths then move up together — O(h). Walk-through:"

Find depths of p and q. Bring deeper node up to same depth.

Then advance both until they meet (LCA found).

`depth(node): count parent steps to root.`

Equalize: while depth(p) > depth(q): p=p.parent. while depth(q) > depth(p): q=q.parent.

Advance together: while p != q: p=p.parent; q=q.parent.

Return p.

WHY equalize first: at the same depth, both pointers are equidistant from their LCA.

WHY parent pointer available: Q1650 has a parent pointer (unlike standard LCA).

Alternative: collect ancestor sets for both nodes; find first common ancestor.

Time: O(h). Space: O(1).

PHASE 4 CLEAN CODE

```
class _1650_LCA_III:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -
> 'Node':
        a, b = p, q
        while a is not b:
            a = a.parent if a else q
            b = b.parent if b else p
        return a
```

PHASE 5 TEST & DEBUG

"Let me trace LCA with parent pointers — equalize depths then march together."

p=5 (depth=2: 5->3->root), q=1 (depth=2: 1->3->root). Same depth.

March: p=5, q=1. 5!=1. p=5.parent=3, q=1.parent=3. 3==3 -> return 3 ok

p=6 (depth=3), q=1 (depth=2). Equalize: p goes up once -> p=6.parent=5 (depth 2 now).

March: p=5, q=1. 5!=1. p=3, q=3. 3==3 -> return 3 ok

WHY equalize first: at different depths, marching together would overshoot the LCA.

Once both at same depth, each upward step is the same 'distance to LCA' for both.

WHY use parent pointers: allows upward traversal without full tree structure.

WHY tree details: return values from children drive decisions at the parent (post-order), so base cases must be correct.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(h)	**O(1)** This is the two-pointer intersection trick from Linked List Intersection (#160)

Both pointers traverse depth(p) + depth(q) steps total and meet at LCA.

Given more time I'd ask: what if the tree is very deep and we want to avoid the full traversal?

Use a set: walk p to root recording all ancestors, then walk q to root and stop at first ancestor in the set.

O(depth) time, O(depth) space — same asymptotic but possibly faster in practice.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trees & BST | LeetCode #2096 | 45 of 45 | 45/45

Step-By-Step Directions From a Binary Tree Node to Another

QUESTION

You are given the root of a binary tree with n nodes. Each node is uniquely assigned a value from 1 to n. You are also given an integer startValue representing the value of the start node s, and a different integer destValue representing the value of the destination node t.

Find the shortest path starting from node s and ending at node t. Generate step-by-step directions of such path as a string consisting of only the uppercase letters 'L', 'R', and 'U'. Each letter indicates a specific direction:

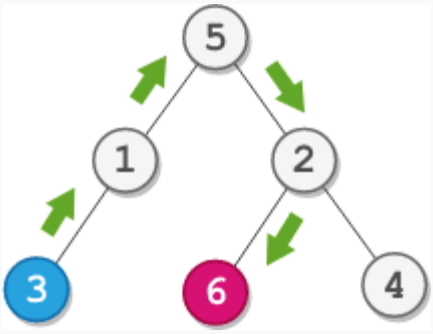
CONSTRAINTS

- 'L' means to go from a node to its left child node.
- 'R' means to go from a node to its right child node.
- 'U' means to go from a node to its parent node.

Return the step-by-step directions of the shortest path from node s to node t.

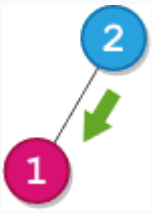
Example 1

Input	`root = [5,1,2,3,null,6,4], startValue = 3, destValue = 6`
Output	`"UURL"`
Explanation	*The shortest path is: 3 -> 1 -> 5 -> 2 -> 6.*



Example 2

Input	`root = [2,1], startValue = 2, destValue = 1`
Output	`"L"`
Explanation	*The shortest path is: 2 -> 1.*



- The number of nodes in the tree is n.
- $2 <= n <= 10^5$
- 1 <= Node.val <= n

QUESTION

- All the values in the tree are unique.
- 1 <= startValue, destValue <= n
- startValue != destValue

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the step-by-step directions from node startValue to node destValue in a binary tree.

Use 'L' (go left), 'R' (go right), 'U' (go to parent). Right?"

"A few quick questions:

Both nodes guaranteed to exist? Yes. Path may go up then down? Yes."

"Let me walk: root=[5,1,2,3,null,6,4], start=3, dest=6 -> 'UURL' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
build the path from the root to the start and the
    path from the root to the destination (as strings
    of L and R)
strip the common prefix
output 'U' for each remaining step of the first path
    then the second path
```

TIME	SPACE
O(n)	O(h)

NOTE

Not slow: both paths are found in one traversal each. Finding the lowest common ancestor first and measuring from there gives the same bound.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"LCA (Lowest Common Ancestor) + path building: LCA->src = 'U's, LCA->dest = 'L'/'R' turns — O(n). Walk-through:"

Find LCA of startValue and destValue.

Path from LCA to startValue: follow left/right pointers -> record as 'U's (going up from start to LCA).

Path from LCA to destValue: record actual 'L'/'R' directions.

Result = 'U'*len(path_to_start) + path_LCA_to_dest.

HOW to find LCA: post-order DFS checking if node contains startValue or destValue in subtree.

WHY convert start path to 'U's: from startValue to LCA, we go UP the tree.

Direction L/R is only meaningful going DOWN (from LCA to dest).

PHASE 3 OPTIMIZE

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2096_StepByStepDirections:
    def getDirections(self, root, startValue, destValue):
        def find_path(node, target, path):
            if not node:
                return False
            if node.val == target:
                return True
            for child, direction in [(node.left, 'L'), (no
de.right, 'R')]:
                path.append(direction)
                if find_path(child, target, path):
                    return True
                path.pop()
            return False
        path_to_start, path_to_dest = [], []
        find_path(root, startValue, path_to_start)
        find_path(root, destValue, path_to_dest)
        lca_len = 0
        while (lca_len < len(path_to_start) and lca_len <
len(path_to_dest)
               and path_to_start[lca_len] == path_to_dest[
lca_len]):
            lca_len += 1
        ups = 'U' * (len(path_to_start) - lca_len)
        downs = ''.join(path_to_dest[lca_len:])
        return ups + downs
```

PHASE 5 TEST & DEBUG

"Let me trace getDirections on tree [5,1,2,3,null,6,4], startValue=3, destValue=6."

Find LCA of 3 and 6: dfs finds LCA=5 (root). Path 3->LCA=5: 3 is at 5.left.left -> 2 steps up.

Path from LCA to 3: L,L (root->1->3). Convert to 'U's: 'UU'.

Path from LCA to 6: root->2->6 = 'LR' (left at 5, right at 2).

Result: 'UU' + 'LR' = 'UULR' ok

startValue=3, destValue=6: [go up 2 to reach LCA=root, go down L then R]

WHY UUs for start path: from start node, every step toward LCA is 'Up'.

WHY L/R for dest path: going down the tree from LCA, record actual left/right turns.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
two DFS scans	for paths

Follow-up: if tree is very deep, iterative DFS avoids stack overflow.

Follow-up: LCA (#236) + two shorter paths — same approach, find LCA first for clarity.